In [1]:
```
# data sets
from tdc.utils import get_reaction_type

# rdkit things
from rdkit import Chem
from rdkit.Chem import GetPeriodicTable
from rdkit.Chem import rdChemReactions
from rdkit.Chem import Draw
from rdkit.Chem.Draw import ReactionToImage
from rdkit.Chem.Draw import MolToImage
from rdkit.Chem import AllChem
from rdkit import RDLogger
RDLogger.DisableLog("rdApp.*")

# standard libraries
import glob
import os
import math
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import time
import copy
from tqdm import tqdm

# machine learning libraries
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.nn.utils.rnn import pad_sequence
from torch_geometric.nn import GINEConv, global_mean_pool
from torch_geometric.data import Data
from torch_geometric.data import Dataset, DataLoader # for the gnn
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold

# used to explore hyperparameter space
from sklearn.model_selection import RandomizedSearchCV # took way too long
from sklearn.experimental import enable_halving_search_cv
from sklearn.model_selection import HalvingRandomSearchCV
from sklearn.base import BaseEstimator

# data reduction
from scipy.stats import uniform, loguniform, randint
from sklearn.decomposition import PCA
```

(1) Retrieve the cleaned data from data_exploration.ipynb to use in our model.

Structure it to use in model.

In [2]:
```
alchemy_elements = {'C', 'Cl', 'F', 'H', 'N', 'O', 'S'}

molecule_types_df = pd.read_pickle(
   "/Users/joshuahoang/Documents/Chem-C242-Final-Project-Organic-Reactions/C
)

reaction_types_df = pd.read_csv(
    "/Users/joshuahoang/Documents/Chem-C242-Final-Project-Organic-Reactions/
)

display(molecule_types_df)
display(reaction_types_df)
```

	gdb_idx	atom number	zpve\n(Ha, zero point vibrational energy)	Cv\n(cal/molK, heat capacity at 298.15 K)	gap\n(Ha, LUMO-HOMO)	G\n(Ha, Free energy at 298.15 K)	HOMO\ ene H
0	2859833	9	0.226164	0.000067	0.300531	-352.128828	-0.2
1	3148292	9	0.180782	0.000058	0.293098	-386.820766	-0.2
2	3607838	9	0.205922	0.000057	0.307544	-388.062328	-0.2
3	9540153	11	0.160845	0.000056	0.195923	-515.046508	-0.2
4	340363	10	0.177712	0.000057	0.254495	-403.843164	-0.2
...	...	...	...	...	...	...	
202574	9581016	10	0.120222	0.000051	0.179768	-450.915828	-0.2
202575	9706527	10	0.130179	0.000053	0.189401	-438.632624	-0.2
202576	9760179	10	0.119919	0.000051	0.163827	-450.920199	-0.2

	gdb_idx	atom number	zpve\n(Ha, zero point vibrational energy)	Cv\n(cal/molK, heat capacity at 298.15 K)	gap\n(Ha, LUMO-HOMO)	G\n(Ha, Free energy at 298.15 K)	HOMO\ ene H
202577	9845175	10	0.119740	0.000050	0.191354	-454.722866	-0.2
202578	9880714	10	0.142321	0.000056	0.174621	-418.749178	-0.2(

202579 rows × 17 columns

	reactant	
0	C1=COCCC1.COC(=O)CCC(=O)c1ccc(O)cc1O	COC(=O)CCC(=O)c1ccc(OC2C(
1	COC(=O)c1cccc(C(=O)O)c1.Nc1cccnc1N	COC(=O)c1cccc(-c2nc3ccc
2	CC(C)(C)OC(=O)NC1CCC(C(=O)O)CC1.CNOC	CON(C)C(=O)C1CCC(N
3	Nc1ccc(O)cc1.O=[N+]([O-])c1ccc(Cl)nc1Cl	O=[N+]([O-])c1ccc(Cl)nc1N
4	[N-]=[N+]=NCC1=CC[C@@H](c2ccc(Cl)cc2Cl)[C@H]([...	NCC1=CC[C@@H](c2ccc(Cl)(([N
...	...	
35111	COC(=O)c1ccc(C(=O)OCc2ccccc2)cc1F	COC(=O)c1ccc(
35112	C=CCOC(=O)N1CCc2nnc(NN)cc2C1.CC(C)=O	C=CCOC(=O)N1CCc2nnc(NN=(
35113	FC1(F)CCNC1.O=C(O)c1ccncc1NC(=O)c1nc(C2CC2)ccc...	O=C(Nc1cnccc1C(= (F)C1)c1nc((
35114	CC(=O)Cl.CC(C)(C)OC(=O)NCCO	CC(=O)OCCNC(=(
35115	CC(C)(C)O.O=CC1=C[C@H]2CC(O)O[C@H]2C1	(C)OC1C[C@@H]2C=C(C=O)C

35116 rows × 3 columns

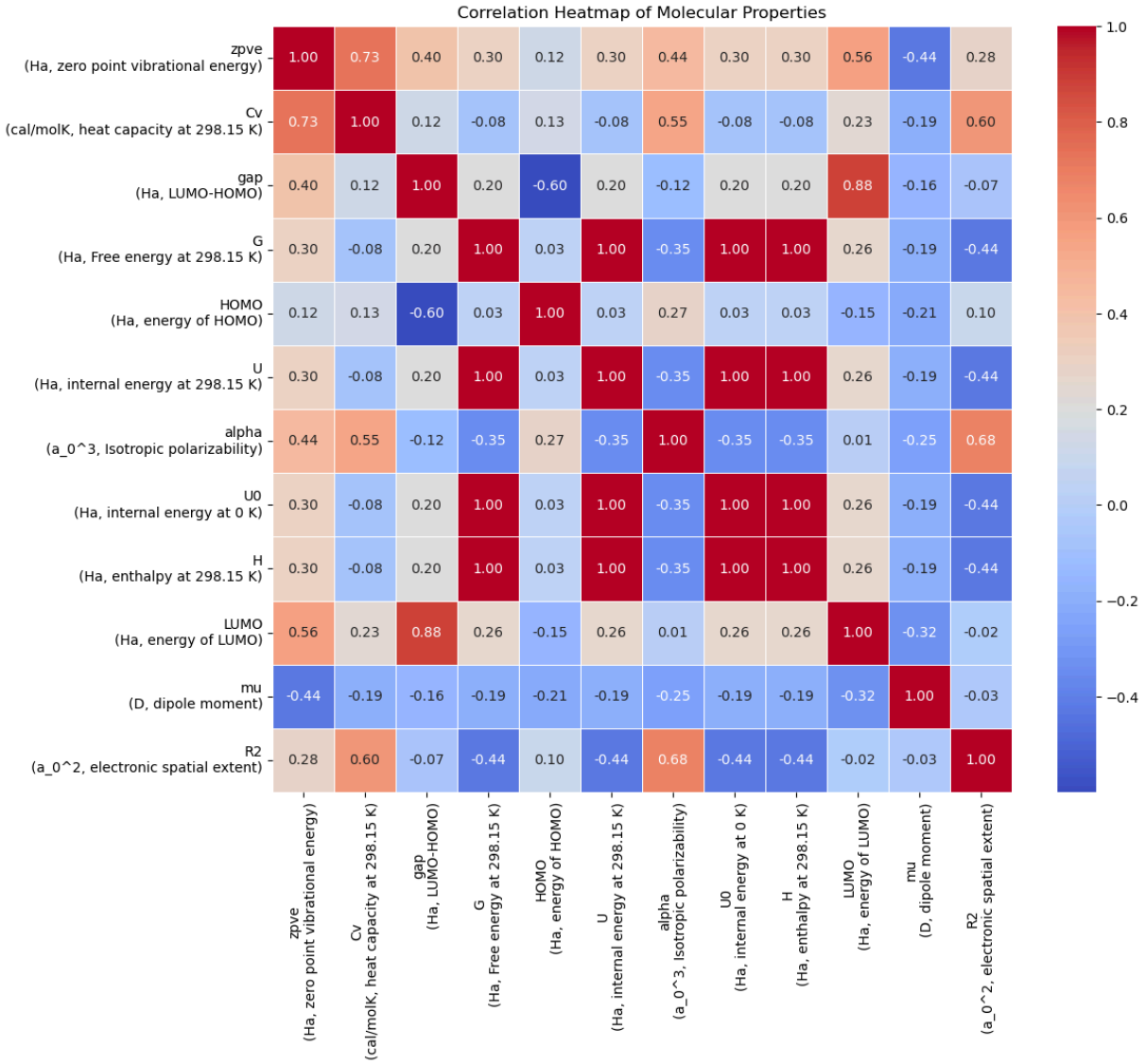
Pipeline

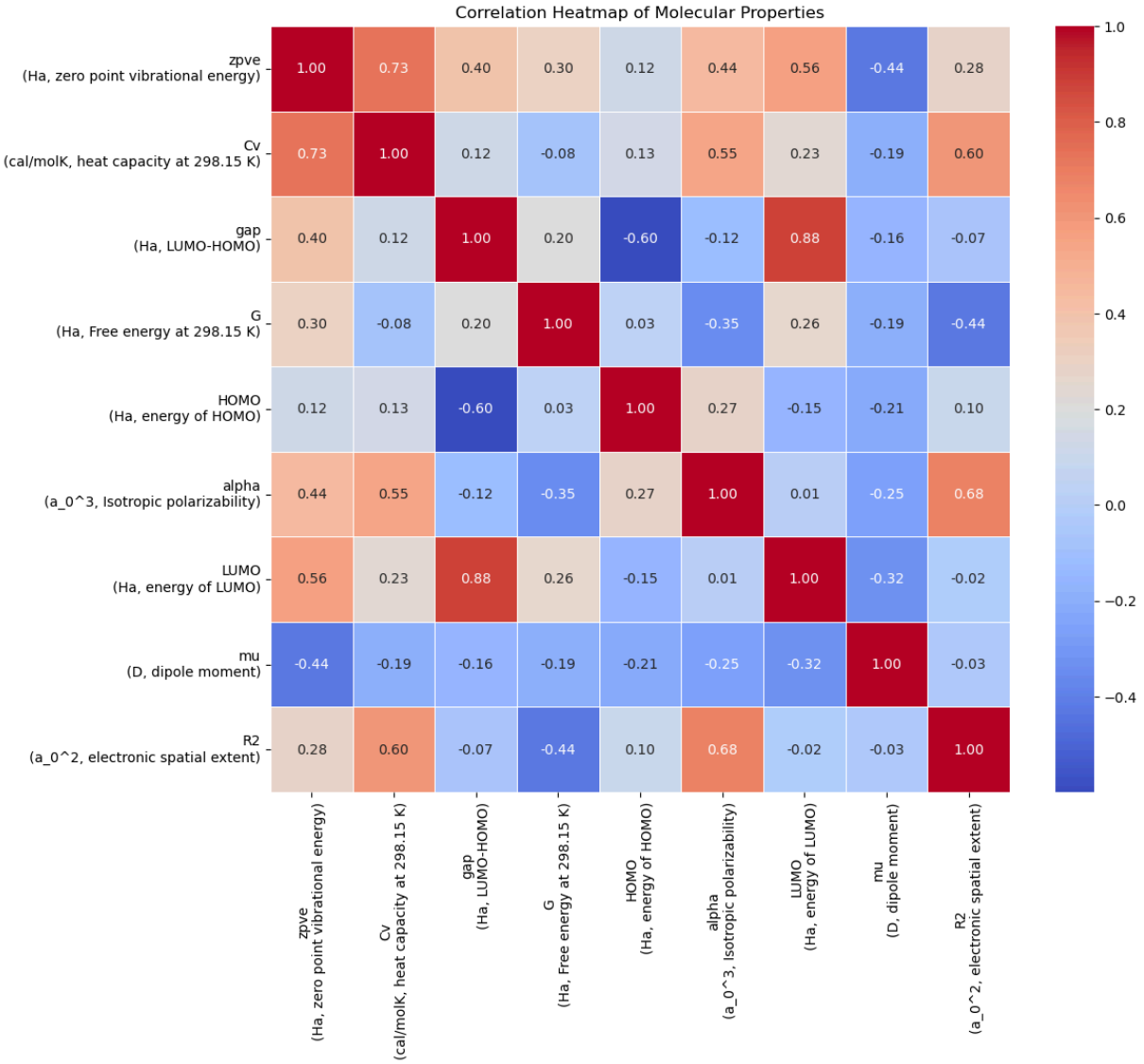
(a) The GNN will take each molecule as a pooling of the atoms as nodes and the bonds as edges and lets each atom update its features by aggreating from its neighbors.

(b) After this messaging, pool atoms to molecule to predict the target features that will be used in the decision tree for the reaction type.

```
 - 12 columns are physics-relevent feature columns
 - 9/12 of the physics-relevent ones do not have too bad
overlap
```

In [3]:
```
def plot_heatmap(df):
    plt.figure(figsize=(12,10))
    sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt=".2f", linewidth
    plt.title("Correlation Heatmap of Molecular Properties")
    plt.show()

# 12 that are physics-relevent target features
molecule_targets_df = molecule_types_df.drop(columns=[
    'smiles',
    'gdb_idx',
    'atom number',
    'coords',
    'node_angles'
    ])
plot_heatmap(molecule_targets_df)

# G, U, U0, H: G = U + PV - TS = H - TS, so we only need 1 -> keep G
molecule_targets_df = molecule_types_df.drop(columns=[
    'smiles',
    'gdb_idx',
    'atom number',
    'coords',
    'node_angles',
    'U\n(Ha, internal energy at 298.15 K)',
    'U0\n(Ha, internal energy at 0 K)',
    'H\n(Ha, enthalpy at 298.15 K)',
])
plot_heatmap(molecule_targets_df)
```


Correlation Heatmap of Molecular Properties
zpve (Ha, zero point vibrational energy)
Cv (cal/molK, heat capacity at 298.15 K)
gap (Ha, LUMO-HOMO)
G (Ha, Free energy at 298.15 K)
HOMO (Ha, energy of HOMO)
U (Ha, internal energy at 298.15 K)
alpha (a_0^3, Isotropic polarizability)
U0 (Ha, internal energy at 0 K)
H (Ha, enthalpy at 298.15 K)
LUMO (Ha, energy of LUMO)
mu (D, dipole moment)
R2 (a_0^2, electronic spatial extent)
1.00 0.73 0.40 0.30 0.12 0.30 0.44 0.30 0.30 0.56 -0.44 0.28
0.73 1.00 0.12 -0.08 0.13 -0.08 0.55 -0.08 -0.08 0.23 -0.19 0.60
0.40 0.12 1.00 0.20 -0.60 0.20 -0.12 0.20 0.20 0.88 -0.16 -0.07
0.30 -0.08 0.20 1.00 0.03 1.00 -0.35 1.00 1.00 0.26 -0.19 -0.44
0.12 0.13 -0.60 0.03 1.00 0.03 0.27 0.03 0.03 -0.15 -0.21 0.10
0.30 -0.08 0.20 1.00 0.03 1.00 -0.35 1.00 1.00 0.26 -0.19 -0.44
0.44 0.55 -0.12 -0.35 0.27 -0.35 1.00 -0.35 -0.35 0.01 -0.25 0.68
0.30 -0.08 0.20 1.00 0.03 1.00 -0.35 1.00 1.00 0.26 -0.19 -0.44
0.30 -0.08 0.20 1.00 0.03 1.00 -0.35 1.00 1.00 0.26 -0.19 -0.44
0.56 0.23 0.88 0.26 -0.15 0.26 0.01 0.26 0.26 1.00 -0.32 -0.02
-0.44 -0.19 -0.16 -0.19 -0.21 -0.19 -0.25 -0.19 -0.19 -0.32 1.00 -0.03
0.28 0.60 -0.07 -0.44 0.10 -0.44 0.68 -0.44 -0.44 -0.02 -0.03 1.00
zpve (Ha, zero point vibrational energy)
Cv (cal/molK, heat capacity at 298.15 K)
gap (Ha, LUMO-HOMO)
G (Ha, Free energy at 298.15 K)
HOMO (Ha, energy of HOMO)
U (Ha, internal energy at 298.15 K)
alpha (a_0^3, Isotropic polarizability)
U0 (Ha, internal energy at 0 K)
H (Ha, enthalpy at 298.15 K)
LUMO (Ha, energy of LUMO)
mu (D, dipole moment)
R2 (a_0^2, electronic spatial extent)
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4



(2) Focus on making the GNN with the alchemy DFT data.

Turn all the data into a graph structure. We choose the features that only the organic chemists care for which is geometrty, resonance, if it is in a ring, etc. Aromaticity is already in bond order.

In [4]:
```
"""
NOTE:
Using one-hot encoding instead of raw integer values for categorical feature
hybridization, etc.) prevents model from assuming a false numeric ordering (
Previously, and stupidly, values like 6, 7, 8 implicitly suggested linear re
Each category is treated as distinct and orthogonal.
"""

# discrete categories for features in node
alchemy_elements = list(alchemy_elements)
degree_set = [0, 1, 2, 3, 4, 5]
numH_set = [0, 1, 2, 3, 4]
charge_set = [-1, 0, 1]
hybridization_set = [
```

```
    Chem.rdchem.HybridizationType.SP,
    Chem.rdchem.HybridizationType.SP2,
    Chem.rdchem.HybridizationType.SP3,
    Chem.rdchem.HybridizationType.SP3D,
    Chem.rdchem.HybridizationType.SP3D2,
]

# helper functions
def one_hot(value, choices):
    return [int(value == c) for c in choices]

def compute_angle(a, b, c, pos):
    ba = pos[a] - pos[b]
    bc = pos[c] - pos[b]

    cos_theta = torch.dot(ba, bc) / (torch.norm(ba) * torch.norm(bc) + 1e-8)
    cos_theta = torch.clamp(cos_theta, -1.0, 1.0)

    angle = torch.acos(cos_theta).item()  # radians
    return angle

def padded_angle_vector(atom_idx, mol, pos, max_angles=6):
    """
    Compute all pairwise bond angles around one atom, then pad/truncate to f
    Also returns a mask so the model knows which angle entries are real and
    (Model will learn this mask as well in the node.)
    """
    atom = mol.GetAtomWithIdx(atom_idx)
    nbr_indices = [nbr.GetIdx() for nbr in atom.GetNeighbors()]
    # ONLY HEAVY ATOMS AND NOT HYDROGENS

    angles = []
    for i in range(len(nbr_indices)):
        for j in range(i + 1, len(nbr_indices)):
            a = nbr_indices[i]
            c = nbr_indices[j]
            angle = compute_angle(a, atom_idx, c, pos)
            angles.append(angle)

    angles = sorted(angles)

    real_count = min(len(angles), max_angles)
    angles = angles[:max_angles]
    mask = [1.0] * real_count

    if len(angles) < max_angles:
        pad = max_angles - len(angles)
        angles = angles + [0.0] * pad
        mask = mask + [0.0] * pad

    return angles + mask

# one-hot by turning into basis vectors
def atom_feature_vector(atom, mol, pos, max_angles=6):
    return (
        one_hot(atom.GetSymbol(), alchemy_elements) +  # atomic number, e.g.
```

```
        one_hot(atom.GetDegree(), degree_set) +        # number of directly
        one_hot(atom.GetTotalNumHs(), numH_set) +      # total attached hydr
        one_hot(atom.GetFormalCharge(), charge_set) +  # formal charge on th
        [int(atom.GetIsAromatic())] +                  # aromatic atom flag:
        [int(atom.IsInRing())] +                       # ring membership fla
        one_hot(atom.GetHybridization(), hybridization_set) +  # hybridizati
        padded_angle_vector(atom.GetIdx(), mol, pos, max_angles=max_angles)
    )

# discrete categories in bond features
bond_types = [
    Chem.rdchem.BondType.SINGLE,
    Chem.rdchem.BondType.DOUBLE,
    Chem.rdchem.BondType.TRIPLE,
    Chem.rdchem.BondType.AROMATIC,
]
stereo_types = [
    Chem.rdchem.BondStereo.STEREONONE,
    Chem.rdchem.BondStereo.STEREOZ,
    Chem.rdchem.BondStereo.STEREOE,
]

# one-hot by turning into basis vectors
def bond_feature_vector(bond, pos):

    i = bond.GetBeginAtomIdx()
    j = bond.GetEndAtomIdx()
    distance = torch.norm(pos[i] - pos[j]).item()  # now uses coordinates pu

    return (
        one_hot(bond.GetBondType(), bond_types) +   # bond order: categorica
        [int(bond.IsInRing())] +                    # ring membership flag f
        one_hot(bond.GetStereo(), stereo_types) +   # stereochemistry enum f
        [distance]                                  # bond distance, have to
    )

# this function is specifically to turn alchemy molecules to graphs
# refer @smiles_to_graph_uspto below for when we encounter the USPTO data se
# for now, this is just for the gnn
def smiles_to_graph_alchemy(smi, coords, y, max_angles=6):
    """
    Convert a SMILES string + Alchemy coordinates to a PyTorch Geometric Dat
    Nodes represent atoms, and edges represent bonds.
    """
    mol = Chem.MolFromSmiles(smi)
    if mol is None: return None

    # coordinates are now taken directly from the Alchemy sdf file, not appr
    pos = torch.tensor(coords, dtype=torch.float)

    # coordinates must align with atom order in the SMILES-derived RDKit mol
    if pos.shape[0] != mol.GetNumAtoms(): return None

    # atom features
    atom_features = [atom_feature_vector(atom, mol, pos, max_angles=max_angl
    x = torch.tensor(atom_features, dtype=torch.float)
```

```
    # edge features
    edge_index = []
    edge_attr = []

    for bond in mol.GetBonds():
        i = bond.GetBeginAtomIdx()
        j = bond.GetEndAtomIdx()
        bf = bond_feature_vector(bond, pos)

        edge_index.append((i, j))
        edge_attr.append(bf)

        edge_index.append((j, i))
        edge_attr.append(bf)
        # undirected graph, both directions

    # convert to tensor
    edge_feature_dim = len(
        one_hot(Chem.rdchem.BondType.SINGLE, bond_types) +
        [0] +
        one_hot(Chem.rdchem.BondStereo.STEREONONE, stereo_types) +
        [0.0]
    )

    if len(edge_index) == 0:
        edge_index = torch.empty((2, 0), dtype=torch.long)
        edge_attr = torch.empty((0, edge_feature_dim), dtype=torch.float)
    else:
        edge_index = torch.tensor(edge_index, dtype=torch.long).t().contiguc
        edge_attr = torch.tensor(edge_attr, dtype=torch.float)

    # targets
    y = torch.tensor(y, dtype=torch.float).view(1, -1)

    return Data(
        x=x,
        edge_index=edge_index,
        edge_attr=edge_attr,
        pos=pos,   # 3D coordinates from the Alchemy sdf file
        y=y
    )
```

Convert all the data into graphs to work in the GNN. Then, normalize the molecule node and edge features.

In [5]:
```
# make the graph dataset
graph_dataset = []
for idx, row in molecule_types_df.iterrows():
    graph = smiles_to_graph_alchemy(
        row['smiles'],
        row['coords'], # added to get higher R^2
        row[molecule_targets_df.columns].astype(float).values
    )
    if graph is not None:
```

```
        graph_dataset.append(graph)

# g = graph_dataset[0]
# print(g.x.shape)            # [num_atoms, num_node_features]
# print(g.edge_index.shape)   # [2, num_edges]
# print(g.y.shape)            # should be 9
# okay great nothing exploded

# normalize features that the gnn predicts
# these are all continuous values so normalize them all
all_y = torch.stack([g.y for g in graph_dataset])
y_mean = all_y.mean(dim = 0)
y_std = all_y.std(dim = 0).clamp_min(1e-8) # avoid div by 0 error

for g in graph_dataset:
    g.y = (g.y - y_mean) / y_std

# no need to normalize most original node features as they are all one hot a
# HOWEVER, the padded node angle vector is continuous, while the mask is bir
# so normalize only the padded angle part, not the mask
# convention from atom_feature_vector:
# ... + padded angles (length = max_angles) + mask (length = max_angles)
max_angles = 6 # square with an x for 6 pairs

all_x = torch.cat([g.x for g in graph_dataset], dim=0)

angle_mean = all_x[:, -2*max_angles:-max_angles].mean(dim=0)
angle_std = all_x[:, -2*max_angles:-max_angles].std(dim=0).clamp_min(1e-8)

for g in graph_dataset:
    g.x[:, -2*max_angles:-max_angles] = (
        g.x[:, -2*max_angles:-max_angles] - angle_mean
    ) / angle_std

# normalize edge feature that is continuous i.e. the distance of the bond or
# check @bond_feature_vector

# these are all categorical and shove into one hot
# except distance check @bond_feature_vector
all_edge_attr = torch.cat([g.edge_attr for g in graph_dataset], dim=0)

dist_mean = all_edge_attr[:, -1].mean()
dist_std = all_edge_attr[:, -1].std().clamp_min(1e-8)

for g in graph_dataset:
    g.edge_attr[:, -1] = (g.edge_attr[:, -1] - dist_mean) / dist_std

# NOTE: PCA could be another option to do, but I want the chemical variation
# (I'm a greedy gremlin.)
# and come back later if PCA is really needed
#
# all_y = torch.stack([g.y.squeeze(0) for g in graph_dataset]).numpy()
#
# # fit PCA
# pca = PCA(n_components=5)
# y_reduced = pca.fit_transform(all_y)
```

```
#
# # replace targets with reduced representation
# for g, y_new in zip(graph_dataset, y_reduced):
#     g.y = torch.tensor(y_new, dtype=torch.float).view(1, -1)
#
# # check variance retained
# print(pca.explained_variance_ratio_)
# print("Total variance retained:", pca.explained_variance_ratio_.sum())
```

Now use an architechture that does not ignore bond or edges like GCNConv.

- GINEConv: Propagates node features but not edge features.
- I have tried others that propograte the bond features but they exponentially increase run time even on savio.

In [6]:
```
class GNN(torch.nn.Module):
    # screw around with the hidden_dim to get better regression statistic
    # number of y predictions is out_dim in case of future change
    def __init__(self, in_channels, edge_dim, hidden_dim, out_dim = molecule
        super().__init__()

        # node features into hidden space
        self.node_encoder = nn.Linear(in_channels, hidden_dim)

        # edge features into hidden space
        self.edge_encoder = nn.Linear(edge_dim, hidden_dim)

        # GINEConv
        self.conv1 = GINEConv(
            nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim)
            ),
            edge_dim=hidden_dim
        )

        self.conv2 = GINEConv(
            nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim)
            ),
            edge_dim=hidden_dim
        )

        self.conv3 = GINEConv(
            nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim)
            ),
            edge_dim=hidden_dim
        )
```

```
        self.bn1 = nn.BatchNorm1d(hidden_dim)
        self.bn2 = nn.BatchNorm1d(hidden_dim)
        self.bn3 = nn.BatchNorm1d(hidden_dim)

        self.lin1 = nn.Linear(hidden_dim, hidden_dim)
        self.lin2 = nn.Linear(hidden_dim, out_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, edge_index, edge_attr, batch):
        # encode raw node and edge features into hidden space
        x = self.node_encoder(x)
        edge_attr = self.edge_encoder(edge_attr)

        # first graph convolution block
        x_res = x # skip connection
        x = self.conv1(x, edge_index, edge_attr)
        x = self.bn1(x)
        x = F.relu(x)
        x = self.dropout(x)
        x = x + x_res # skip connection

        # second graph convolution block
        x_res = x # skip connection
        x = self.conv2(x, edge_index, edge_attr)
        x = self.bn2(x)
        x = F.relu(x)
        x = self.dropout(x)
        x = x + x_res # skip connection

        # third graph convolution block
        x_res = x # skip connection
        x = self.conv3(x, edge_index, edge_attr)
        x = self.bn3(x)
        x = F.relu(x)
        x = x + x_res # skip connection

        # pool from atom space to molecule space
        x = global_mean_pool(x, batch)

        # readout MLP
        x_res = x # skip connection
        x = self.lin1(x)
        x = F.relu(x)
        x = self.dropout(x)
        x = x + x_res # skip connection
        x = self.lin2(x)
        return x

class Trainer:
    def __init__(
        self,
        model,
        optimizer=torch.optim.Adam,
        lr=1e-3,
        loss_type="smoothl1",   # "mse" or "smoothl1"
        beta=0.5, # for smooth l1; 1-2 is more MSE like, 0.5 is more L1 like
```

```
        device="cpu"
    ):
        super().__init__()

        self.model = model.to(device)
        self.device = device
        self.optimizer = optimizer(self.model.parameters(), lr=lr)

        if loss_type == "mse": self.loss_function = torch.nn.MSELoss()
        elif loss_type == "smoothl1": self.loss_function = torch.nn.SmoothL1
        else: raise ValueError(f"Unsupported loss_type: {loss_type}'")

    def train_one_epoch(self, loader):
        self.model.train() # training 'mode'
        total_loss = 0.0

        for batch in loader:
            batch = batch.to(self.device)
            self.optimizer.zero_grad() # past grad not needed as we evaluate

            # mps needs contiguous tensors for smooth l1, so reshape and mak
            pred = self.model(batch.x, batch.edge_index, batch.edge_attr, ba
            target = batch.y
            pred = pred.reshape(target.shape).contiguous()
            target = target.contiguous()

            loss = self.loss_function(pred, target)
            loss.backward() # find the gradients
            self.optimizer.step() # update the weights
            total_loss += loss.item() # tack on loss

        return total_loss / len(loader) # average loss per batch

    # used for testing phase
    @torch.no_grad()
    def evaluate(self, loader):
        self.model.eval()
        total_loss = 0.0
        all_preds = []  # Collect all predictions
        all_targets = []  # Collect all targets

        for batch in loader:
            batch = batch.to(self.device)
            # prediction and actual (i.e. target)
            pred = self.model(batch.x, batch.edge_index, batch.edge_attr, ba
            target = batch.y
            # mps needs contiguous tensors for smoothl1, so reshape and make
            pred = pred.reshape(target.shape).contiguous()
            target = target.contiguous()
            loss = self.loss_function(pred, target)
            total_loss += loss.item()
            all_preds.append(pred.cpu())  # Store predictions for R² calcula
            all_targets.append(target.cpu())  # Store targets for R² calcula
            # move to cpu else gpu memory kaboom like grandma

        all_preds = torch.cat(all_preds, dim=0)  # Concatenate all predictio
```

```
        all_targets = torch.cat(all_targets, dim=0)  # Concatenate all targe

        # Compute average R² across outputs (measure of accuracy for regress
        ss_res = ((all_preds - all_targets) ** 2).sum(dim=0)  # Sum of squar
        ss_tot = ((all_targets - all_targets.mean(dim=0)) ** 2).sum(dim=0).c

        r2_per_output = 1 - (ss_res / ss_tot)  # R² per output
        avg_r2 = r2_per_output.mean().item()  # Average R² across all output

        return total_loss / len(loader), avg_r2


    def fit(self, train_loader, val_loader, epochs, show_progress = True):
        train_losses = []
        val_losses = []
        train_r2s = []
        val_r2s = []  # Track R² history

        best_val = float("inf")
        best_train = None
        best_val_r2 = -float("inf")  # Track best R²
        best_train_r2 = None

        for epoch in range(1, epochs + 1):
            train_loss = self.train_one_epoch(train_loader)
            _, train_r2 = self.evaluate(train_loader)     # R² on 66%
            val_loss, val_r2 = self.evaluate(val_loader)  # R² on 33% on lik

            train_losses.append(train_loss)
            val_losses.append(val_loss)
            train_r2s.append(train_r2)
            val_r2s.append(val_r2)

            if val_loss < best_val:
                best_val = val_loss
                best_train = train_loss
                best_val_r2 = val_r2
                best_train_r2 = train_r2

            if show_progress:
                print(
                    f"Epoch {epoch}: "
                    f"train={train_loss:.4f}, "
                    f"train_r2={train_r2:.4f}, "
                    f"val={val_loss:.4f}, "
                    f"val_r2={val_r2:.4f}"
                )

        return {
            # arrays for plotting history
            "train_losses": train_losses,
            "val_losses": val_losses,
            "train_r2s": train_r2s,
            "val_r2s": val_r2s,

            # best values from validation
```

```
            "best_train_loss": best_train,
            "best_val_loss": best_val,
            "best_train_r2": best_train_r2,
            "best_val_r2": best_val_r2
        }
```

Functions to facilitate running the model.

In [7]:
```
def run_single_split(
        model_class,
        train_graphs,
        val_graphs,
        batch_size=32,
        epochs=20,
        hidden_dim=64,
        dropout=0.2,
        lr=1e-3,
        optimizer=torch.optim.Adam,
        loss_type="smoothl1",
        beta=0.5,
        device="cpu",
        show_progress=False
):
    """
    Train one model on one train/validation split.
    Returns metrics and the trained model.
    """
    # arguments for model class (fixed)
    in_channels = train_graphs[0].x.shape[1]
    edge_dim = train_graphs[0].edge_attr.shape[1]
    out_dim = train_graphs[0].y.shape[1]

    train_loader = DataLoader(train_graphs, batch_size=batch_size, shuffle=T
    val_loader = DataLoader(val_graphs, batch_size=batch_size, shuffle=False

    model = model_class(
        in_channels=in_channels,
        edge_dim=edge_dim,
        hidden_dim=hidden_dim,
        out_dim=out_dim,
        dropout=dropout
    )

    trainer = Trainer(
        model=model,
        optimizer=optimizer,
        lr=lr,
        loss_type=loss_type,
        beta=beta,
        device=device
    )

    results = trainer.fit(
        train_loader=train_loader,
        val_loader=val_loader,
```

```
        epochs=epochs,
        show_progress=show_progress
    )

    return {
        "model": trainer.model,

        "best_train_loss": results["best_train_loss"],
        "best_val_loss": results["best_val_loss"],
        "best_train_r2": results["best_train_r2"],
        "best_val_r2": results["best_val_r2"],

        "train_losses": results["train_losses"],
        "val_losses": results["val_losses"],
        "train_r2s": results["train_r2s"],
        "val_r2s": results["val_r2s"],
    }


# replace accuracy with the R^2 regression statistic
# function used to see if the model scales well and does not explode
def KFoldCrossValidation(
    model_class,
    graph_dataset,
    k=3,
    batch_size=32,
    epochs=20,
    hidden_dim=64,
    dropout=0.2,
    lr=1e-3,
    optimizer=torch.optim.Adam,
    loss_type="smoothl1",
    beta=0.5,
    device="cpu",
    early_break=False,
    show_progress=False,
    make_plots=True,
    verbose=True, # print per fold results
    random_state=67
):
    """
    Run either:
    - k=1: a single train/validation split
    - k>1: k-fold cross validation

    Returns only metrics for evaluation / hyperparameter search.
    Does NOT return a model. (No need.)
    """
    if k == 1: # k = 1 was used during code creation to check fast
        indices = list(range(len(graph_dataset)))
        train_idx, val_idx = train_test_split(
            indices,
            test_size=0.2,
            random_state=random_state,
            shuffle=True
        )
```

```
        splits = [(train_idx, val_idx)]
    else:
        kf = KFold(n_splits=k, shuffle=True, random_state=random_state)
        splits = kf.split(graph_dataset)

    # contain results for each fold to compute average and std dev
    train_loss_list = []
    val_loss_list = []
    train_r2_list = []
    val_r2_list = []

    all_train_losses = []
    all_val_losses = []
    all_train_r2s = []
    all_val_r2s = []

    for fold, (train_idx, val_idx) in enumerate(splits):

        # split data
        train_graphs = [graph_dataset[i] for i in train_idx]
        val_graphs = [graph_dataset[i] for i in val_idx]

        # train and evaluate model on this fold
        fold_results = run_single_split(
            model_class=model_class,
            train_graphs=train_graphs,
            val_graphs=val_graphs, # run_single_split expects raw graphs, nc
            batch_size=batch_size,
            epochs=epochs,
            hidden_dim=hidden_dim,
            dropout=dropout,
            lr=lr,
            optimizer=optimizer,
            loss_type=loss_type,
            beta=beta,
            device=device,
            show_progress=show_progress
        )

        train_loss_best = fold_results["best_train_loss"]
        val_loss_best = fold_results["best_val_loss"]
        train_r2_best = fold_results["best_train_r2"]
        val_r2_best = fold_results["best_val_r2"]

        train_loss_list.append(train_loss_best)
        val_loss_list.append(val_loss_best)
        train_r2_list.append(train_r2_best)
        val_r2_list.append(val_r2_best)  # Append best R² for final averagir

        # append it all on
        all_train_losses.append(fold_results["train_losses"])
        all_val_losses.append(fold_results["val_losses"])
        all_train_r2s.append(fold_results["train_r2s"])
        all_val_r2s.append(fold_results["val_r2s"])

        if verbose:
```

```
            print(f"\nFold {fold+1}")
            print(f"Best train loss: {train_loss_best:.4f}")
            print(f"Best val loss: {val_loss_best:.4f}")
            print(f"Best train R²: {train_r2_best:.4f}")
            print(f"Best val R²: {val_r2_best:.4f}")  # Print R² for this fo

        if early_break: break

    # Compute average losses and R² per epoch across folds
    avg_train_losses = np.mean(np.array(all_train_losses), axis=0)  # Averag
    avg_val_losses = np.mean(np.array(all_val_losses), axis=0)  # Average va
    avg_train_r2s = np.mean(np.array(all_train_r2s), axis=0)  # Average trai
    avg_val_r2s = np.mean(np.array(all_val_r2s), axis=0)  # Average val R² p

    if make_plots:
        plt.figure()
        plt.plot(avg_train_losses, label='Average Train Loss')
        plt.plot(avg_val_losses, label='Average Validation Loss')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.title('Training and Validation Loss Curves')
        plt.legend()
        plt.grid(True)
        plt.show()

        plt.figure()
        plt.plot(avg_train_r2s, label="Train R²")
        plt.plot(avg_val_r2s, label="Validation R²")
        plt.xlabel("Epoch")
        plt.ylabel("R²")
        plt.title("Train vs Validation R²")
        plt.legend()
        plt.grid(True)
        plt.show()

    summary = {
        # hyperparameters I tracked here
        "hyperparameters": {
            "k": k,
            "batch_size": batch_size,
            "epochs": epochs,
            "hidden_dim": hidden_dim,
            "dropout": dropout,
            "lr": lr,
            "optimizer": optimizer.__name__ if hasattr(optimizer, "__name__"
            "loss_type": loss_type,
            "beta": beta,
            "device": device,
            "random_state": random_state,
        },

        # main metrics
        "mean_train_loss": float(np.mean(train_loss_list)),
        "std_train_loss": float(np.std(train_loss_list)),
        "mean_val_loss": float(np.mean(val_loss_list)),
        "std_val_loss": float(np.std(val_loss_list)),
```

```
        "mean_train_r2": float(np.mean(train_r2_list)),
        "std_train_r2": float(np.std(train_r2_list)),
        "mean_val_r2": float(np.mean(val_r2_list)),
        "std_val_r2": float(np.std(val_r2_list)),

        # per-fold metrics
        "train_loss_per_fold": train_loss_list,
        "val_loss_per_fold": val_loss_list,
        "train_r2_per_fold": train_r2_list,
        "val_r2_per_fold": val_r2_list,

        # average curves
        "avg_train_losses": avg_train_losses,
        "avg_val_losses": avg_val_losses,
        "avg_train_r2s": avg_train_r2s,
        "avg_val_r2s": avg_val_r2s,
    }

    if make_plots: # information to help read the plots
        print("\nFinal results:")
        print(f"Train loss: {summary['mean_train_loss']:.4f} ± {summary['std
        print(f"Val loss: {summary['mean_val_loss']:.4f} ± {summary['std_val
        print(f"Train R²: {summary['mean_train_r2']:.4f} ± {summary['std_tra
        print(f"Val R²: {summary['mean_val_r2']:.4f} ± {summary['std_val_r2'

    return summary

# final function to train/val/test on the whole alchemy dataset
def train_and_test(
        model_class,
        graph_dataset,
        test_size=0.1,
        val_size=0.1,
        batch_size=32,
        epochs=20,
        hidden_dim=64,
        dropout=0.2,
        lr=1e-3,
        optimizer=torch.optim.Adam,
        loss_type="smoothl1",
        beta=0.5,
        device="cpu",
        random_state=67,
        show_progress=False,
        verbose=True,
        make_plots=True,
):
    """
    Used in the final evaluation phase after hyperparameter tuning to train
    a model on the full training set and evaluate on the test set.
    1. Split full dataset into train_pool and test
    2. Split train_pool into train and val
    3. Train on train
    4. Save best checkpoint based on val R2
    5. Reload best checkpoint
```

```
    6. Evaluate once on train/val/test
    """

    # train pool and test
    train_pool_graphs, test_graphs = train_test_split(
        graph_dataset,
        test_size=test_size,
        random_state=random_state,
        shuffle=True
    )

    # train and val
    adjusted_val_size = val_size / (1 - test_size) # size relative to the fi

    train_graphs, val_graphs = train_test_split(
        train_pool_graphs,
        test_size=adjusted_val_size,
        random_state=random_state,
        shuffle=True
    )

    # inferred dimensions of for the model class (fixed)
    in_channels = train_graphs[0].x.shape[1]
    edge_dim = train_graphs[0].edge_attr.shape[1]
    out_dim = train_graphs[0].y.shape[1]

    train_loader = DataLoader(train_graphs, batch_size=batch_size, shuffle=T
    val_loader = DataLoader(val_graphs, batch_size=batch_size, shuffle=False
    test_loader = DataLoader(test_graphs, batch_size=batch_size, shuffle=Fal

    # building the mdoel + trainer
    model = model_class(
        in_channels=in_channels,
        edge_dim=edge_dim,
        hidden_dim=hidden_dim,
        out_dim=out_dim,
        dropout=dropout
    )

    trainer = Trainer(
        model=model,
        optimizer=optimizer,
        lr=lr,
        loss_type=loss_type,
        beta=beta,
        device=device
    )

    model = trainer.model.to(device)

    # tracking all of this
    train_losses = []
    val_losses = []
    train_r2s = []
    val_r2s = []
```

```
    best_val_r2 = -float("inf")
    best_epoch = None
    best_state_dict = None

    # training loop
    for epoch in range(1, epochs + 1):
        train_loss = trainer.train_one_epoch(train_loader)

        _, train_r2 = trainer.evaluate(train_loader)
        val_loss, val_r2 = trainer.evaluate(val_loader)

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_r2s.append(train_r2)
        val_r2s.append(val_r2)

        if val_r2 > best_val_r2:
            best_val_r2 = val_r2
            best_epoch = epoch
            best_state_dict = copy.deepcopy(model.state_dict())

        if show_progress:
            print(
                f"Epoch {epoch}: "
                f"train_loss={train_loss:.4f}, "
                f"train_r2={train_r2:.4f}, "
                f"val_loss={val_loss:.4f}, "
                f"val_r2={val_r2:.4f}"
            )

    if best_state_dict is None: # in case, safety
        best_state_dict = copy.deepcopy(model.state_dict())
        best_epoch = epoch

    # reload best checkpoint
    model.load_state_dict(best_state_dict)
    model.eval()

    # pick loss func for the final evaluation
    if loss_type == "mse": loss_function = torch.nn.MSELoss()
    elif loss_type == "smoothl1": loss_function = torch.nn.SmoothL1Loss(beta
    else: raise ValueError("Unsupported loss_type: {loss_type}")

    # helper function for final eval
    @torch.no_grad()
    def detailed_evaluation(loader):
        total_loss = 0.0
        pred_list = []
        target_list = []

        for batch in loader:
            batch = batch.to(device)

            target = batch.y.contiguous()
            pred = model(batch.x, batch.edge_index, batch.edge_attr, batch.b
            pred = pred.reshape(target.shape).contiguous()
```

```
            loss = loss_function(pred, target)
            total_loss += loss.item()

            pred_list.append(pred.cpu())
            target_list.append(target.cpu())

        preds = torch.cat(pred_list, dim=0)
        targets = torch.cat(target_list, dim=0)

        ss_res = ((preds - targets) ** 2).sum(dim=0)
        ss_tot = ((targets - targets.mean(dim=0)) ** 2).sum(dim=0).clamp_mir

        r2_per_output = 1 - (ss_res / ss_tot)
        avg_r2 = r2_per_output.mean().item()
        avg_loss = total_loss / len(loader)

        return avg_loss, avg_r2, r2_per_output, preds, targets

    train_loss_final, train_r2_final, train_r2_per_output, _, _ = detailed_e
    val_loss_final, val_r2_final, val_r2_per_output, _, _ = detailed_evaluat
    test_loss_final, test_r2_final, test_r2_per_output, test_preds, test_tar

    if make_plots:
        # train and validation loss
        plt.figure()
        plt.plot(range(1, epochs + 1), train_losses, label="Train Loss")
        plt.plot(range(1, epochs + 1), val_losses, label="Validation Loss")
        plt.axvline(best_epoch, linestyle="--", label=f"Best Epoch = {best_e
        plt.xlabel("Epoch")
        plt.ylabel("Loss")
        plt.title("Train and Validation Loss")
        plt.legend()
        plt.grid()
        plt.show()

        # train and validation r2
        plt.figure()
        plt.plot(range(1, epochs + 1), train_r2s, label="Train R²")
        plt.plot(range(1, epochs + 1), val_r2s, label="Validation R²")
        plt.axvline(best_epoch, linestyle="--", label=f"Best Epoch = {best_e
        plt.xlabel("Epoch")
        plt.ylabel("R²")
        plt.title("Train and Validation R²")
        plt.legend()
        plt.grid()
        plt.show()

        # predicted vs true plot aka parity plot
        plt.figure()
        plt.scatter(test_targets.flatten().numpy(), test_preds.flatten().num
        plt.xlabel("True")
        plt.ylabel("Predicted")
        plt.title("Predicted vs True (Test)")
        plt.grid()
        plt.show()
```

```
        # per r2 property plot
        clean_property_names = [
            "ZPVE",
            "Cv",
            "Gap",
            "G",
            "HOMO",
            "Polarizability",
            "LUMO",
            "Dipole (μ)",
            "Electron extent (⟨r²⟩)"
        ]

        plt.figure(figsize=(10, 5))
        plt.bar(clean_property_names, test_r2_per_output.cpu().numpy())
        plt.xlabel("Molecular Property")
        plt.ylabel("R² score")
        plt.title("Test R² per Molecular Property")
        plt.xticks(rotation=45)
        plt.grid(axis="y")
        plt.tight_layout()
        plt.show()


    if verbose:
        print(f"Best Epoch (by val R²): {best_epoch}")
        print(f"Final Train R²: {train_r2_final:.4f}")
        print(f"Final Train Loss: {train_loss_final:.4f}")
        print(f"Final Val R²: {val_r2_final:.4f}")
        print(f"Final Val Loss: {val_loss_final:.4f}")
        print(f"Final Test R²: {test_r2_final:.4f}")
        print(f"Final Test Loss: {test_loss_final:.4f}")

    return {
        # best model since I saved the best checkpoint
        "model": model,
        "best_epoch": best_epoch,

        # array of train and validation losses and r2s
        "train_losses": train_losses,
        "val_losses": val_losses,
        "train_r2s": train_r2s,
        "val_r2s": val_r2s,

        # final values from the model
        "train_r2": train_r2_final,
        "train_loss": train_loss_final,
        "train_r2_per_output": train_r2_per_output,

        "val_r2": val_r2_final,
        "val_loss": val_loss_final,
        "val_r2_per_output": val_r2_per_output,

        "test_r2": test_r2_final,
        "test_loss": test_loss_final,
```

```
        "test_r2_per_output": test_r2_per_output
    }
```

Now I will find hyperparameters for out model. Sklearn conveniently has a package for this.

In [8]:
```
# the pain and suffering of a mac user
device = "mps" if torch.backends.mps.is_available() else "cpu"

class GNNRegressor(BaseEstimator):
    def __init__(
        self,
        hidden_dim=64,
        lr=1e-3,
        dropout=0.1,
        batch_size=64,
        beta=0.5,
        epochs=15,
        device="cpu"
    ):
        self.hidden_dim = hidden_dim
        self.lr = lr
        self.dropout = dropout
        self.batch_size = batch_size
        self.beta = beta
        self.epochs = epochs
        self.device = device

    def fit(self, X, y=None):
        in_channels = X[0].x.shape[1]
        edge_dim = X[0].edge_attr.shape[1]
        out_dim = X[0].y.shape[1]

        self.model_ = GNN(
            in_channels=in_channels,
            edge_dim=edge_dim,
            hidden_dim=self.hidden_dim,
            out_dim=out_dim,
            dropout=self.dropout
        )

        self.trainer_ = Trainer(
            model=self.model_,
            lr=self.lr,
            loss_type="smoothl1",
            beta=self.beta,
            device=self.device
        )

        train_loader = DataLoader(X, batch_size=self.batch_size, shuffle=Tru

        for _ in range(self.epochs):
            self.trainer_.train_one_epoch(train_loader)

        return self
```

```
    def score(self, X, y=None):
        val_loader = DataLoader(X, batch_size=self.batch_size, shuffle=False
        _, val_r2 = self.trainer_.evaluate(val_loader)
        return val_r2


indices = np.random.RandomState(67).choice(len(graph_dataset), size=1000, re
subset = [graph_dataset[i] for i in indices]

param_dist = {
    "hidden_dim": list(range(64, 132, 4)),
    "lr": loguniform(3e-4, 1e-3),
    "dropout": uniform(0.0, 0.35),
    "batch_size": randint(64, 160),
    "beta": uniform(0.2, 0.7),
}

search = HalvingRandomSearchCV(
    estimator=GNNRegressor(device=device),
    param_distributions=param_dist,
    factor=2, # this factor controls how aggresive pruning is; 1/factor
    resource="epochs",
    max_resources=15,
    min_resources=5,
    cv=3,
    verbose=0,
    refit=False,
    random_state=67
) # https://www.youtube.com/watch?v=rSJd25w6e-0

search.fit(subset)

df = pd.DataFrame(search.cv_results_)

df = df[[
    "mean_test_score",
    "std_test_score",
    "param_epochs",
    "param_hidden_dim",
    "param_lr",
    "param_dropout",
    "param_batch_size",
    "param_beta"
]].rename(columns={
    "mean_test_score": "r2",
    "std_test_score": "std",
    "param_epochs": "epochs",
    "param_hidden_dim": "hidden_dim",
    "param_lr": "lr",
    "param_dropout": "dropout",
    "param_batch_size": "batch_size",
    "param_beta": "beta"
})

df = df.sort_values(
```

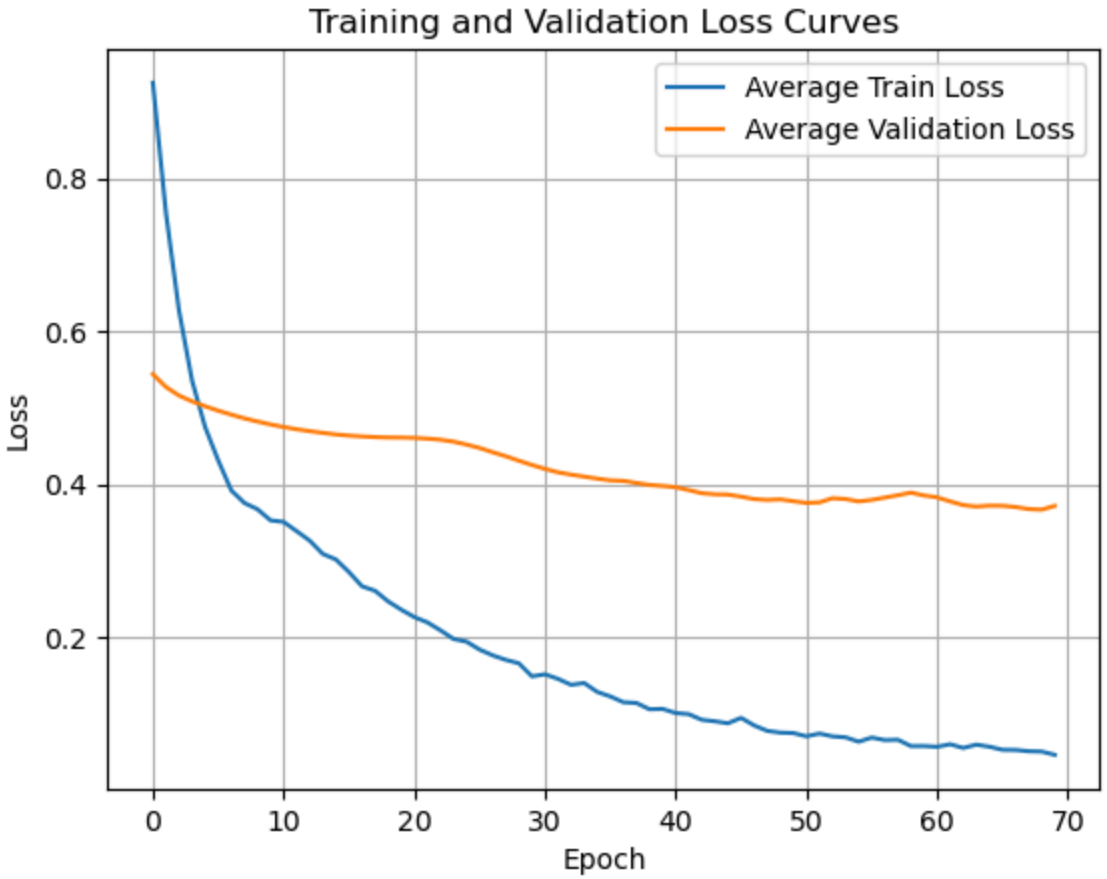
```
    by=["r2", "std"],
    ascending=[False, True]
).reset_index(drop=True)

df
# engineering: so take the mean of a few of these best parameters and we sho
```

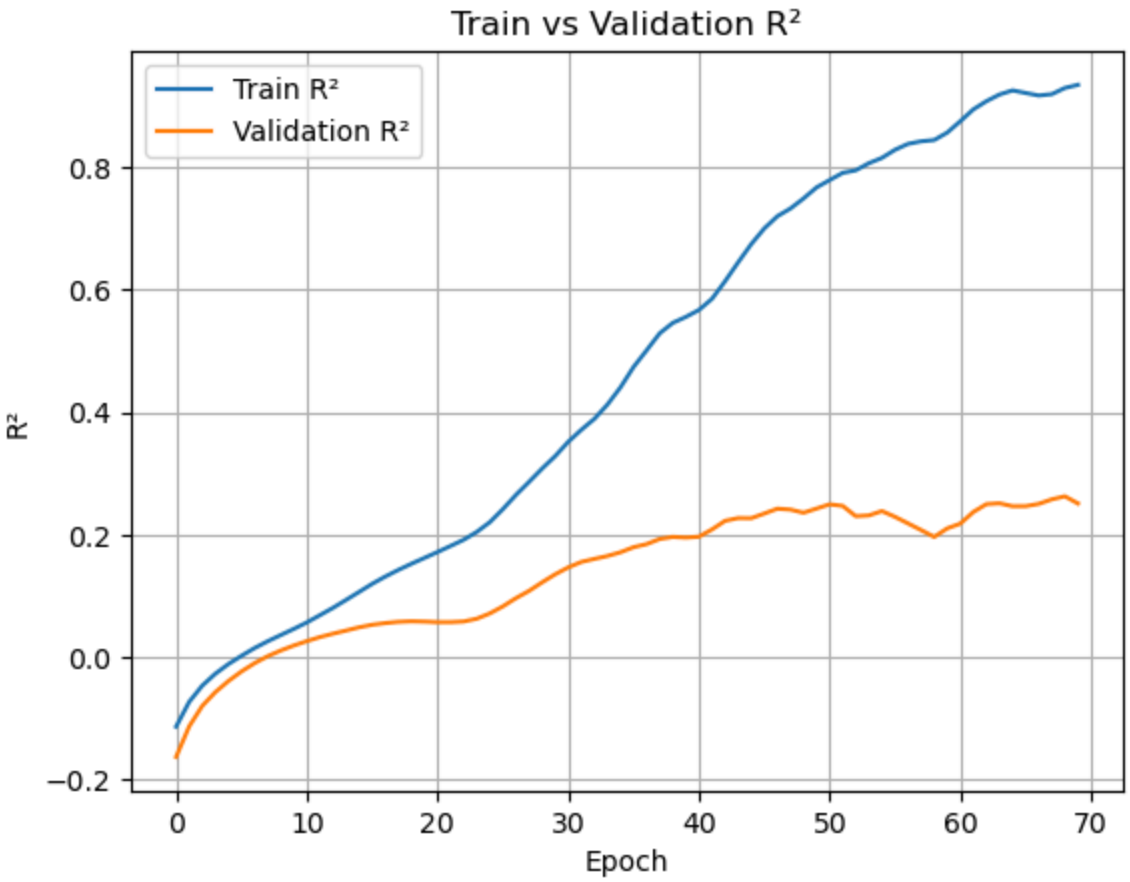
Out[8]:

	r2	std	epochs	hidden_dim	lr	dropout	batch_size	be
0	0.603448	0.006179	10	96	0.000907	0.074602	86	0.47039
1	0.464481	0.023498	10	100	0.000825	0.332974	131	0.72599
2	0.389480	0.031255	5	96	0.000907	0.074602	86	0.47039
3	0.202821	0.016887	5	100	0.000825	0.332974	131	0.72599
4	0.108129	0.011445	5	120	0.000527	0.287459	158	0.23282

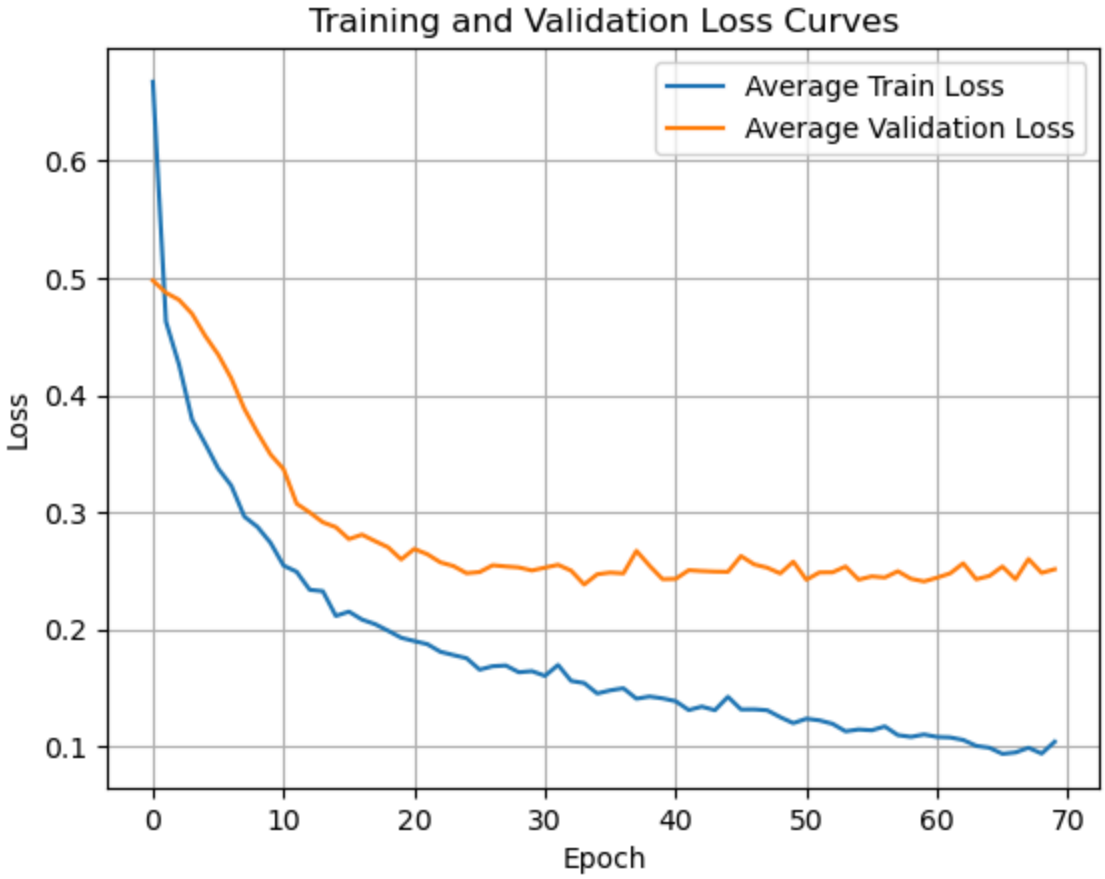
In [9]:

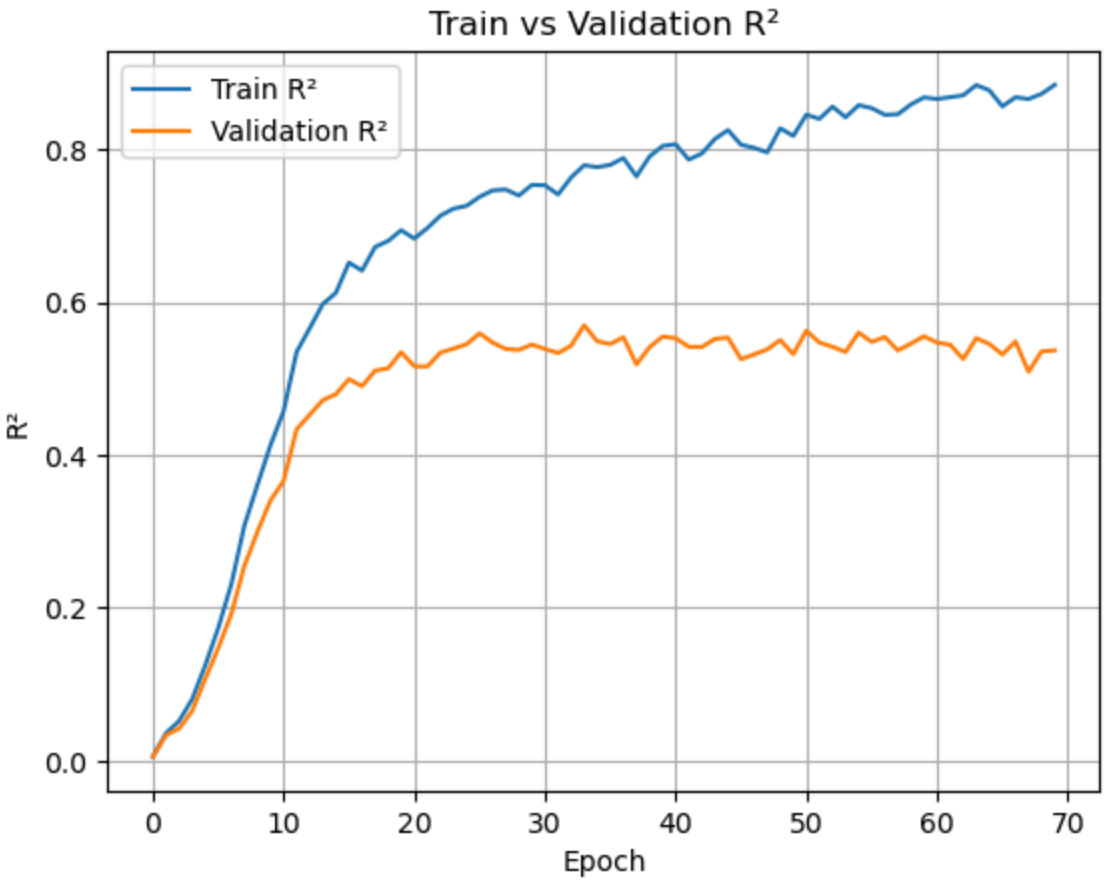
```
# check if R^2 validation scales with number of examples as my model learns
# if it does scale, I can continue and make my final @test_and_train functio
for sample in [100, 500, 1000]:
    # small subset
    subset = graph_dataset[:sample]

    results = KFoldCrossValidation(
        model_class=GNN,
        graph_dataset=subset,
        k=3,
        batch_size=100,
        epochs=70,
        hidden_dim=104,
        dropout=0.18,
        lr=7e-4,
        loss_type="smoothl1",
        beta=0.76,
        device=device,
        make_plots=True,
        verbose=False # do not turn True your eyes will bleed
        # YOU HAVE BEEN WARNED
    ) # R^2 val does scale with number of samples
```


Training and Validation Loss Curves
Average Train Loss
Average Validation Loss
Loss
Epoch
0.8
0.6
0.4
0.2
0
10
20
30
40
50
60
70

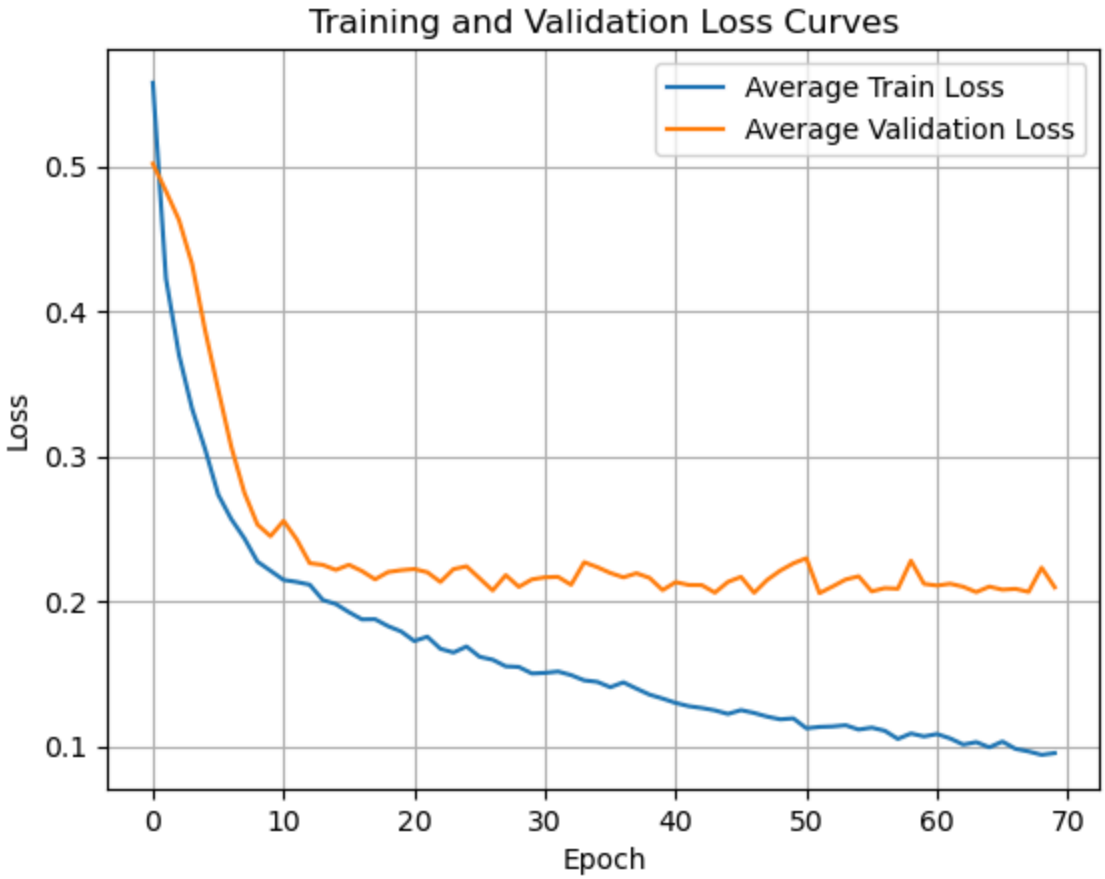

Train vs Validation R²
Train R²
Validation R²
R²
Epoch
0.8
0.6
0.4
0.2
0.0
−0.2
0
10
20
30
40
50
60
70

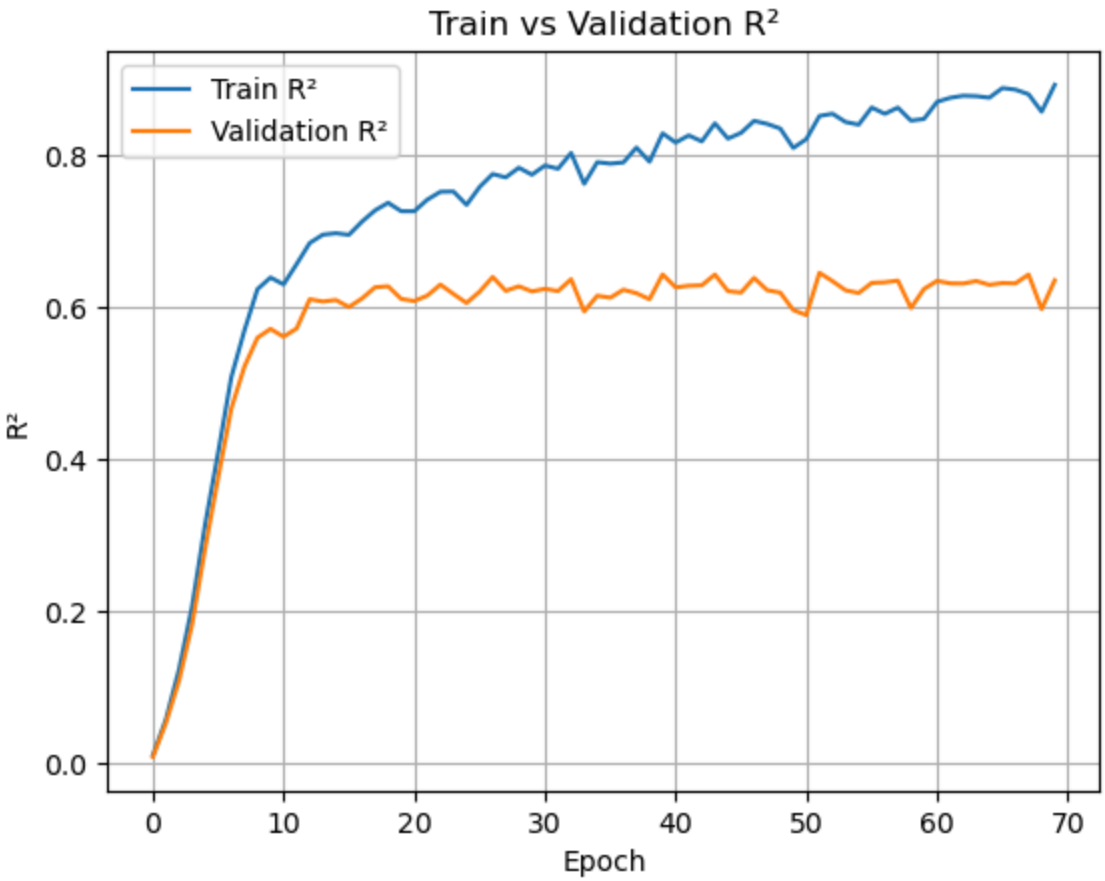
```
Final results:
Train loss: 0.0574 ± 0.0068
Val loss: 0.3568 ± 0.0638
Train R²: 0.8817 ± 0.0625
Val R²: 0.2943 ± 0.0300
```






```
Final results:
Train loss: 0.1207 ± 0.0296
Val loss: 0.2321 ± 0.0091
Train R²: 0.8430 ± 0.0515
Val R²: 0.5712 ± 0.0285
```

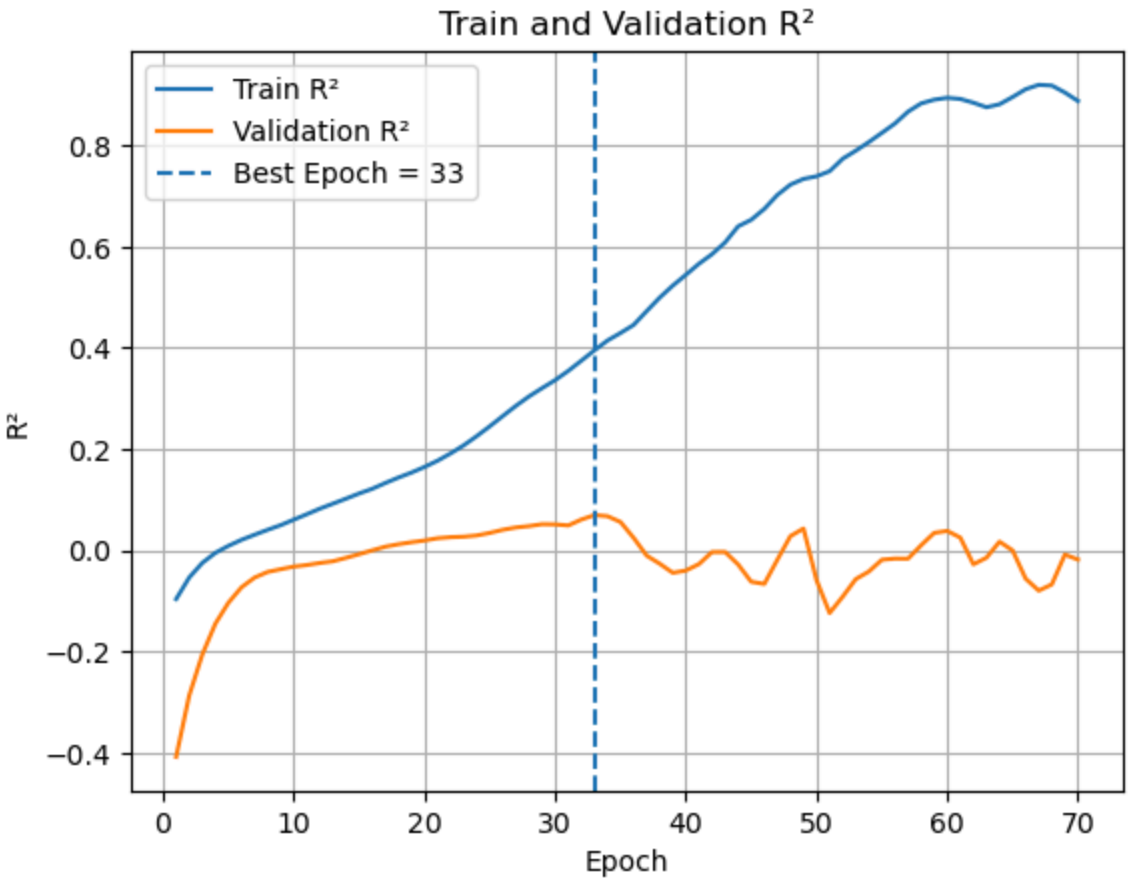

Training and Validation Loss Curves
Average Train Loss
Average Validation Loss
Loss
0.1
0.2
0.3
0.4
0.5
0
10
20
30
40
50
60
70
Epoch

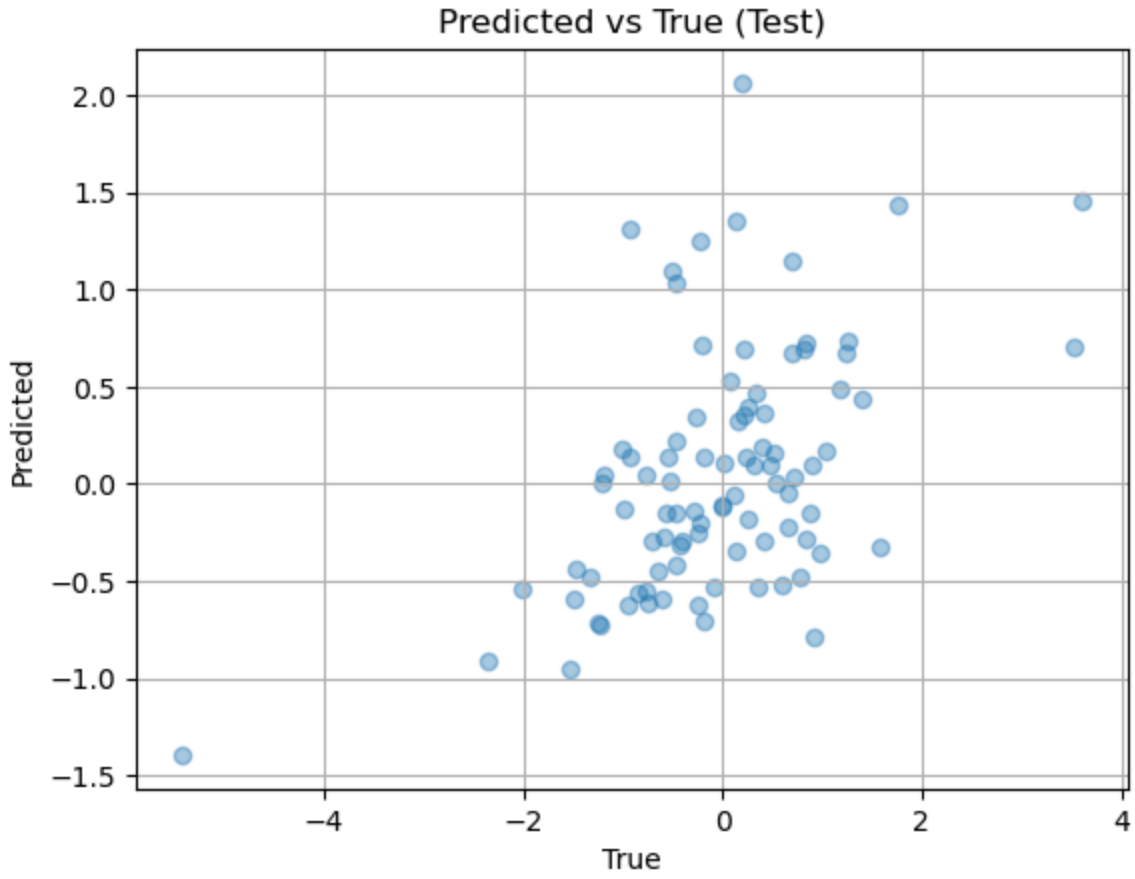

Train vs Validation R²
Train R²
Validation R²
R²
0.0
0.2
0.4
0.6
0.8
0
10
20
30
40
50
60
70
Epoch

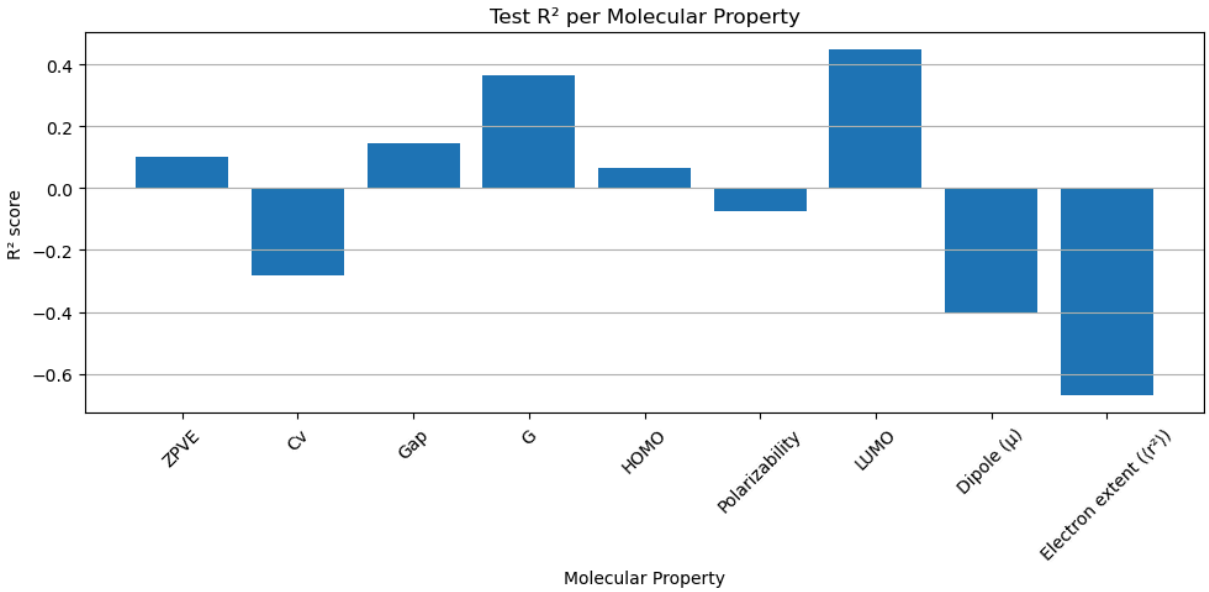
```
Final results:
Train loss: 0.1011 ± 0.0046
Val loss: 0.1969 ± 0.0146
Train R²: 0.8931 ± 0.0134
Val R²: 0.6551 ± 0.0265
```

In [10]:
```python
# testing if my final @train_and_test function works well
# using hyperparameters approximated above

for sample in [100, 500, 1000]:
    subset = graph_dataset[:sample]

    results = train_and_test(
        model_class=GNN,
        graph_dataset=subset,
        test_size=0.1,
        val_size=0.1,
        batch_size=100,
        epochs=70,
        hidden_dim=104,
        dropout=0.18,
        lr=7e-4,
        loss_type="smoothl1",
        beta=0.75,
        device=device,
        random_state=67,
        show_progress=False,
        verbose=True,
        make_plots=True
    )
```

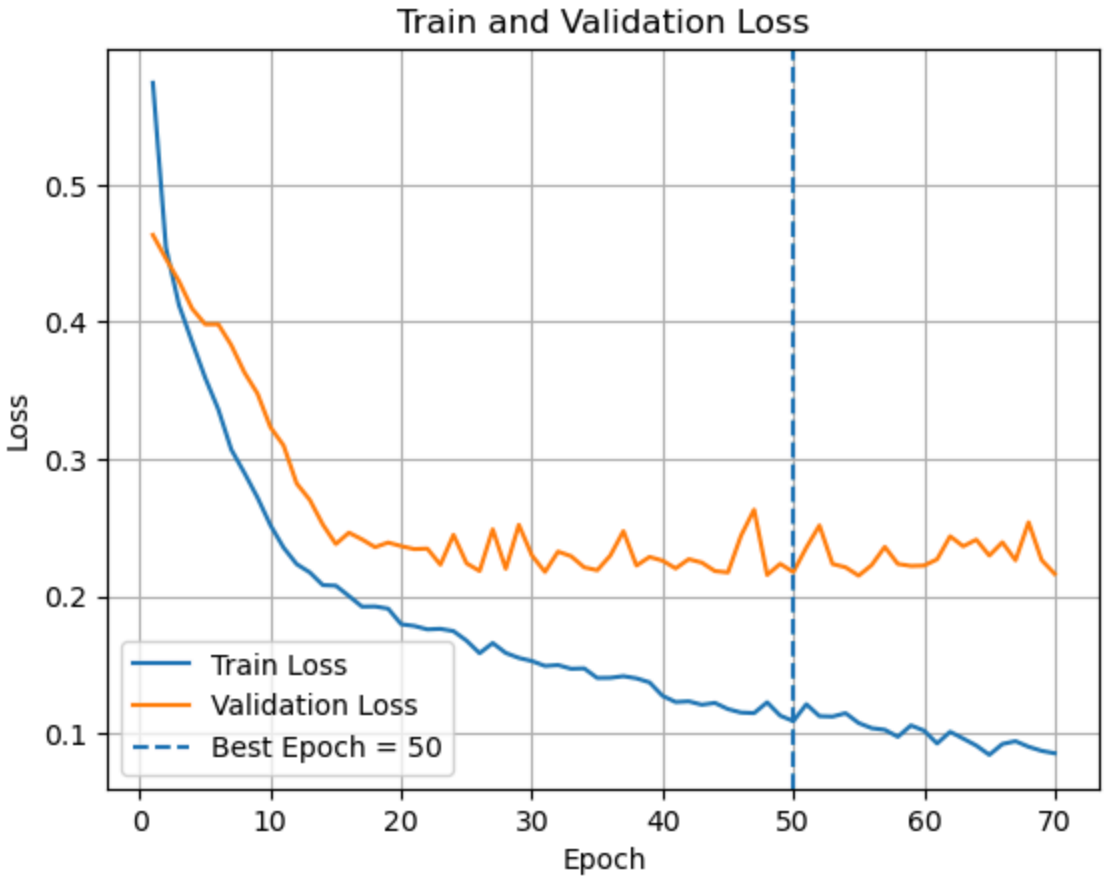

Train and Validation Loss
Train Loss
Validation Loss
Best Epoch = 33
Loss
Epoch
0.8
0.6
0.4
0.2
0
10
20
30
40
50
60
70

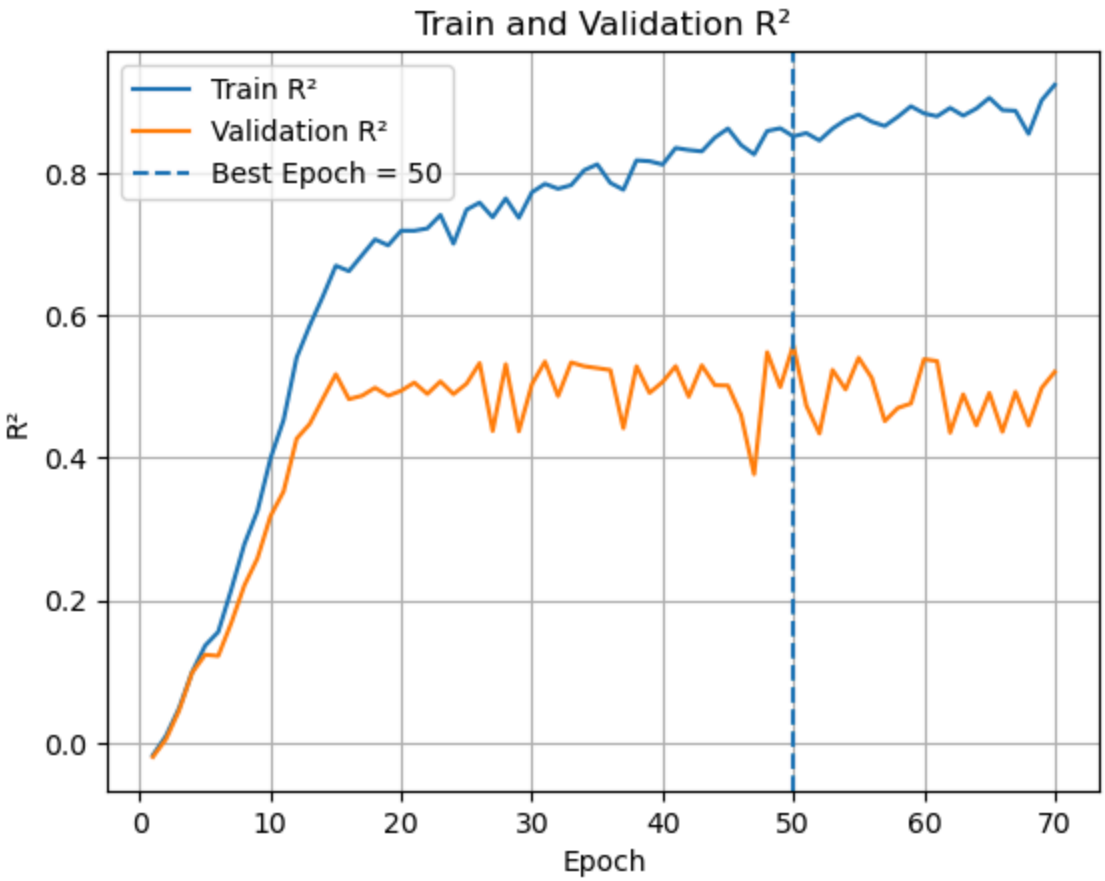

Train and Validation R²
Train R²
Validation R²
Best Epoch = 33
R²
Epoch
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
0
10
20
30
40
50
60
70

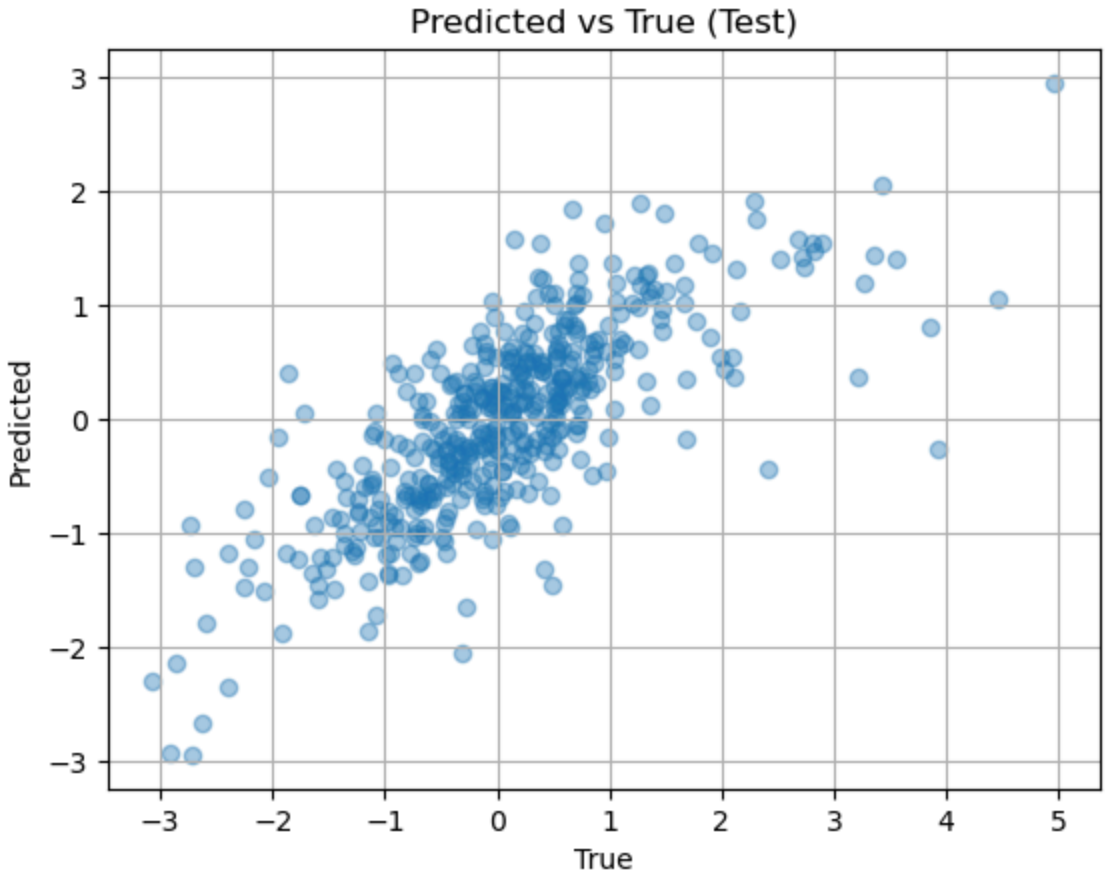


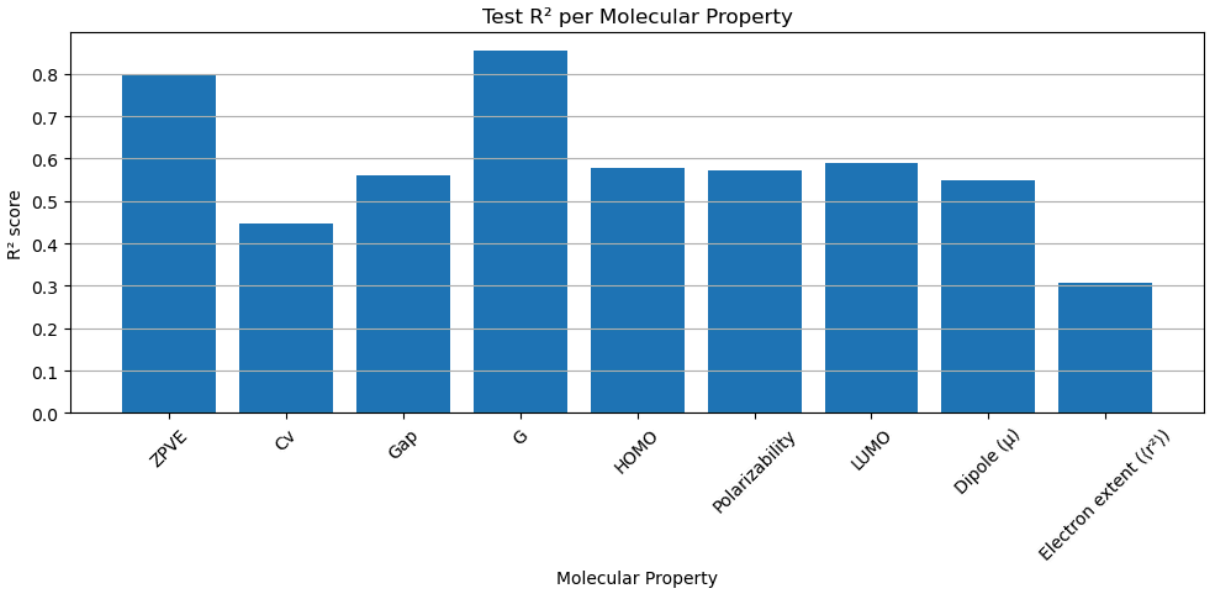



```
Best Epoch (by val R²): 33
Final Train R²: 0.3952
Final Train Loss: 0.3387
Final Val R²: 0.0695
Final Val Loss: 0.3456
Final Test R²: -0.0337
Final Test Loss: 0.4356
```

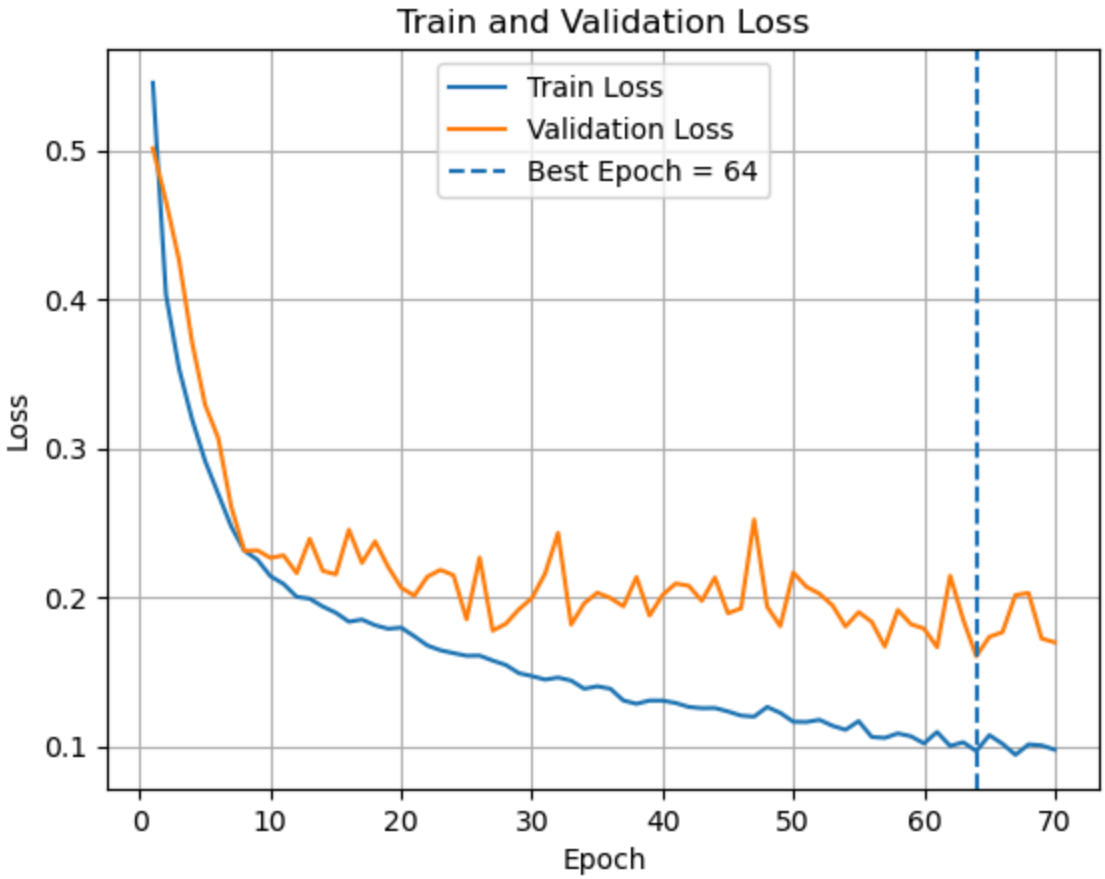

Train and Validation Loss
Loss
Epoch
Train Loss
Validation Loss
Best Epoch = 50
0.1
0.2
0.3
0.4
0.5
0
10
20
30
40
50
60
70

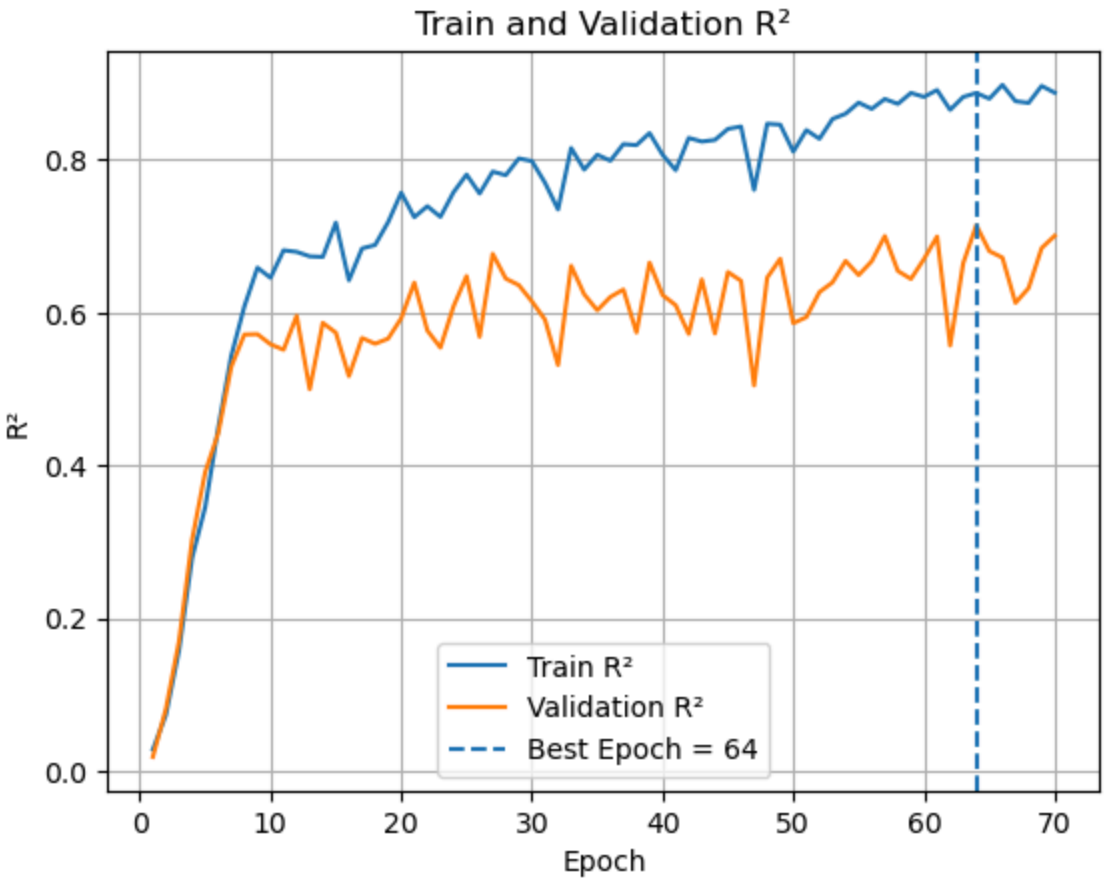

Train and Validation R²
R²
Epoch
Train R²
Validation R²
Best Epoch = 50
0.0
0.2
0.4
0.6
0.8
0
10
20
30
40
50
60
70

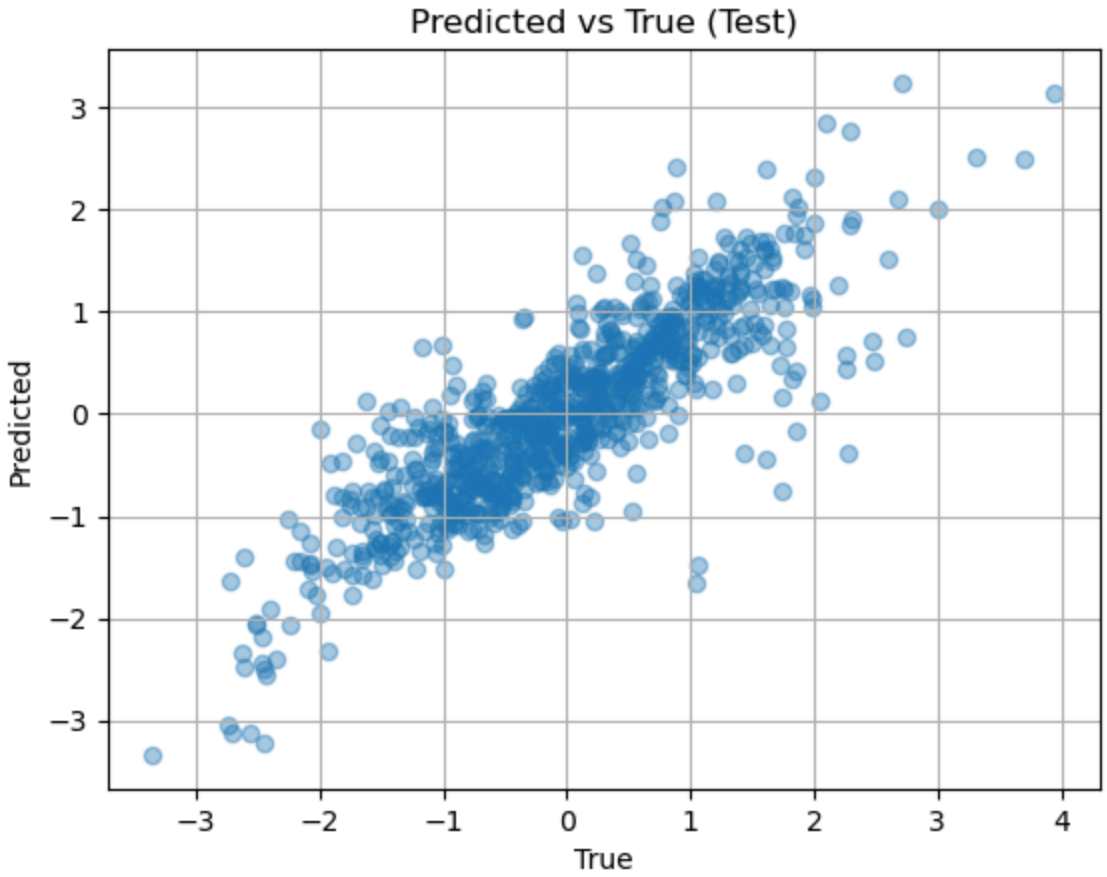


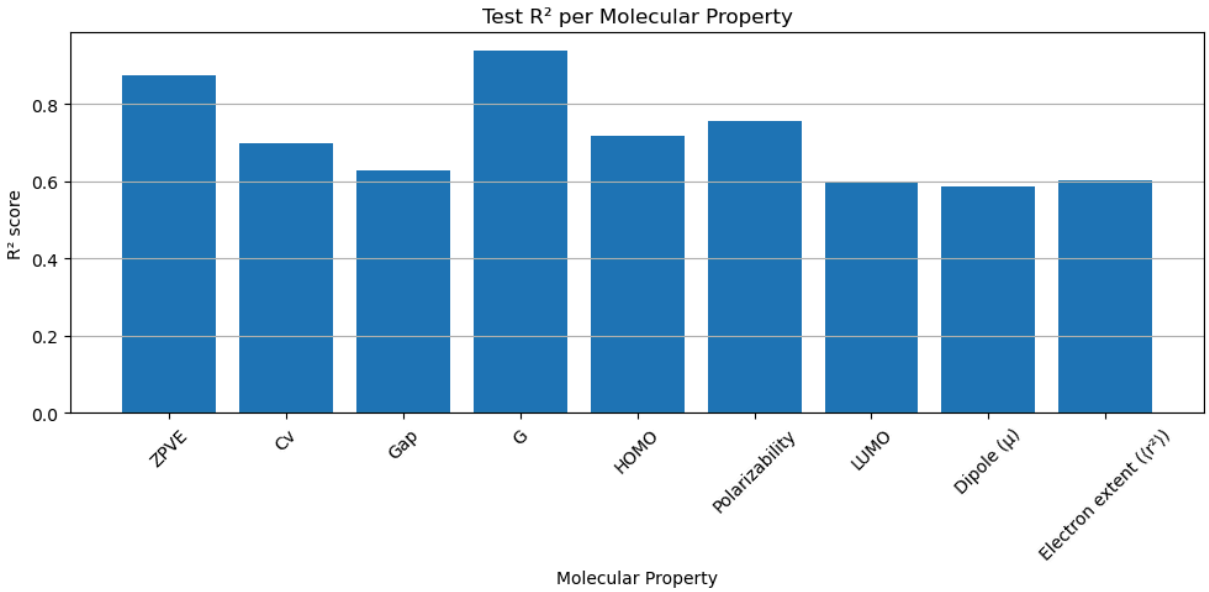



```
Best Epoch (by val R²): 50
Final Train R²: 0.8508
Final Train Loss: 0.0966
Final Val R²: 0.5582
Final Val Loss: 0.2178
Final Test R²: 0.5834
Final Test Loss: 0.2701
```


Train and Validation Loss
Train Loss
Validation Loss
Best Epoch = 64
Loss
Epoch
0.1
0.2
0.3
0.4
0.5
0
10
20
30
40
50
60
70


Train and Validation R²
Train R²
Validation R²
Best Epoch = 64
R²
Epoch
0.0
0.2
0.4
0.6
0.8
0
10
20
30
40
50
60
70






```
Best Epoch (by val R²): 64
Final Train R²: 0.8876
Final Train Loss: 0.0761
Final Val R²: 0.7144
Final Val Loss: 0.1606
Final Test R²: 0.7122
Final Test Loss: 0.1741
```

Note to self:

I might want to add like atomic radii and electronetivity to node get the r^2 val higher for:

- electron extent
- dipole moment

I might wanna add like moelcule mass, if it is conjugated, or some other electronic desriptor:

- Cv for heat capacity
- homo, lumo, and gap

This is something to return to after checkpoint three because I need to get the transformer done. Else, this is like 99% complete now.

(3) Create the smiles transformer that will predict the chemical reaction type. One transformer will use GNN derived information while the other will not and then compare.

Once we run the GNN model, we will need to then run the model to get the parameters from molecules we encounter in the USPTO dataset. The predictions will be then used as features to help the transformer perform the prediction for the reaction type being performed. Make these predictions with the GNN model and append them to the reaction_types_df as a list of the list of 9 properties. For example say 2 reactants and 1 product will add onto reaction_type_df two columns: reactant_preds and product_preds. The first will have two lists for the two molecules in the reactants, each list having the 9 features. The second will just have one list/molecule again with the 9 properties.

In [11]:
```
# This is used to get the graph structure to the model to get the 9 features
# Refer to @smiles_to_graph_alchemy
def smiles_to_graph_uspto(smiles, max_angles = 6):
    """
    NOTE: This is a new smiles to graph function
    meant to be used for smiles in the USPTO dataset to obtain their feature
    """
    mol = Chem.MolFromSmiles(smiles)
    if mol is None: return None

    # NOTE: Original data for GNN does not have H's
    # This is just to get rough angles and length of bonds as features
    # To use the GNN, then we will remove the H's and proceed
    mol = Chem.AddHs(mol)
    params = AllChem.ETKDGv3()
    params.randomSeed = 67

    # attempt no.1 of getting coords (least accurate, always will return som
    if AllChem.EmbedMolecule(mol, params) != 0: return None
    if AllChem.MMFFHasAllMoleculeParams(mol):
        try: AllChem.MMFFOptimizeMolecule(mol)
        except: pass # attempt no. 2 (most accurate for small organic molecu
    elif AllChem.UFFHasAllMoleculeParams(mol):
```

```
        try: AllChem.UFFOptimizeMolecule(mol)
        except: pass # attempt no. 3 (less accurate than MMFF but more accur

    conf = mol.GetConformer()
    coords = [
        [conf.GetAtomPosition(i).x,
         conf.GetAtomPosition(i).y,
         conf.GetAtomPosition(i).z]
        for i in range(mol.GetNumAtoms())]

    # get heavy atom indices BEFORE removing H's (important for correct inde
    # did not use Chem.RemoveAllHs(mol) else alignment with variable coords
    heavy_idx = [i for i, atom in enumerate(mol.GetAtoms()) if atom.GetAtomi
    coords = [coords[i] for i in heavy_idx]

    y_dummy = [0.0] * 9 # not needed, shove in dummy

    return smiles_to_graph_alchemy(
        smiles, # the original smiles string that had all the chiral hydroge
        coords,
        y_dummy,
        max_angles=max_angles
    )


@torch.no_grad()
def predict_smiles(model, smiles, device):
    model.eval()

    data = smiles_to_graph_uspto(smiles)
    if data is None: return None
    data = data.to(device)

    pred = model(data.x, data.edge_index, data.edge_attr, data.batch)
    return pred.reshape(1, -1).cpu().numpy().flatten().tolist()


def get_structure_preds(model, smiles_string, device):
    """
    Input
    =====
        smiles_string : string like "A.B.C" or "A"
    Output
    ======
        list of predictions (one per molecule)
        where each prediction is a list of 9 values or None
    """
    if pd.isna(smiles_string): return []
    smiles_list = [s.strip() for s in smiles_string.split(".") if s.strip()]

    return [predict_smiles(model, smi, device) for smi in smiles_list]
```

In [12]:
```
# For now, convert a subset of the reaction data just to work on the next st
# which is the transformer

N = 1000
```

```
# 100 of each of the 10 types of reaction to test out
subset = pd.concat([
    reaction_types_df[reaction_types_df["category"] == c]
    .sample(n=min(100, len(reaction_types_df[reaction_types_df["category"] =
    for c in reaction_types_df["category"].unique()
]).sample(frac=1, random_state=67).reset_index(drop=True)

subset["reactant_preds"] = [
    get_structure_preds(results["model"], x, device)
    for x in tqdm(subset["reactant"])
]

subset["product_preds"] = [
    get_structure_preds(results["model"], x, device)
    for x in tqdm(subset["product"])
]

display(subset)
```

```
100%|██████████| 926/926 [01:34<00:00,  9.84it/s]
100%|██████████| 926/926 [01:22<00:00, 11.23it/s]
```

		reactant
0	CN(C)c1cccc(CN)c1.COc1cc(OC)nc(-n2c(=S)[nH]c3c...	COc1cc(OC)nc(-n2c(=S)[n
1	CN.O=[N+]([O-])c1ccc(OCCCl)cc1	Cl
2	CS(=O)(=O)N[C@@H]1CCCC[C@H]1NS(C)(=O)=O.Cc1ccc...	
3	NN.O=C1CCCCC1C(=O)CC(F)(F)F	
4	C[C@@H]1CN(c2nc3c(N4CCOCC4)nc(-c4cnc(N)nc4)nc3...	C[C@H](O)C(=O)N1CCN(c2nc
...	...	
921	COC(=O)COc1ccc(NC(=O)OC(C)(C)C)cc1C.Cc1nc(-c2c...	COC(=O)COc1ccc(N(Cc2s
922	COC(=O)c1ccc(COC(Cn2ccnc2)c2ccc(F)cc2)cc1-c1cc...	O=C(O)c1ccc(COC(Cn2cc
923	O[C@@H]1C[C@@H]2CC3(OCCO3)[C@@H]2C1	O=C1C[C@@
924	CCOC(=O)C1=C(Nc2ccc(OCC(O)CO)cc2C)OCC1=O.O=Cc1...	CCOC(=O)C1=C(Nc2ccc(OCC
925	C=O.COc1c(O)cc(O)c2c(=O)cc(-c3ccccc3)oc12.OC1C...	COc1c(O)c(CN2CCC(O)CC2

926 rows × 5 columns

We can now get started with making the transformer herself. We will only have one transfomer model code with some boolean include_gnn that will include our curated GNN data.

In [13]:
```
# special tokens that will be used in transformer
pad_token = "[PAD]" # pad things out
unk_token = "[UNK]" # unknown

react_mol_token = "[RMOL]" # beginning of reactant molecule block
prod_mol_token = "[PMOL]"  # beginning of product molecule block
react_gnn_token = "[RGNN]" # token to be replaced by the reactant gnn vector
prod_gnn_token = "[PGNN]"  # token to be replaced by the product gnn vector
```

```
sep_token = "[SEP]"        # seperator between the reactant side and the prc


# token functions and smiles
def tokenize_smiles(smi):
    """
    Tokenizes SMILES while preserving:
        1. Bracketed groups like [NH4+], [O-]
        2. Multicharacter atoms (namely in our study, just Cl) (refer to @al
        3. Reaction Arrow (>>)
        4. Ring closures (%10)
        5. Everything else is single-character (C, H, O, F, ...)
    """
    tokens = []
    i = 0
    n = len(smi)

    while i < n: # have not used a while loop in years omg
        # this block preserves groups like [NH4+]
        if smi[i] == "[":
            j = i + 1
            while j < n and smi[j] != "]":
                j += 1
            if j < n:
                tokens.append(smi[i:j+1])
                i = j + 1
            else:
                tokens.append(smi[i])
                i += 1
        # whenever you see a reaction arrow
        elif i + 1 < n and smi[i:i+2] == ">>":
            tokens.append(">>")
            i += 2
        # whenever you close a ring (%10)
        elif i + 2 < n and smi[i] == "%" and smi[i+1:i+3].isdigit():
            tokens.append(smi[i:i+3])
            i += 3
        # whenever you see chlorine
        elif i + 1 < n and smi[i:i+2] in ["Cl"]:
            tokens.append(smi[i:i+2])
            i += 2
        # everything else just single char
        else:
            tokens.append(smi[i])
            i += 1

    return tokens


# helper functions to clean the gnn prediction vectors just in case
# make sure the gnn vec always is always length 9
def clean_pred_vec(pred, dim = 9):
    if pred is None: return [0.0] * dim
    if isinstance(pred, list) and len(pred) == dim: return [float(x) for x i
    return [0.0] * dim # if all goes bad at least
```

```
# make sure the list of gnn vecs is always a list and not None
def ensure_prediction_list(preds):
    if preds is None: return []
    if isinstance(preds, list): return preds
    return []


# build a mixed token sequence for one reaction
def build_reaction_tokens(
        reactant_smiles,
        product_smiles,
        reactant_preds = None,
        product_preds = None,
        include_gnn_vec = False,
):
    """
    Output
    ======
    include_gnn_vec = False
        uses the smiles strings only
    include_gnn_vec = True
        uses the smiles strings and gnn vec

    Examples
    ========
    SMILES-only:
        [RMOL] A [RMOL] B [SEP] [PMOL] C
    GNN mode:
        [RMOL] [RGNN] A [RMOL] [RGNN] B [SEP] [PMOL] [PGNN] C
    """

    reactant_parts = [s.strip() for s in reactant_smiles.split(".") if s.str
    product_parts = [s.strip() for s in product_smiles.split(".") if s.strip

    if include_gnn_vec:
        reactant_preds = ensure_prediction_list(reactant_preds)
        product_preds = ensure_prediction_list(product_preds)

    tokens = []
    gnn_mask = []
    gnn_values = []

    # reactants
    for i, smi in enumerate(reactant_parts):

        tokens.append(react_mol_token)
        gnn_mask.append(0)
        gnn_values.append([0.0] * 9)

        if include_gnn_vec:
            mol_pred = clean_pred_vec(
                reactant_preds[i] if i < len(reactant_preds) else None
            )
            tokens.append(react_gnn_token)
            gnn_mask.append(1) # is a gnn vec
```

```
            gnn_values.append(mol_pred)

        for tok in tokenize_smiles(smi):
            tokens.append(tok)
            gnn_mask.append(0) # is not a gnn vec
            gnn_values.append([0.0] * 9)

    # rxn arrow seperator
    tokens.append(sep_token)
    gnn_mask.append(0)
    gnn_values.append([0.0] * 9)

    # products
    for i, smi in enumerate(product_parts):

        tokens.append(prod_mol_token)
        gnn_mask.append(0) # is not a gnn vec
        gnn_values.append([0.0] * 9)

        if include_gnn_vec:
            mol_pred = clean_pred_vec(
                product_preds[i] if i < len(product_preds) else None
            )

            tokens.append(prod_gnn_token)
            gnn_mask.append(1) # is a gnn vec
            gnn_values.append(mol_pred)

        for tok in tokenize_smiles(smi):
            tokens.append(tok)
            gnn_mask.append(0) # is not a gnn vec
            gnn_values.append([0.0] * 9)

    return tokens, gnn_mask, gnn_values


class ReactionTokenizer:
    def __init__(self):
        # token2index is the main vocab dict mapping token to int id
        self.token2idx = {
            pad_token: 0,
            unk_token: 1,
            react_mol_token: 2,
            prod_mol_token: 3,
            react_gnn_token: 4,
            prod_gnn_token: 5,
            sep_token: 6,
        }

        # reverse mapping: int id -> token
        self.idx2token = {idx: tok for tok, idx in self.token2idx.items()}

    def add_token(self, token):
        if token not in self.token2idx:
            idx = len(self.token2idx)
            self.token2idx[token] = idx
```

```
            self.idx2token[idx] = token

    def build_vocab(self, df):
        # add all smile tokens from react and prod
        for _, row in df.iterrows():
            for tok in tokenize_smiles(row["reactant"]): self.add_token(tok)
            for tok in tokenize_smiles(row["product"]): self.add_token(tok)

    def encode_tokens(self, tokens):
        # return an array integers for the token
        # if not return the int for the unknown (.get(...))
        unk_idx = self.token2idx[unk_token]
        return [self.token2idx.get(tok, unk_idx) for tok in tokens]

    def decode_ids(self, ids):
        # return an array of the ids of from the idx
        # if not return the idx for the unknown (.get(...))
        return [self.idx2token.get(i, unk_token) for i in ids]

    @property # decorator for a class variable
    # example: tokenizer.vocab_size
    def vocab_size(self):
        return len(self.token2idx)


# convert df of reactions I made into something pytorch can train on
class ReactionDataset(Dataset):
    def __init__(self, df, tokenizer, include_gnn_vec = False):
        self.df = df.reset_index(drop=True)
        self.tokenizer = tokenizer
        self.include_gnn_vec = include_gnn_vec
        self.labels = self.df["category"].tolist()

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]

        tokens, gnn_mask, gnn_values = build_reaction_tokens(
            reactant_smiles=row["reactant"],
            product_smiles=row["product"],
            reactant_preds=row.get("reactant_preds", None),
            product_preds=row.get("product_preds", None),
            include_gnn_vec=self.include_gnn_vec
        )

        input_ids = torch.tensor(
            self.tokenizer.encode_tokens(tokens),
            dtype=torch.long
        )

        # 1 means real token, 0 means padding
        attention_mask = torch.ones(len(tokens), dtype=torch.long)

        # marks which positions correspond to [RGNN] or [PGNN]
```

```
        gnn_mask = torch.tensor(gnn_mask, dtype=torch.bool)

        # per-token 9-dim GNN
        gnn_values = torch.tensor(gnn_values, dtype=torch.float)

        label = torch.tensor(int(self.labels[idx]), dtype=torch.long)

        return {
            "input_ids": input_ids,
            "attention_mask": attention_mask,
            "gnn_mask": gnn_mask,
            "gnn_values": gnn_values,
            "label": label,
        }


def reaction_collate_fn(batch):
    """
    Pad variable-length reaction sequences into one batch.
    Make sure that they are all the same length for batching.
    """
    input_ids = [item["input_ids"] for item in batch]
    attention_mask = [item["attention_mask"] for item in batch]
    gnn_mask = [item["gnn_mask"] for item in batch]
    gnn_values = [item["gnn_values"] for item in batch]
    labels = torch.stack([item["label"] for item in batch])

    input_ids = pad_sequence(input_ids, batch_first=True, padding_value=0)
    attention_mask = pad_sequence(attention_mask, batch_first=True, padding_
    gnn_mask = pad_sequence(gnn_mask, batch_first=True, padding_value=0)
    gnn_values = pad_sequence(gnn_values, batch_first=True, padding_value=0.

    return {
        "input_ids": input_ids,
        "attention_mask": attention_mask,
        "gnn_mask": gnn_mask,
        "gnn_values": gnn_values,
        "label": labels,
    }


# positional encoder or else CON = NOC
class PositionalEncoding(nn.Module):
    def __init__(self, d_model, dropout = 0.1, max_len = 512):
        super().__init__()
        self.dropout = nn.Dropout(dropout)

        # initialize
        pe = torch.zeros(max_len, d_model)
        position = torch.arange(max_len).unsqueeze(1).float() # [max_len, 1]
        frequency = torch.exp(
            torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_mo
        )

        # even is sine and odd is cosine, then add a dim for batch
        pe[:, 0::2] = torch.sin(position * frequency)
```

```
        pe[:, 1::2] = torch.cos(position * frequency)
        pe = pe.unsqueeze(0) # [1, max_len, d_model]
        self.register_buffer("pe", pe)

    def forward(self, x): # x is [batch_len, seq_len, d_model]
        x = x + self.pe[:, :x.size(1), :] # positional encoding to embedding
        return self.dropout(x)


# finally, for the actual transformer itself
class ReactionTransformer(nn.Module):
    def __init__(
            self,
            vocab_size,
            num_classes,
            d_model = 128,
            nhead = 4,
            num_layers = 2,
            dim_feedforward = 256,
            dropout = 0.1,
            max_len = 512,
            gnn_dim = 9,
            include_gnn_vec = False,
    ):
        super().__init__()

        self.include_gnn_vec = include_gnn_vec
        self.token_embedding = nn.Embedding(vocab_size, d_model, padding_idx
        self.positional_encoding = PositionalEncoding(d_model, dropout=dropo

        # project gnn vec into same embedding size as token embeds
        self.gnn_projection = nn.Sequential(
            nn.Linear(gnn_dim, d_model),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(d_model, d_model),
        )

        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_feedforward,
            dropout=dropout,
            batch_first=True,
        )

        self.encoder = nn.TransformerEncoder(
            encoder_layer,
            num_layers=num_layers,
            enable_nested_tensor=False # experimental does not always work
        )

        # then finally a classifier for one of the reaction types
        self.classifier = nn.Sequential(
            nn.Linear(d_model, d_model),
            nn.ReLU(),
```

```
            nn.Dropout(dropout),
            nn.Linear(d_model, num_classes),
        )

    def forward(self, input_ids, attention_mask, gnn_mask = None, gnn_values
        x = self.token_embedding(input_ids)

        # if gnn vec, shove it in
        if self.include_gnn_vec:
            gnn_emb = self.gnn_projection(gnn_values)
            x = torch.where(gnn_mask.unsqueeze(-1), gnn_emb, x)

        x = self.positional_encoding(x) # order matters

        # make transformer ignore where attention mask == 0
        # which are fake tokens added for batching
        key_padding_mask = (attention_mask == 0)
        x = self.encoder(x, src_key_padding_mask = key_padding_mask)

        # mean pool only w/ real tokens
        mask = attention_mask.unsqueeze(-1).float()
        pooled = (x * mask).sum(dim=1) / mask.sum(dim=1).clamp_min(1.0)

        logits = self.classifier(pooled)

        return logits
```

In [14]:

```
from sklearn.metrics import f1_score, confusion_matrix, ConfusionMatrixDispl
from torch.utils.data import Dataset, DataLoader # for the transformer

def train_and_test_reaction_transformer(
        df,
        test_size=0.2,
        val_size=0.2,
        batch_size=32,
        epochs=20,
        d_model=128,
        nhead=4,
        num_layers=2,
        dim_feedforward=256,
        dropout=0.1,
        max_len=512,
        gnn_dim=9,
        include_gnn_vec=False,
        lr=1e-3,
        device="cpu",
        random_state=67,
        show_progress=True,
        verbose=True,
        make_plots=True,
):
    """
    Final training pipeline for the reaction transformer.

    Inputs
    ======
```

```
        df               : df with reactant, product, category and optionall
        test_size        : fraction for held-out test set
        val_size         : fraction of the remaining train split used for va
        batch_size       : dataloader batch size
        epochs           : training epochs
        d_model          : transformer embedding size
        nhead            : number of attention heads
        num_layers       : number of transformer encoder layers
        dim_feedforward  : hidden size in feedforward block
        dropout          : dropout probability
        max_len          : maximum positional encoding length
        gnn_dim          : size of each molecule-level gnn vector
        include_gnn_vec  : whether to inject GNN vectors
        lr               : learning rate
        device           : "cpu", "cuda", or "mps"
        random_state     : reproducibility
        show_progress    : print per-epoch metrics
        verbose          : print final metrics
        make_plots       : make learning curves + confusion matrix

    Outputs
    =======
        dictionary containing:
            - model
            - tokenizer
            - train/val curves
            - best epoch
            - final val/test metrics
            - confusion matrix
            - predictions / true labels
    """

    # split into train / val / test
    train_df, test_df = train_test_split(
        df,
        test_size=test_size,
        random_state=random_state,
        stratify=df["category"]
    )

    train_df, val_df = train_test_split(
        train_df,
        test_size=val_size,
        random_state=random_state,
        stratify=train_df["category"]
    )

    # tokenizer from TRAIN only
    tokenizer = ReactionTokenizer()
    tokenizer.build_vocab(train_df)

    # datasets
    train_data = ReactionDataset(
        df=train_df,
        tokenizer=tokenizer,
        include_gnn_vec=include_gnn_vec
```

```
)

val_data = ReactionDataset(
    df=val_df,
    tokenizer=tokenizer,
    include_gnn_vec=include_gnn_vec
)

test_data = ReactionDataset(
    df=test_df,
    tokenizer=tokenizer,
    include_gnn_vec=include_gnn_vec
)

# dataloaders
train_loader = DataLoader(
    train_data,
    batch_size=batch_size,
    shuffle=True,
    collate_fn=reaction_collate_fn
)

val_loader = DataLoader(
    val_data,
    batch_size=batch_size,
    shuffle=False,
    collate_fn=reaction_collate_fn
)

test_loader = DataLoader(
    test_data,
    batch_size=batch_size,
    shuffle=False,
    collate_fn=reaction_collate_fn
)

# model
num_classes = int(df["category"].nunique())

model = ReactionTransformer(
    vocab_size=tokenizer.vocab_size,
    num_classes=num_classes,
    d_model=d_model,
    nhead=nhead,
    num_layers=num_layers,
    dim_feedforward=dim_feedforward,
    dropout=dropout,
    max_len=max_len,
    gnn_dim=gnn_dim,
    include_gnn_vec=include_gnn_vec,
).to(device)

optimizer = torch.optim.Adam(model.parameters(), lr=lr)
loss_function = nn.CrossEntropyLoss()

# helpers
```

```
    def run_one_epoch(loader, train_mode=True):
        if train_mode: model.train()
        else: model.eval()

        total_loss = 0.0
        all_preds = []
        all_targets = []

        context = torch.enable_grad() if train_mode else torch.no_grad()

        with context:
            for batch in loader:
                input_ids = batch["input_ids"].to(device)
                attention_mask = batch["attention_mask"].to(device)
                gnn_mask = batch["gnn_mask"].to(device)
                gnn_values = batch["gnn_values"].to(device)
                labels = batch["label"].to(device)

                if train_mode: optimizer.zero_grad()

                logits = model(
                    input_ids=input_ids,
                    attention_mask=attention_mask,
                    gnn_mask=gnn_mask,
                    gnn_values=gnn_values
                )

                loss = loss_function(logits, labels)

                if train_mode:
                    loss.backward()
                    optimizer.step()

                total_loss += loss.item()

                preds = torch.argmax(logits, dim=1)
                all_preds.append(preds.cpu())
                all_targets.append(labels.cpu())

        all_preds = torch.cat(all_preds).numpy()
        all_targets = torch.cat(all_targets).numpy()

        acc = accuracy_score(all_targets, all_preds)
        f1 = f1_score(all_targets, all_preds, average="macro")

        return total_loss / len(loader), acc, f1, all_preds, all_targets

    # training loop
    train_losses = []
    val_losses = []
    train_accs = []
    val_accs = []
    train_f1s = []
    val_f1s = []

    best_val_f1 = -float("inf")
```

```
    best_epoch = None
    best_state_dict = None

    for epoch in range(1, epochs + 1):
        train_loss, train_acc, train_f1, _, _ = run_one_epoch(train_loader,
        val_loss, val_acc, val_f1, _, _ = run_one_epoch(val_loader, train_mo

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_accs.append(train_acc)
        val_accs.append(val_acc)
        train_f1s.append(train_f1)
        val_f1s.append(val_f1)

        if val_f1 > best_val_f1:
            best_val_f1 = val_f1
            best_epoch = epoch
            best_state_dict = {k: v.detach().cpu().clone() for k, v in model

        if show_progress:
            print(
                f"Epoch {epoch}: "
                f"train_loss={train_loss:.4f}, "
                f"train_acc={train_acc:.4f}, "
                f"train_f1={train_f1:.4f}, "
                f"val_loss={val_loss:.4f}, "
                f"val_acc={val_acc:.4f}, "
                f"val_f1={val_f1:.4f}"
            )

    # best model by val F1
    if best_state_dict is not None: model.load_state_dict(best_state_dict)

    # final evals
    train_loss_final, train_acc_final, train_f1_final, _, _ = run_one_epoch(
    val_loss_final, val_acc_final, val_f1_final, _, _ = run_one_epoch(val_lo
    test_loss_final, test_acc_final, test_f1_final, test_preds, test_targets

    # confusion matrix
    labels_sorted = sorted(df["category"].unique().tolist())
    cm = confusion_matrix(test_targets, test_preds, labels=labels_sorted)

    # plots
    if make_plots:
        plt.figure()
        plt.plot(range(1, epochs + 1), train_losses, label="Train Loss")
        plt.plot(range(1, epochs + 1), val_losses, label="Val Loss")
        plt.axvline(best_epoch, linestyle="--", label=f"Best Epoch = {best_e
        plt.xlabel("Epoch")
        plt.ylabel("Loss")
        plt.title("Train and Validation Loss")
        plt.legend()
        plt.grid()
        plt.show()

        plt.figure()
```

```
        plt.plot(range(1, epochs + 1), train_accs, label="Train Accuracy")
        plt.plot(range(1, epochs + 1), val_accs, label="Val Accuracy")
        plt.axvline(best_epoch, linestyle="--", label=f"Best Epoch = {best_e
        plt.xlabel("Epoch")
        plt.ylabel("Accuracy")
        plt.title("Train and Validation Accuracy")
        plt.legend()
        plt.grid()
        plt.show()

        plt.figure()
        plt.plot(range(1, epochs + 1), train_f1s, label="Train Macro F1")
        plt.plot(range(1, epochs + 1), val_f1s, label="Val Macro F1")
        plt.axvline(best_epoch, linestyle="--", label=f"Best Epoch = {best_e
        plt.xlabel("Epoch")
        plt.ylabel("Macro F1")
        plt.title("Train and Validation Macro F1")
        plt.legend()
        plt.grid()
        plt.show()

        fig, ax = plt.subplots(figsize=(8, 8))
        disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=la
        disp.plot(ax=ax, cmap="Blues", colorbar=False, values_format="d")
        plt.title("Test Confusion Matrix")
        plt.show()

    # verbose summary
    if verbose:
        print(f"Best Epoch (by Val Macro F1): {best_epoch}")
        print(f"Train Loss: {train_loss_final:.4f}")
        print(f"Train Accuracy: {train_acc_final:.4f}")
        print(f"Train Macro F1: {train_f1_final:.4f}")
        print(f"Val Loss: {val_loss_final:.4f}")
        print(f"Val Accuracy: {val_acc_final:.4f}")
        print(f"Val Macro F1: {val_f1_final:.4f}")
        print(f"Test Loss: {test_loss_final:.4f}")
        print(f"Test Accuracy: {test_acc_final:.4f}")
        print(f"Test Macro F1: {test_f1_final:.4f}")
        print(f"Overall Percentage Correct: {100 * test_acc_final:.2f}%")

    return {
        "model": model,
        "tokenizer": tokenizer,
        "best_epoch": best_epoch,

        "train_losses": train_losses,
        "val_losses": val_losses,
        "train_accs": train_accs,
        "val_accs": val_accs,
        "train_f1s": train_f1s,
        "val_f1s": val_f1s,

        "train_loss": train_loss_final,
        "train_acc": train_acc_final,
        "train_f1": train_f1_final,
```

```
        "val_loss": val_loss_final,
        "val_acc": val_acc_final,
        "val_f1": val_f1_final,

        "test_loss": test_loss_final,
        "test_acc": test_acc_final,
        "test_f1": test_f1_final,
        "test_percent_correct": 100 * test_acc_final,

        "confusion_matrix": cm,
        "test_preds": test_preds,
        "test_targets": test_targets,
        "label_order": labels_sorted,

        "train_df": train_df,
        "val_df": val_df,
        "test_df": test_df,
    }
```

In [15]:

```
from sklearn.base import BaseEstimator, ClassifierMixin
from sklearn.metrics import f1_score
from sklearn.model_selection import StratifiedKFold

from scipy.stats import randint, loguniform, uniform
from torch.utils.data import DataLoader as TorchDataLoader

# look for decent hyperparameters
class ReactionTransformerClassifier(BaseEstimator, ClassifierMixin):
    def __init__(
        self,
        include_gnn_vec=False,
        batch_size=32,
        epochs=10,
        d_model=128,
        nhead=4,
        num_layers=2,
        dim_feedforward=256,
        dropout=0.1,
        max_len=512,
        gnn_dim=9,
        lr=1e-3,
        device="cpu",
        random_state=67,
    ):
        self.include_gnn_vec = include_gnn_vec
        self.batch_size = batch_size
        self.epochs = epochs
        self.d_model = d_model
        self.nhead = nhead
        self.num_layers = num_layers
        self.dim_feedforward = dim_feedforward
        self.dropout = dropout
        self.max_len = max_len
        self.gnn_dim = gnn_dim
        self.lr = lr
```

```
        self.device = device
        self.random_state = random_state

    def _build_dataframe(self, X, y=None):
        if isinstance(X, pd.DataFrame):
            df = X.copy()
        else:
            df = pd.DataFrame(X).copy()

        if y is not None:
            df["category"] = np.asarray(y)

        return df.reset_index(drop=True)

    def fit(self, X, y):
        torch.manual_seed(self.random_state)
        np.random.seed(self.random_state)

        df = self._build_dataframe(X, y)

        self.classes_ = np.sort(df["category"].unique())
        self.num_classes_ = len(self.classes_)

        self.tokenizer_ = ReactionTokenizer()
        self.tokenizer_.build_vocab(df)

        train_data = ReactionDataset(
            df=df,
            tokenizer=self.tokenizer_,
            include_gnn_vec=self.include_gnn_vec
        )

        train_loader = TorchDataLoader(
            train_data,
            batch_size=int(self.batch_size),
            shuffle=True,
            collate_fn=reaction_collate_fn
        )

        self.model_ = ReactionTransformer(
            vocab_size=self.tokenizer_.vocab_size,
            num_classes=self.num_classes_,
            d_model=int(self.d_model),
            nhead=int(self.nhead),
            num_layers=int(self.num_layers),
            dim_feedforward=int(self.dim_feedforward),
            dropout=float(self.dropout),
            max_len=int(self.max_len),
            gnn_dim=int(self.gnn_dim),
            include_gnn_vec=self.include_gnn_vec,
        ).to(self.device)

        optimizer = torch.optim.Adam(self.model_.parameters(), lr=float(self
        loss_function = nn.CrossEntropyLoss()

        self.model_.train()
```

```
        for _ in range(int(self.epochs)):
            for batch in train_loader:
                input_ids = batch["input_ids"].to(self.device)
                attention_mask = batch["attention_mask"].to(self.device)
                gnn_mask = batch["gnn_mask"].to(self.device)
                gnn_values = batch["gnn_values"].to(self.device)
                labels = batch["label"].to(self.device)

                optimizer.zero_grad()

                logits = self.model_(
                    input_ids=input_ids,
                    attention_mask=attention_mask,
                    gnn_mask=gnn_mask,
                    gnn_values=gnn_values,
                )

                loss = loss_function(logits, labels)
                loss.backward()
                optimizer.step()

        return self

    @torch.no_grad()
    def predict(self, X):
        df = self._build_dataframe(X)

        data = ReactionDataset(
            df=df,
            tokenizer=self.tokenizer_,
            include_gnn_vec=self.include_gnn_vec
        )

        loader = TorchDataLoader(
            data,
            batch_size=int(self.batch_size),
            shuffle=False,
            collate_fn=reaction_collate_fn
        )

        self.model_.eval()
        all_preds = []

        for batch in loader:
            input_ids = batch["input_ids"].to(self.device)
            attention_mask = batch["attention_mask"].to(self.device)
            gnn_mask = batch["gnn_mask"].to(self.device)
            gnn_values = batch["gnn_values"].to(self.device)

            logits = self.model_(
                input_ids=input_ids,
                attention_mask=attention_mask,
                gnn_mask=gnn_mask,
                gnn_values=gnn_values,
            )
```

```
            preds = torch.argmax(logits, dim=1)
            all_preds.append(preds.cpu().numpy())

        return np.concatenate(all_preds)

    def score(self, X, y):
        preds = self.predict(X)
        return f1_score(y, preds, average="macro")


X = subset[["reactant", "product", "reactant_preds", "product_preds"]].copy(
y = subset["category"].values

param_dist = {
    "batch_size": randint(16, 65),
    "d_model": randint(64, 193),
    "nhead": [2, 4, 8],
    "num_layers": randint(1, 5),
    "dim_feedforward": randint(128, 513),
    "dropout": uniform(0.05, 0.35),
    "lr": loguniform(1e-4, 3e-3),
}

base_estimator = ReactionTransformerClassifier(
    include_gnn_vec=True,
    gnn_dim=9,
    device=device,
    random_state=67,
)

cv = StratifiedKFold(
    n_splits=3,
    shuffle=True,
    random_state=67
)

search = HalvingRandomSearchCV(
    estimator=base_estimator,
    param_distributions=param_dist,
    factor=2,
    resource="epochs",
    max_resources=20,
    min_resources=5,
    cv=cv,
    scoring="f1_macro",
    random_state=67,
    n_jobs=1,
    verbose=1
)

search.fit(X, y)

print(search.best_params_)
print(search.best_score_)

cv_results_df = pd.DataFrame(search.cv_results_)
```

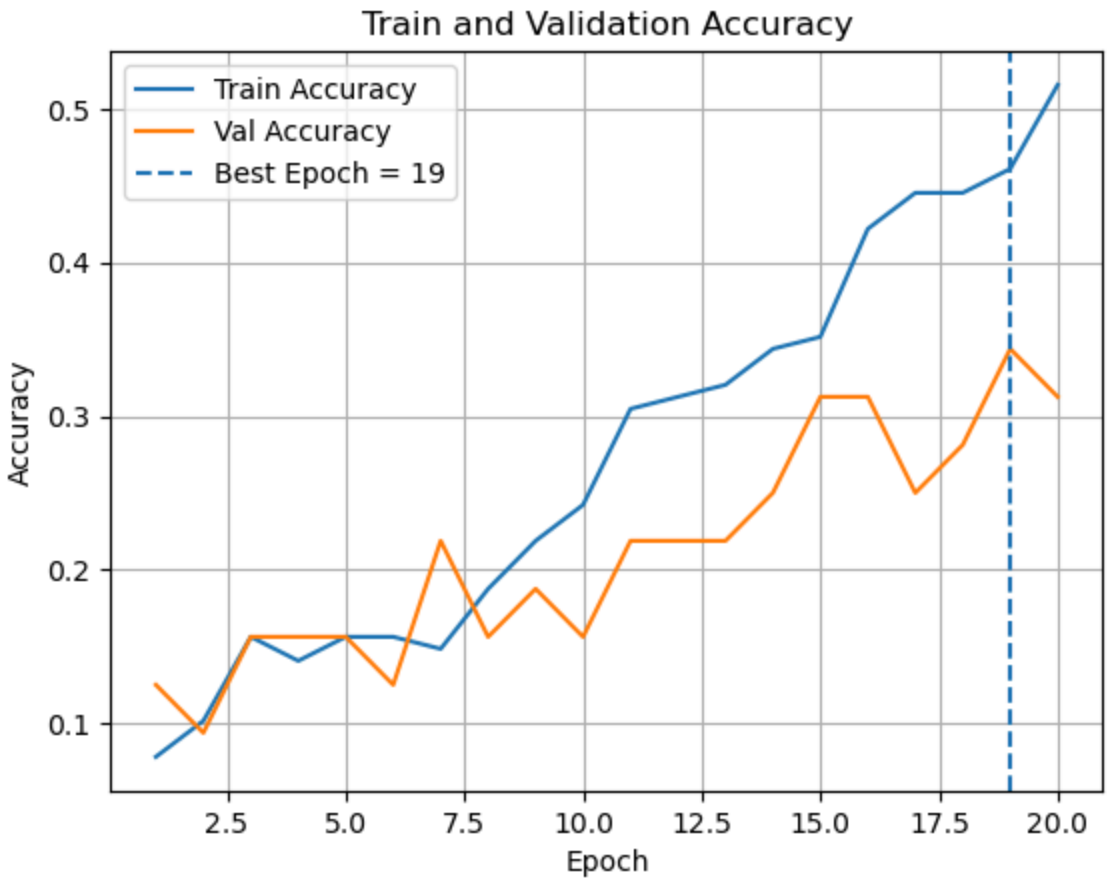
```
cv_results_df = cv_results_df[[
    "mean_test_score",
    "std_test_score",
    "rank_test_score",
    "param_epochs",
    "param_batch_size",
    "param_d_model",
    "param_nhead",
    "param_num_layers",
    "param_dim_feedforward",
    "param_dropout",
    "param_lr",
]].rename(columns={
    "mean_test_score": "f1_macro",
    "std_test_score": "std",
    "rank_test_score": "rank",
    "param_epochs": "epochs",
    "param_batch_size": "batch_size",
    "param_d_model": "d_model",
    "param_nhead": "nhead",
    "param_num_layers": "num_layers",
    "param_dim_feedforward": "dim_feedforward",
    "param_dropout": "dropout",
    "param_lr": "lr",
})

display(cv_results_df.sort_values("rank").reset_index(drop=True))
```

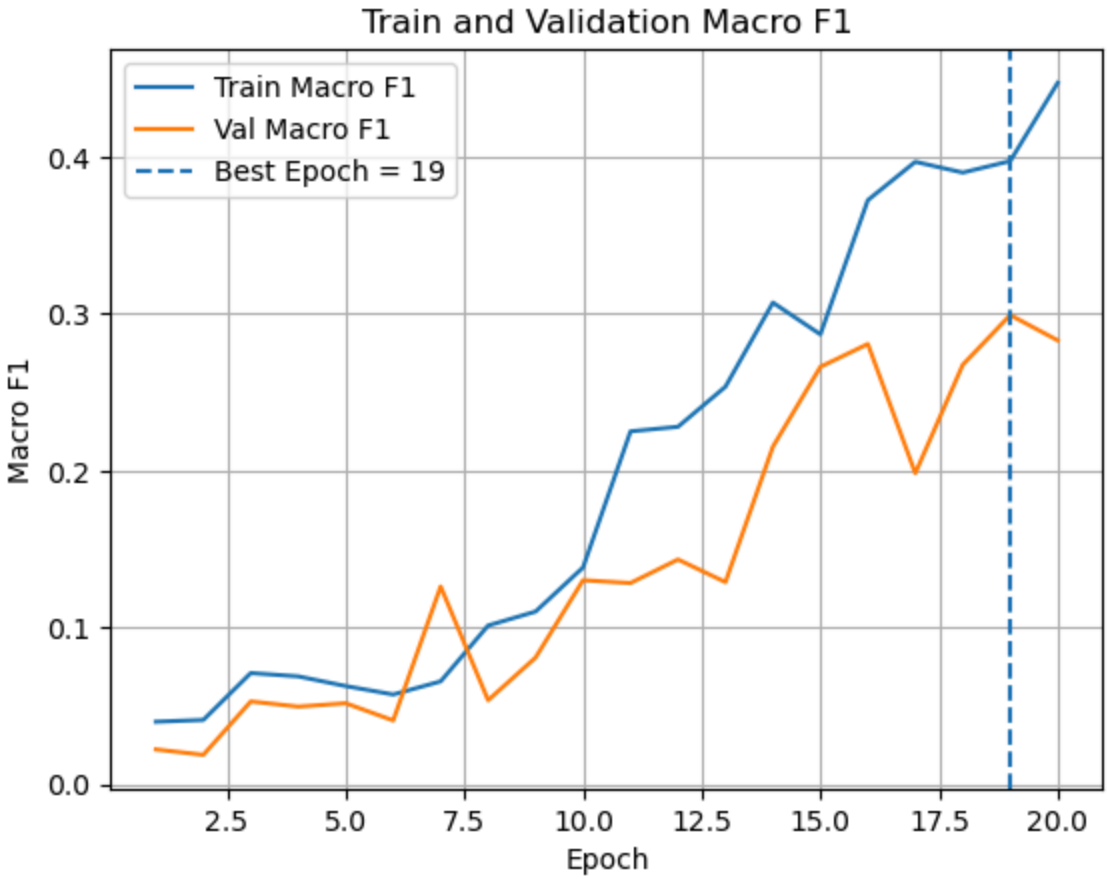
```
n_iterations: 3
n_required_iterations: 3
n_possible_iterations: 3
min_resources_: 5
max_resources_: 20
aggressive_elimination: False
factor: 2
----------
iter: 0
n_candidates: 4
n_resources: 5
Fitting 3 folds for each of 4 candidates, totalling 12 fits
----------
iter: 1
n_candidates: 2
n_resources: 10
Fitting 3 folds for each of 2 candidates, totalling 6 fits
----------
iter: 2
n_candidates: 1
n_resources: 20
Fitting 3 folds for each of 1 candidates, totalling 3 fits
{'batch_size': 55, 'd_model': 98, 'dim_feedforward': 437, 'dropout': 0.19335
597845395652, 'lr': 0.00040639636307460543, 'nhead': 2, 'num_layers': 3, 'ep
ochs': 20}
nan
```

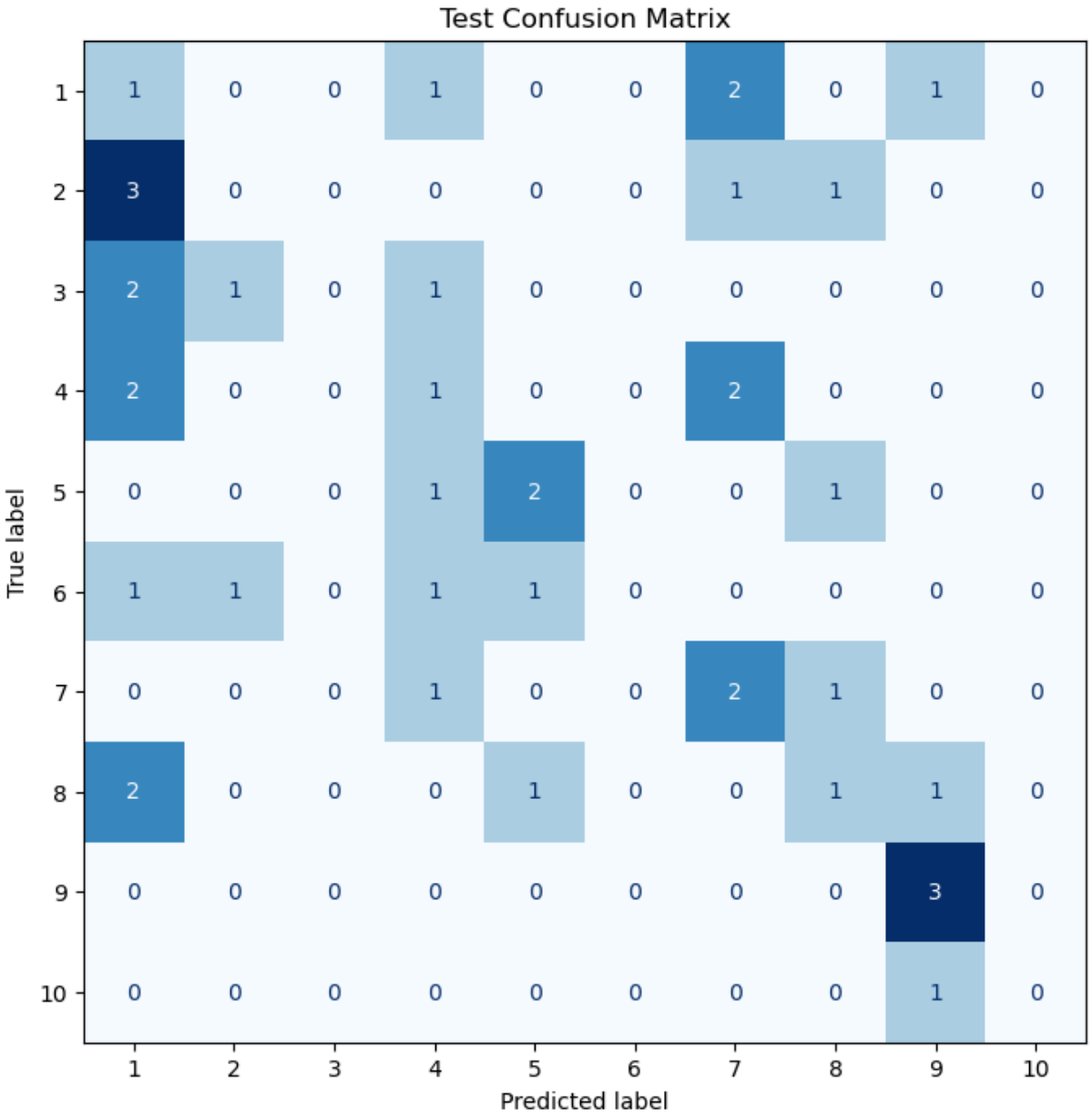
	f1_macro	std	rank	epochs	batch_size	d_model	nhead	num_layers	dim_feedfo
0	NaN	NaN	1	5	19	117	8	2	
1	NaN	NaN	1	5	61	99	8	1	
2	NaN	NaN	1	5	47	119	8	4	
3	NaN	NaN	1	5	55	98	2	3	
4	NaN	NaN	1	10	47	119	8	4	
5	NaN	NaN	1	10	55	98	2	3	
6	NaN	NaN	1	20	55	98	2	3	

In [16]:
```
for sample in [200, 500, 1000]:
    results = train_and_test_reaction_transformer(
        df=subset[:sample],
        test_size=0.2,
        val_size=0.2,
        batch_size=32,
        epochs=20,
        d_model=128,
        nhead=4,
        num_layers=2,
        dim_feedforward=256,
        dropout=0.1,
        max_len=512,
        gnn_dim=9,
        include_gnn_vec=True, # WITH GNN VEC
        lr=1e-3,
        device=device,
        random_state=67,
        show_progress= False, # used when stress testing, eyes will bleed
        verbose=True,
        make_plots=True,
    )
```

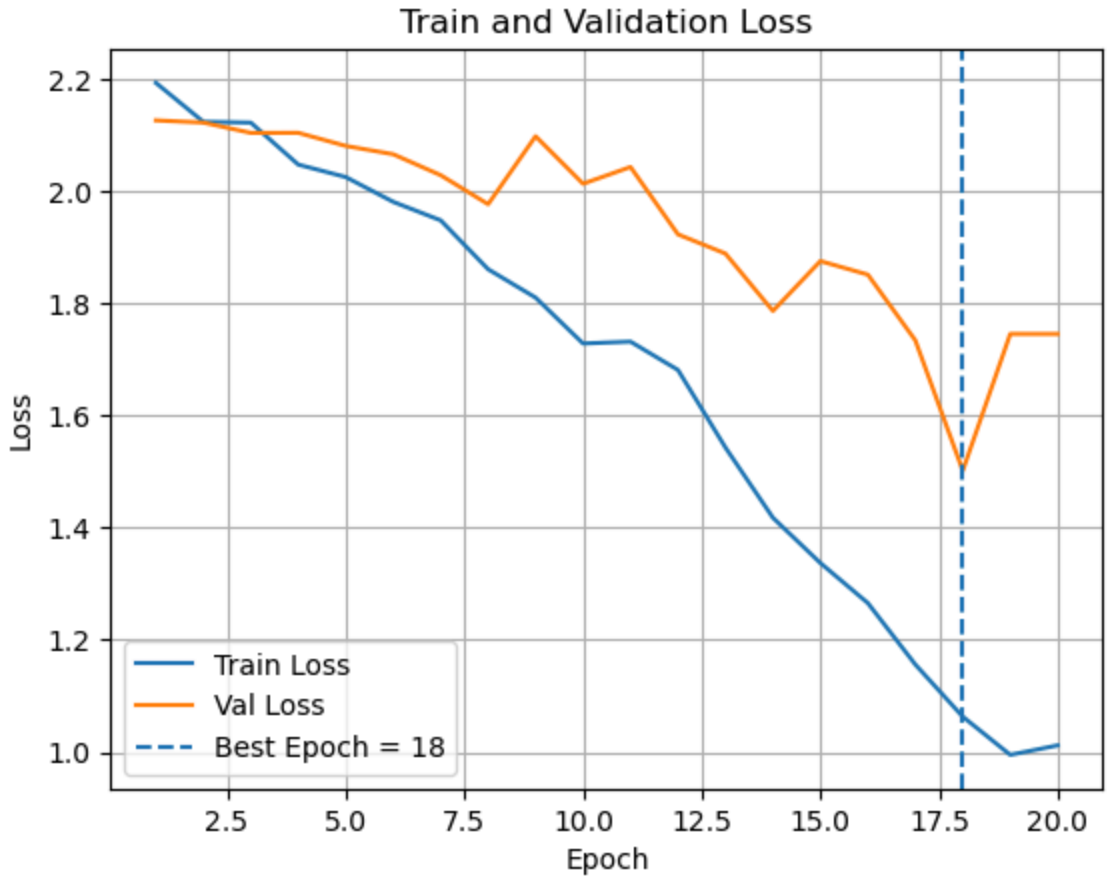

Train and Validation Loss
Loss
Epoch
Train Loss
Val Loss
Best Epoch = 19
2.2
2.0
1.8
1.6
1.4
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

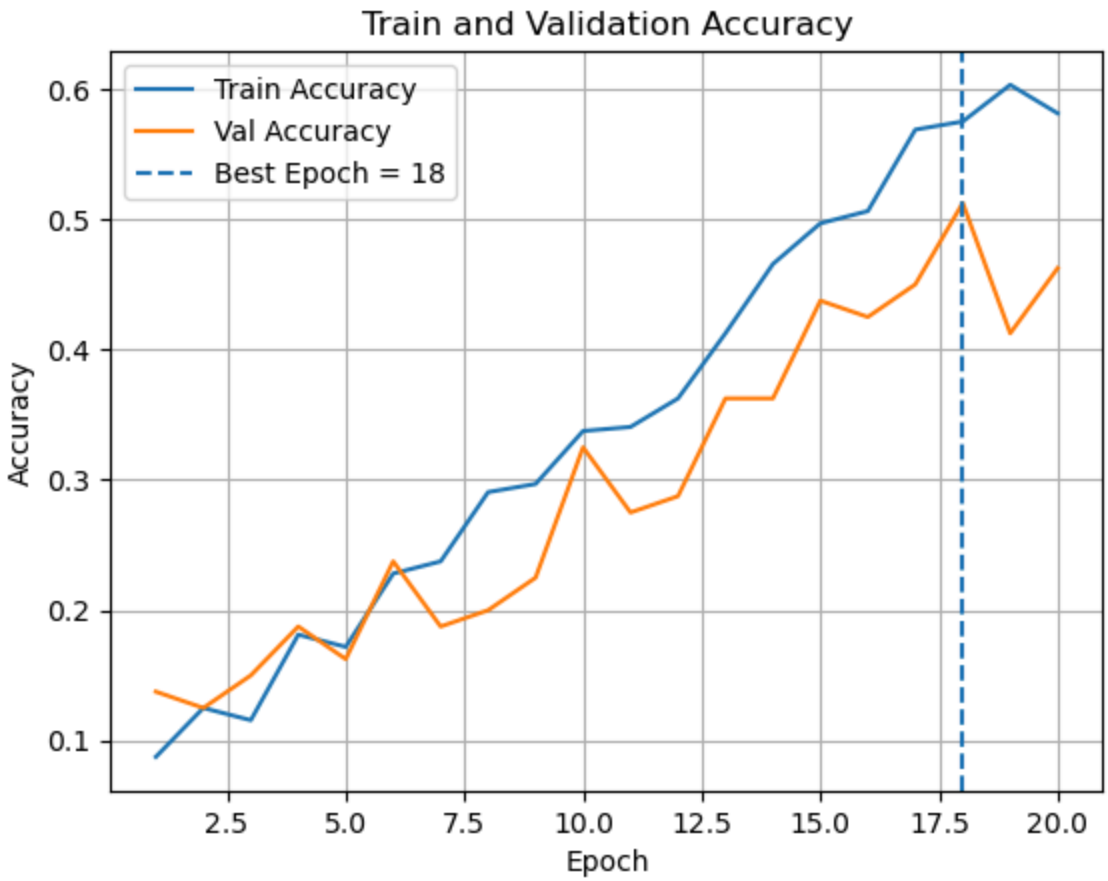

Train and Validation Accuracy
Accuracy
Epoch
Train Accuracy
Val Accuracy
Best Epoch = 19
0.5
0.4
0.3
0.2
0.1
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

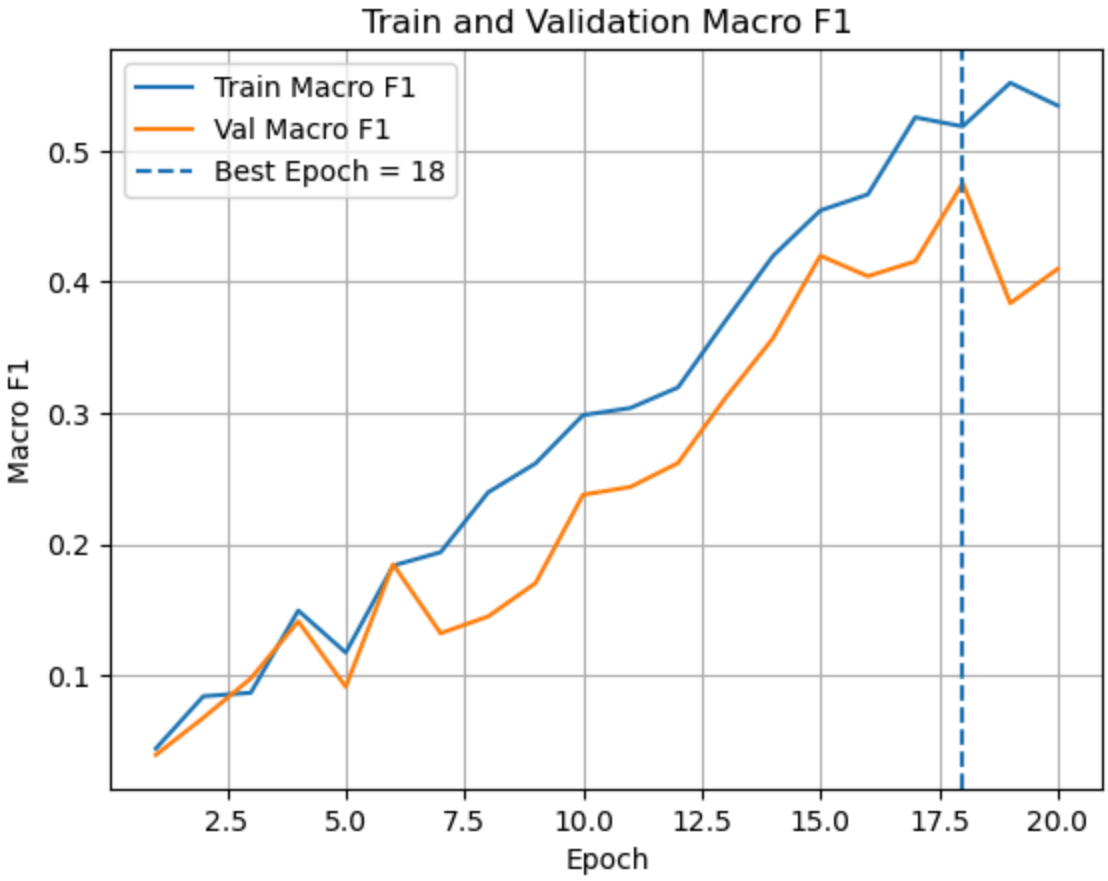

Train and Validation Macro F1
Train Macro F1
Val Macro F1
Best Epoch = 19
Macro F1
Epoch
0.0
0.1
0.2
0.3
0.4
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

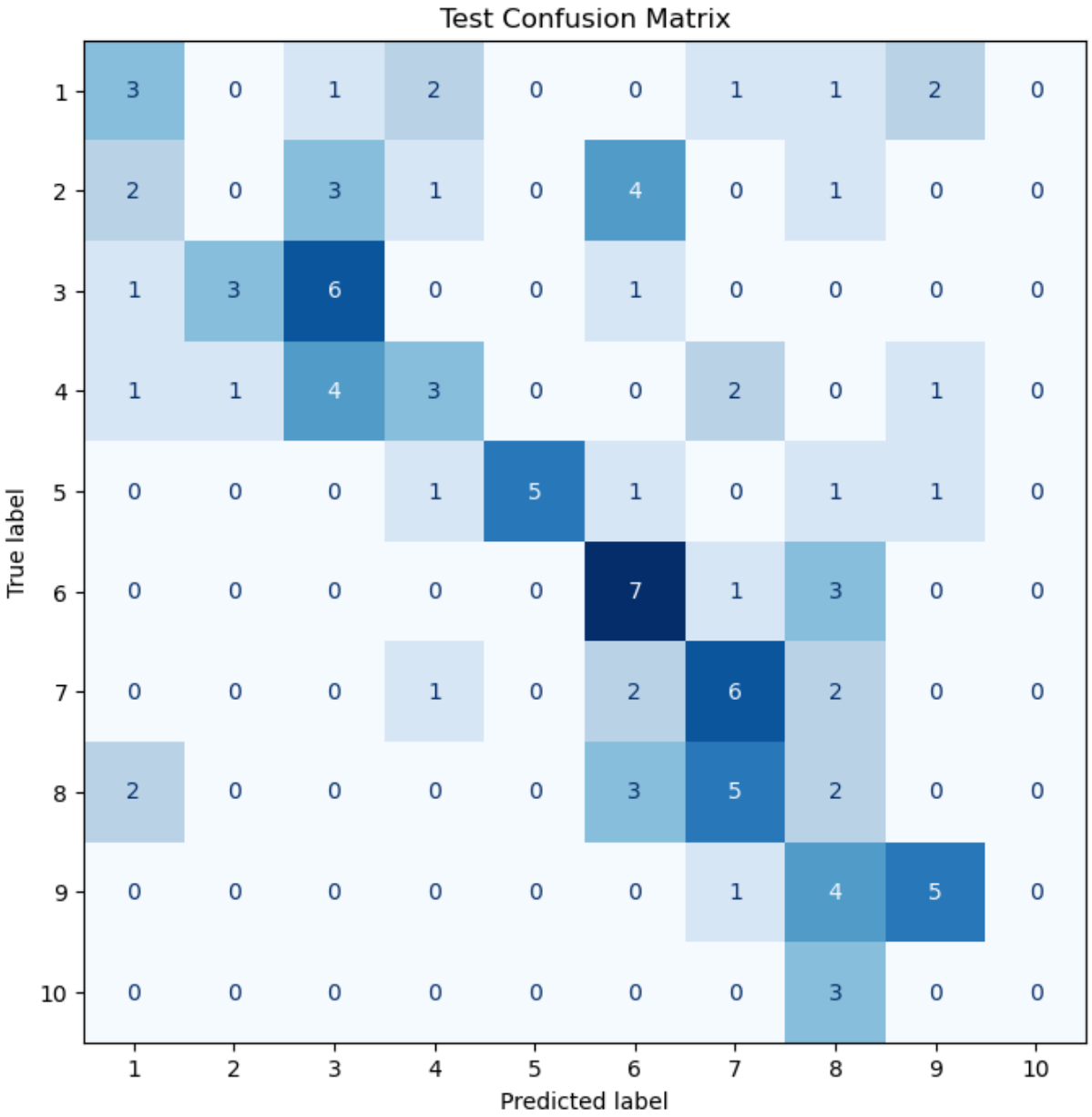



```
Best Epoch (by Val Macro F1): 19
Train Loss: 1.2456
Train Accuracy: 0.5312
Train Macro F1: 0.4838
Val Loss: 2.1219
Val Accuracy: 0.3438
Val Macro F1: 0.2992
Test Loss: 2.2209
Test Accuracy: 0.2500
Test Macro F1: 0.2059
Overall Percentage Correct: 25.00%
```

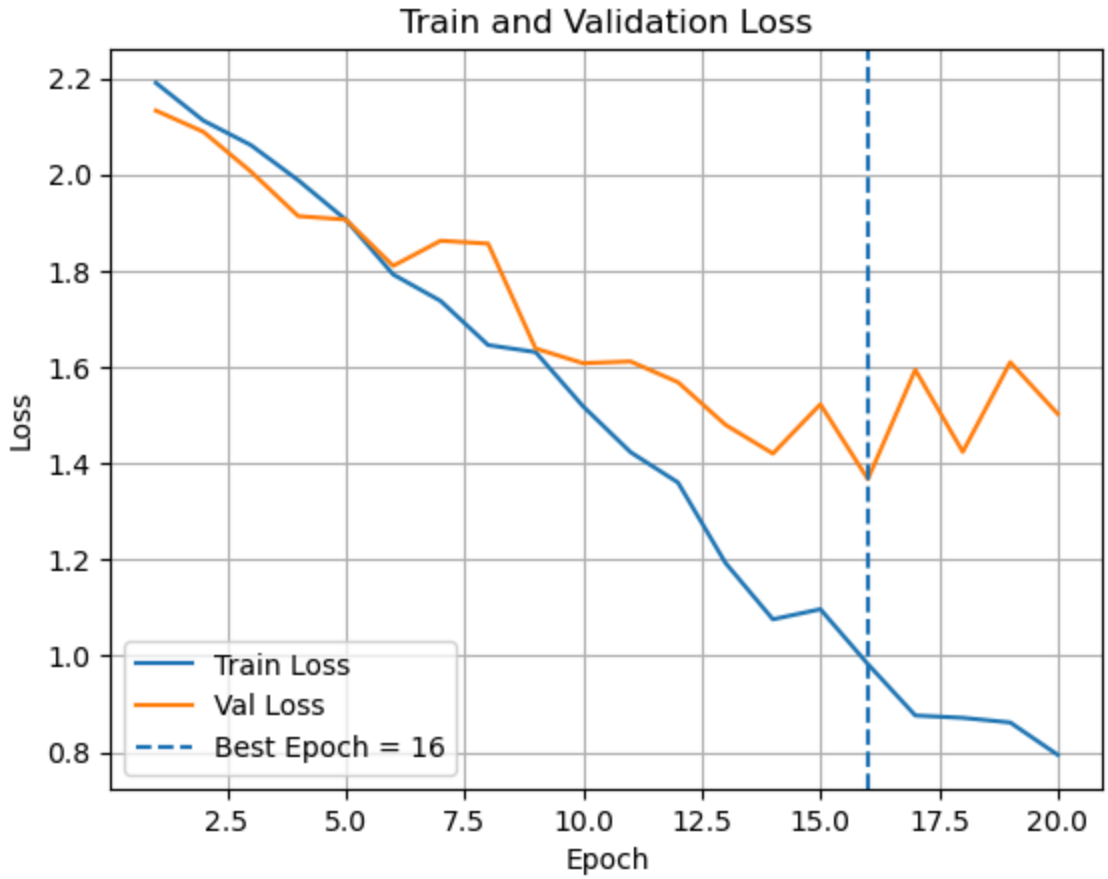

Train and Validation Loss
Loss
Epoch
Train Loss
Val Loss
Best Epoch = 18
2.2
2.0
1.8
1.6
1.4
1.2
1.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

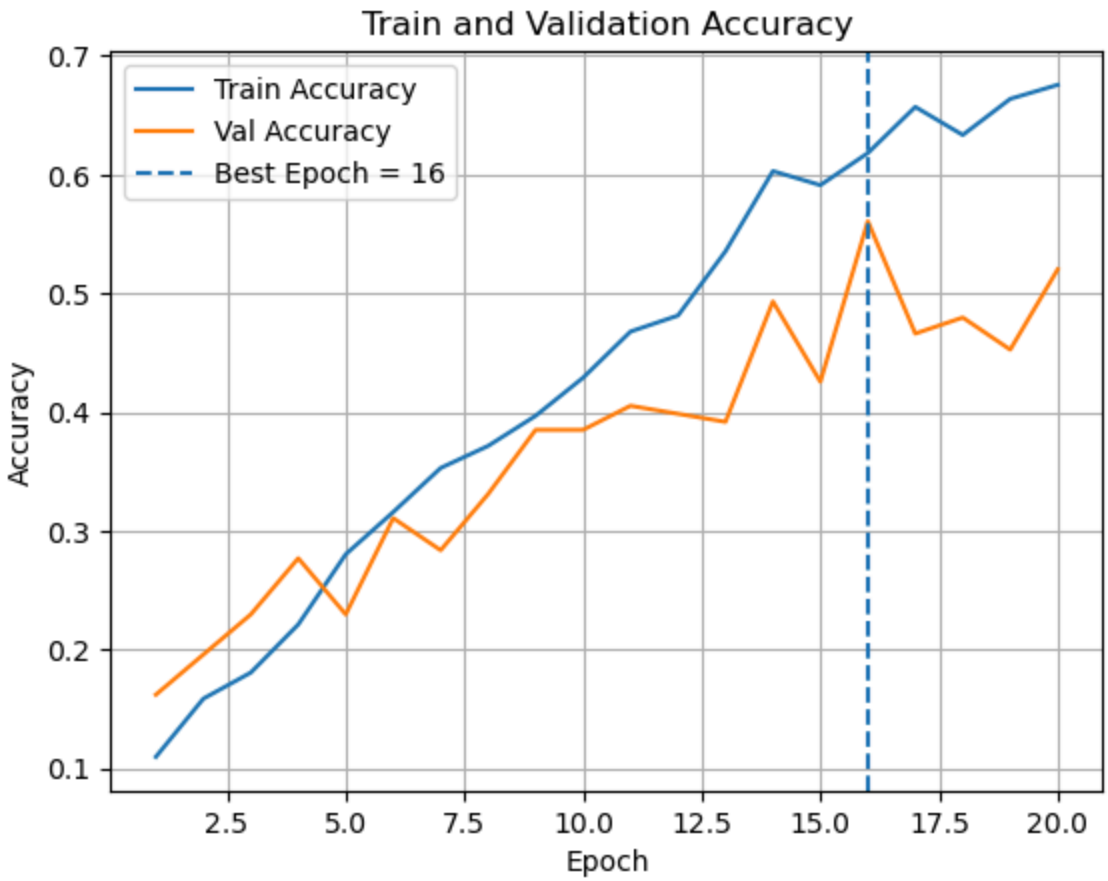

Train and Validation Accuracy
Accuracy
Epoch
Train Accuracy
Val Accuracy
Best Epoch = 18
0.6
0.5
0.4
0.3
0.2
0.1
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

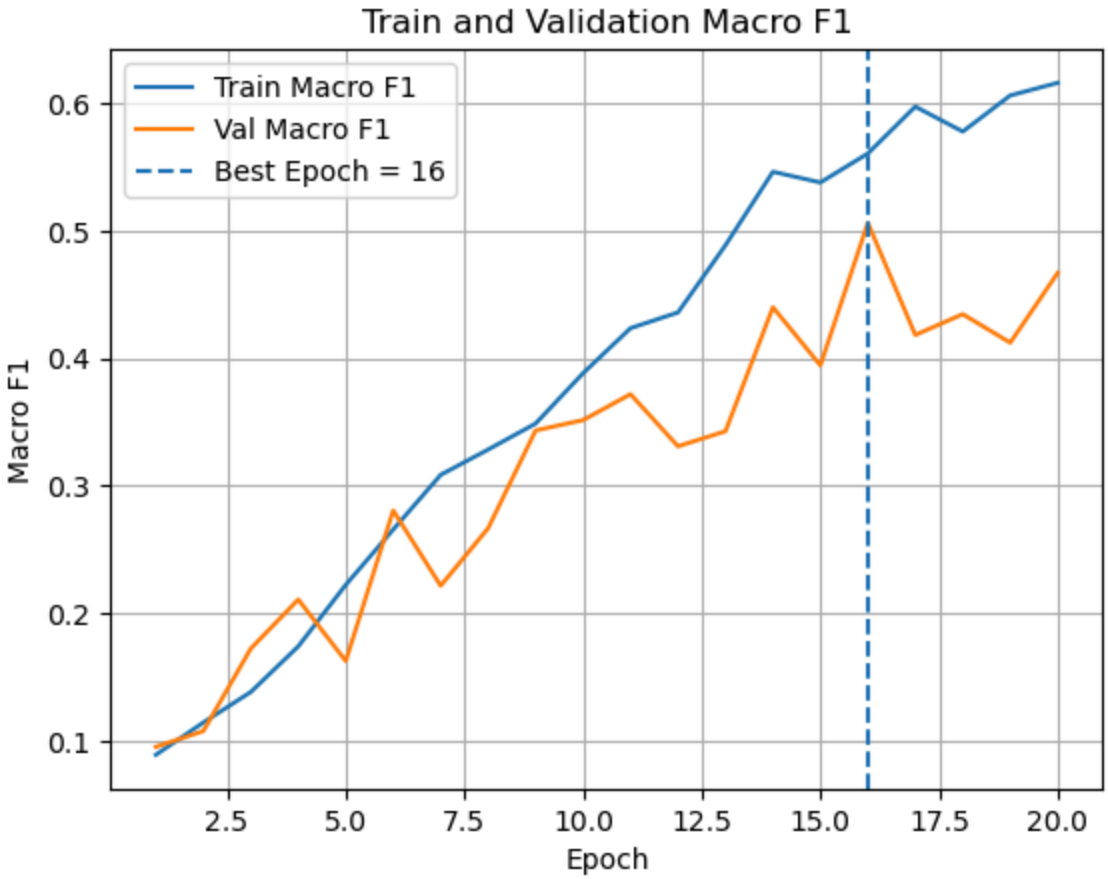

Train and Validation Macro F1
Train Macro F1
Val Macro F1
Best Epoch = 18
Macro F1
Epoch
0.1
0.2
0.3
0.4
0.5
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

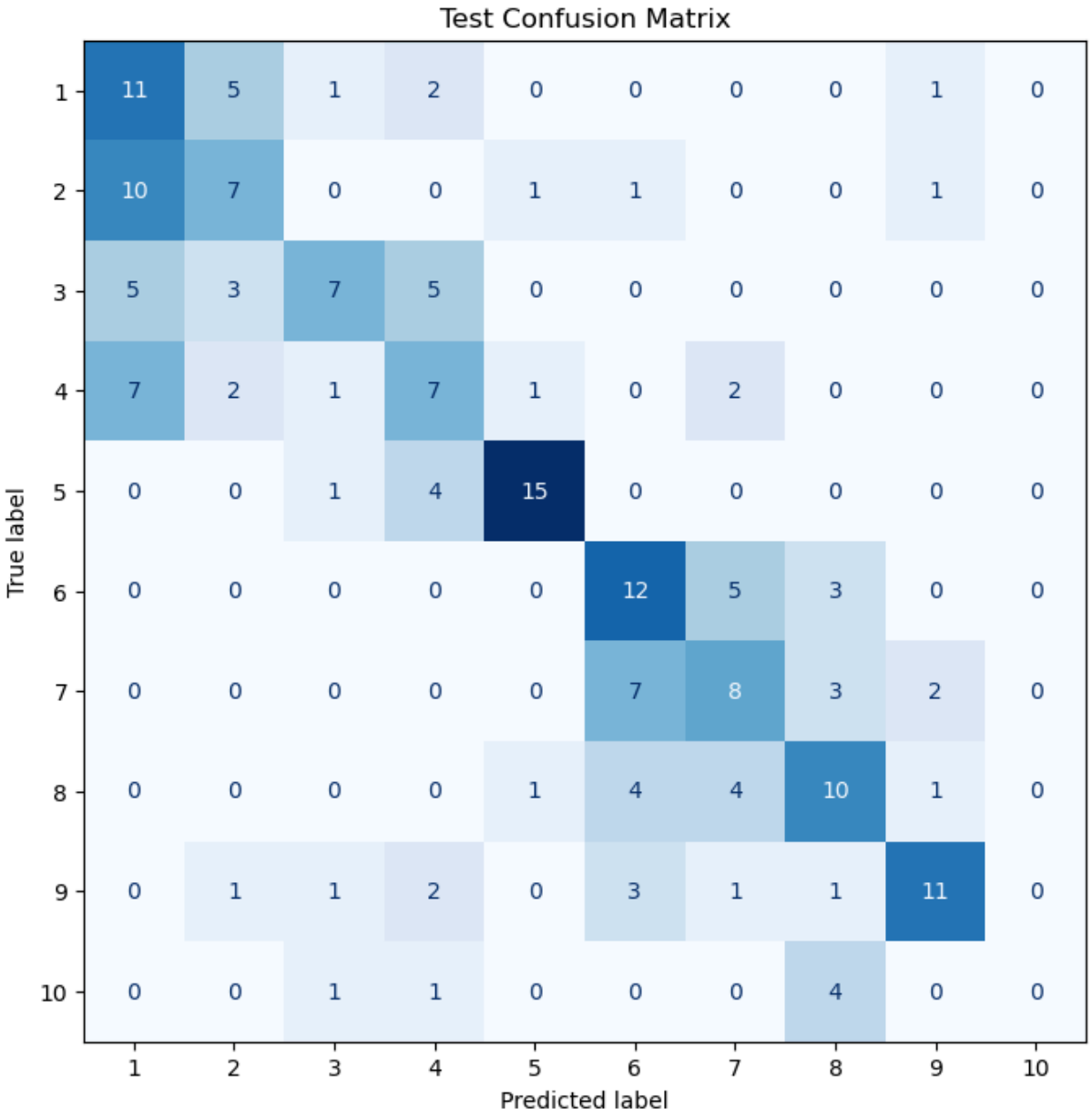



```
Best Epoch (by Val Macro F1): 18
Train Loss: 0.8761
Train Accuracy: 0.6625
Train Macro F1: 0.6031
Val Loss: 1.5017
Val Accuracy: 0.5125
Val Macro F1: 0.4752
Test Loss: 1.7447
Test Accuracy: 0.3700
Test Macro F1: 0.3402
Overall Percentage Correct: 37.00%
```


Train and Validation Loss
Loss
Epoch
Train Loss
Val Loss
Best Epoch = 16
2.2
2.0
1.8
1.6
1.4
1.2
1.0
0.8
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Train and Validation Accuracy
Accuracy
Epoch
Train Accuracy
Val Accuracy
Best Epoch = 16
0.7
0.6
0.5
0.4
0.3
0.2
0.1
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

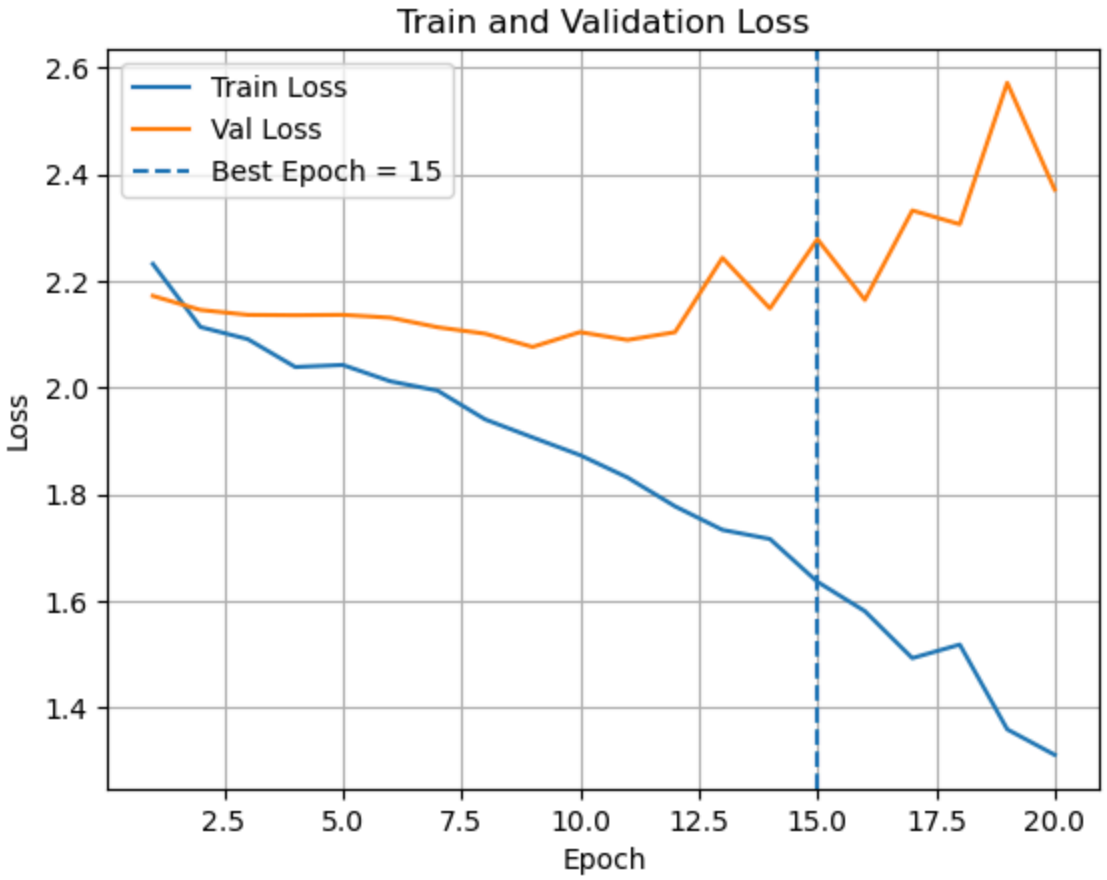

Train and Validation Macro F1
Train Macro F1
Val Macro F1
Best Epoch = 16
Macro F1
Epoch
0.1
0.2
0.3
0.4
0.5
0.6
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

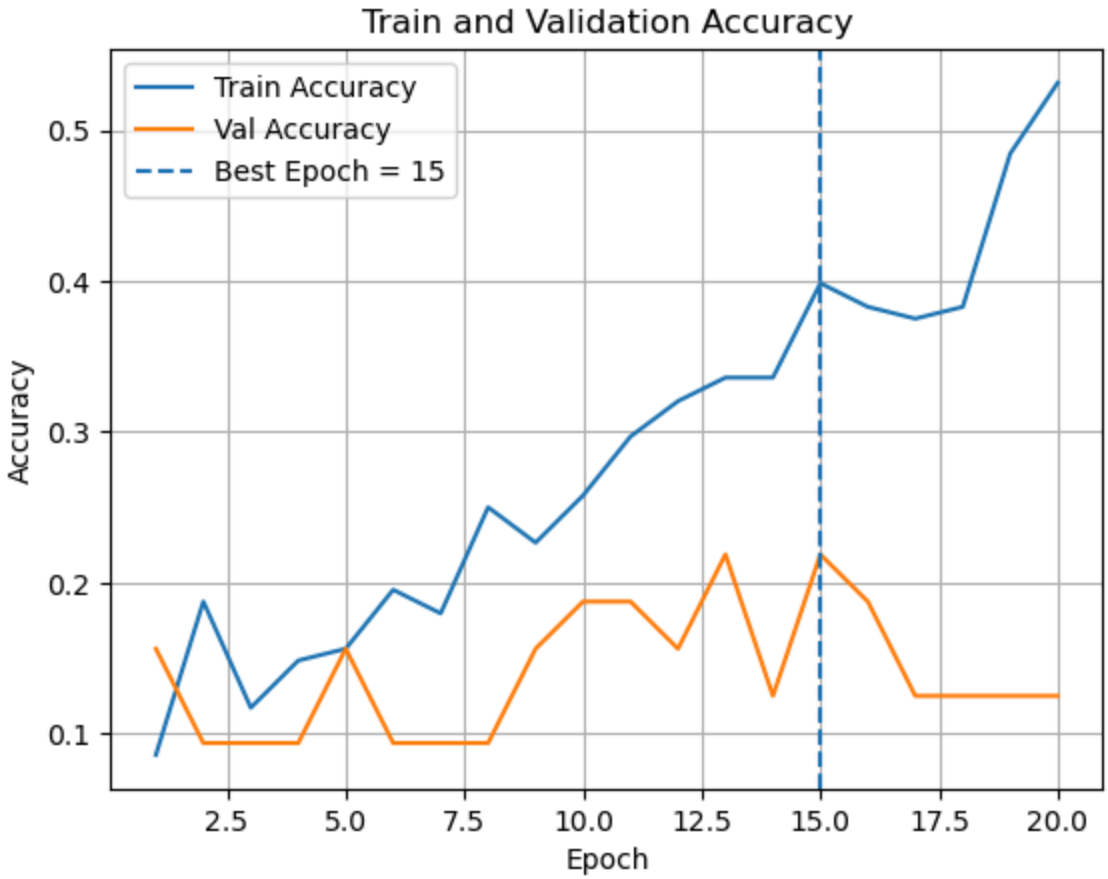


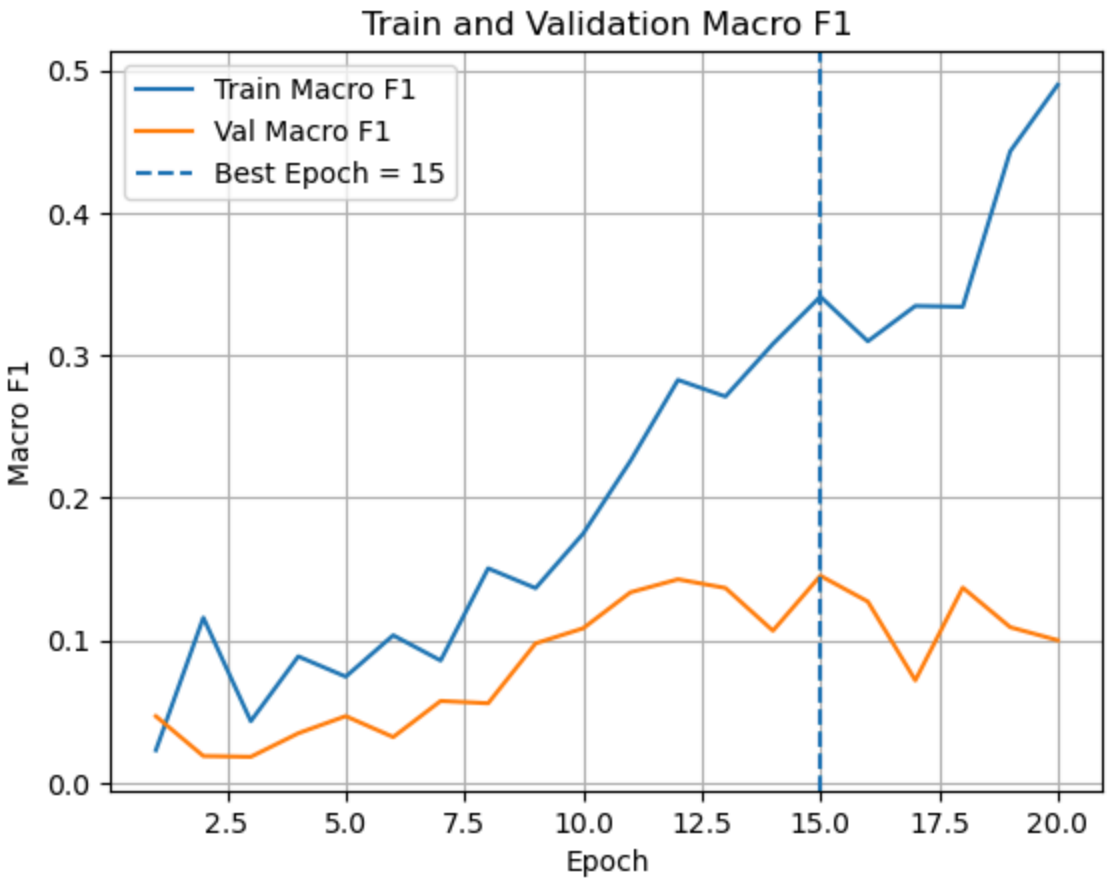

```
Best Epoch (by Val Macro F1): 16
Train Loss: 0.8142
Train Accuracy: 0.6622
Train Macro F1: 0.5999
Val Loss: 1.3669
Val Accuracy: 0.5608
Val Macro F1: 0.5060
Test Loss: 1.3814
Test Accuracy: 0.4731
Test Macro F1: 0.4362
Overall Percentage Correct: 47.31%
```

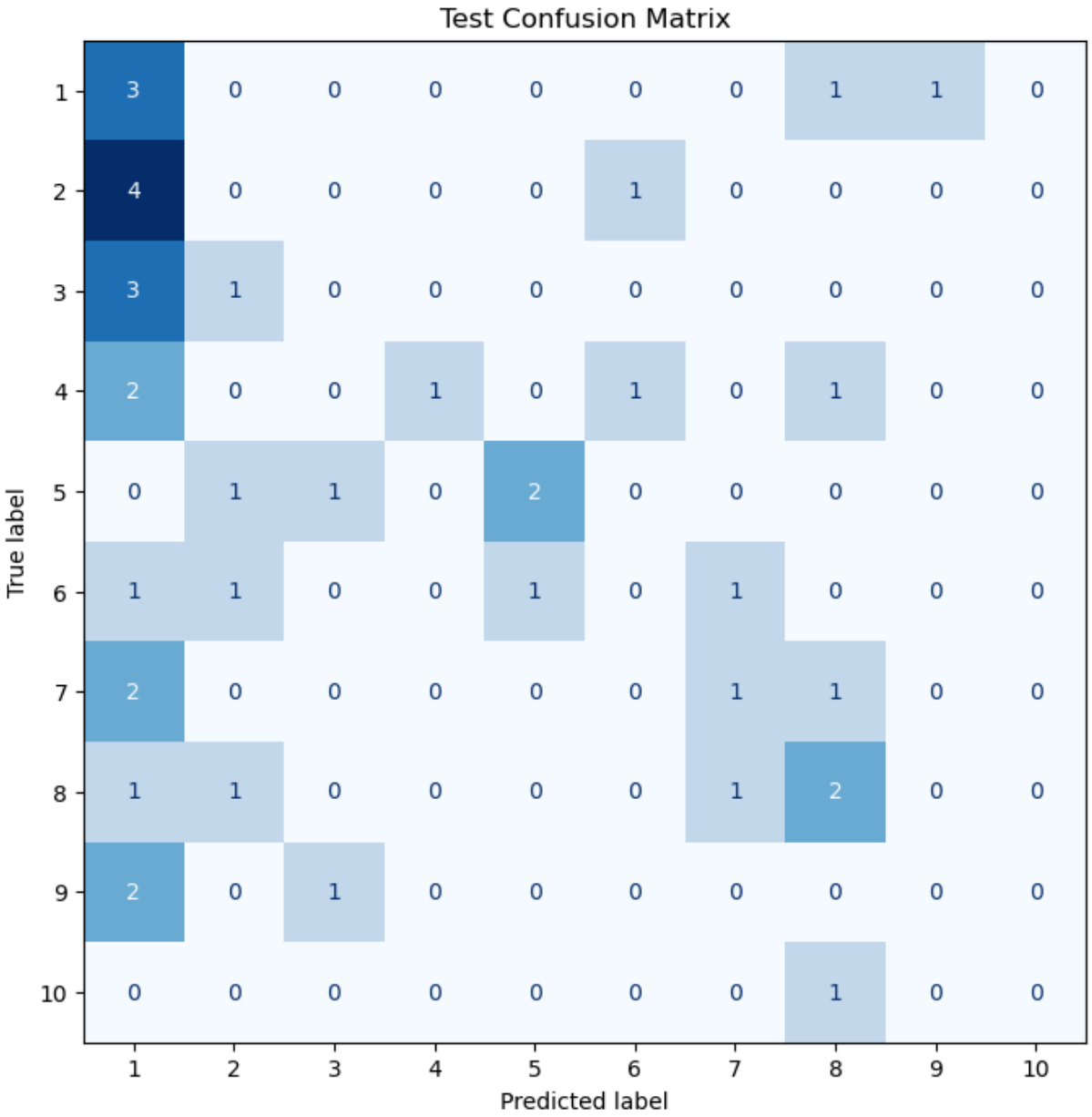
In [17]:
```
for sample in [200, 500, 1000]:
    results = train_and_test_reaction_transformer(
        df=subset[:sample],
        test_size=0.2,
        val_size=0.2,
        batch_size=32,
        epochs=20,
```

```
        d_model=128,
        nhead=4,
        num_layers=2,
        dim_feedforward=256,
        dropout=0.1,
        max_len=512,
        gnn_dim=9,
        include_gnn_vec=True, # W/O GNN VEC
        lr=1e-3,
        device=device,
        random_state=67,
        show_progress= False, # used when stress testing, eyes will bleed
        verbose=True,
        make_plots=True,
    )
```

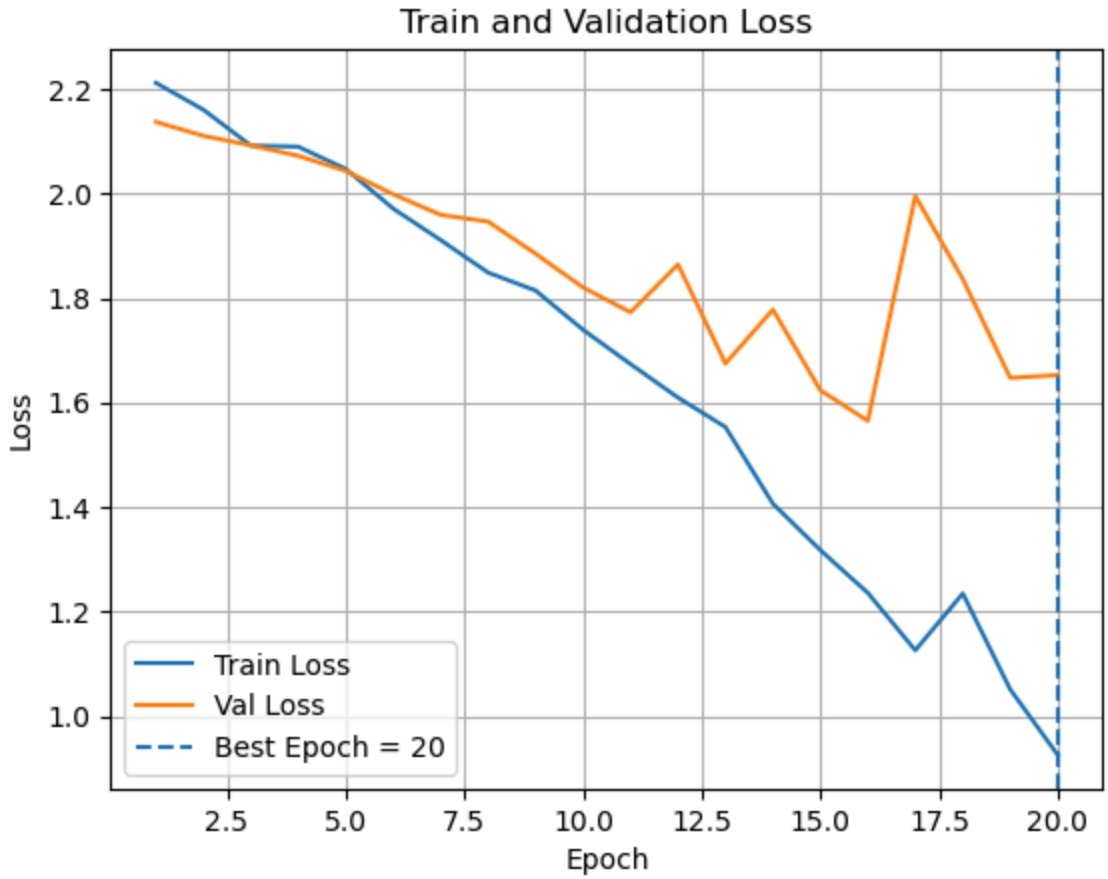


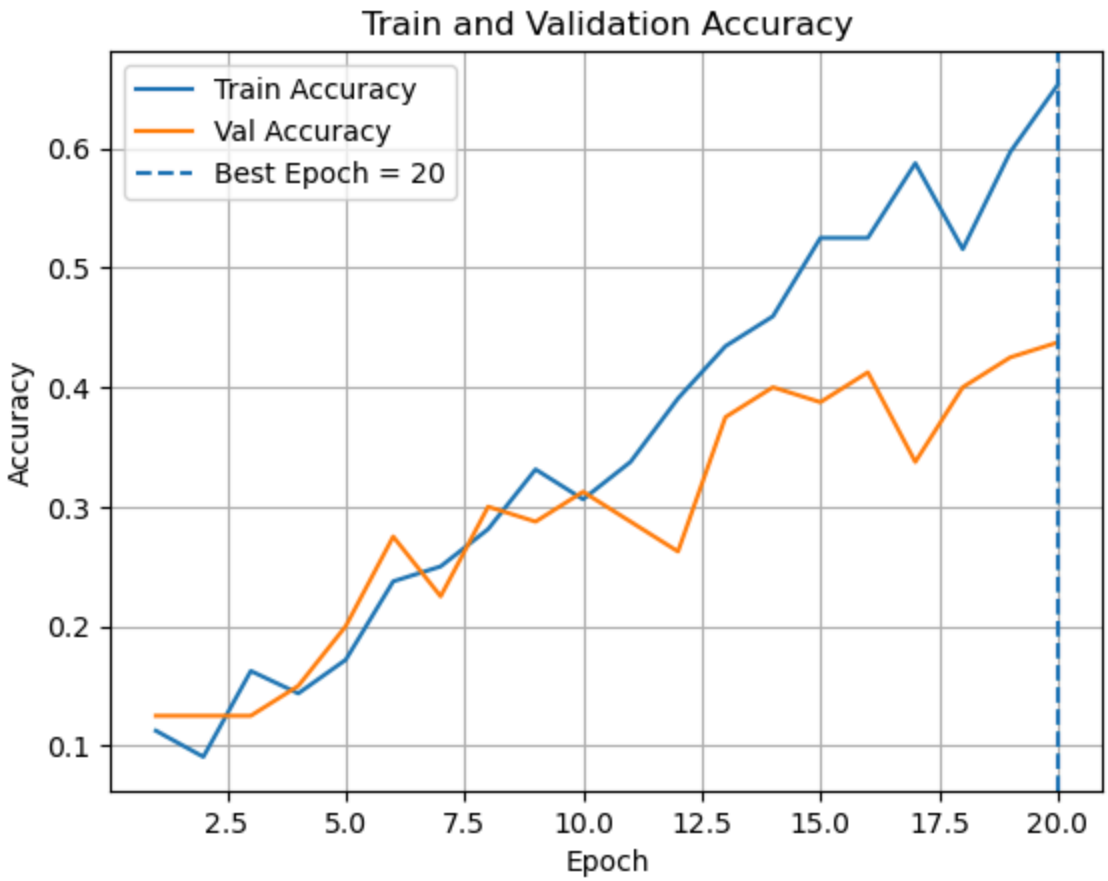

Train and Validation Accuracy
Train Accuracy
Val Accuracy
Best Epoch = 15
Accuracy
Epoch
0.1
0.2
0.3
0.4
0.5
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

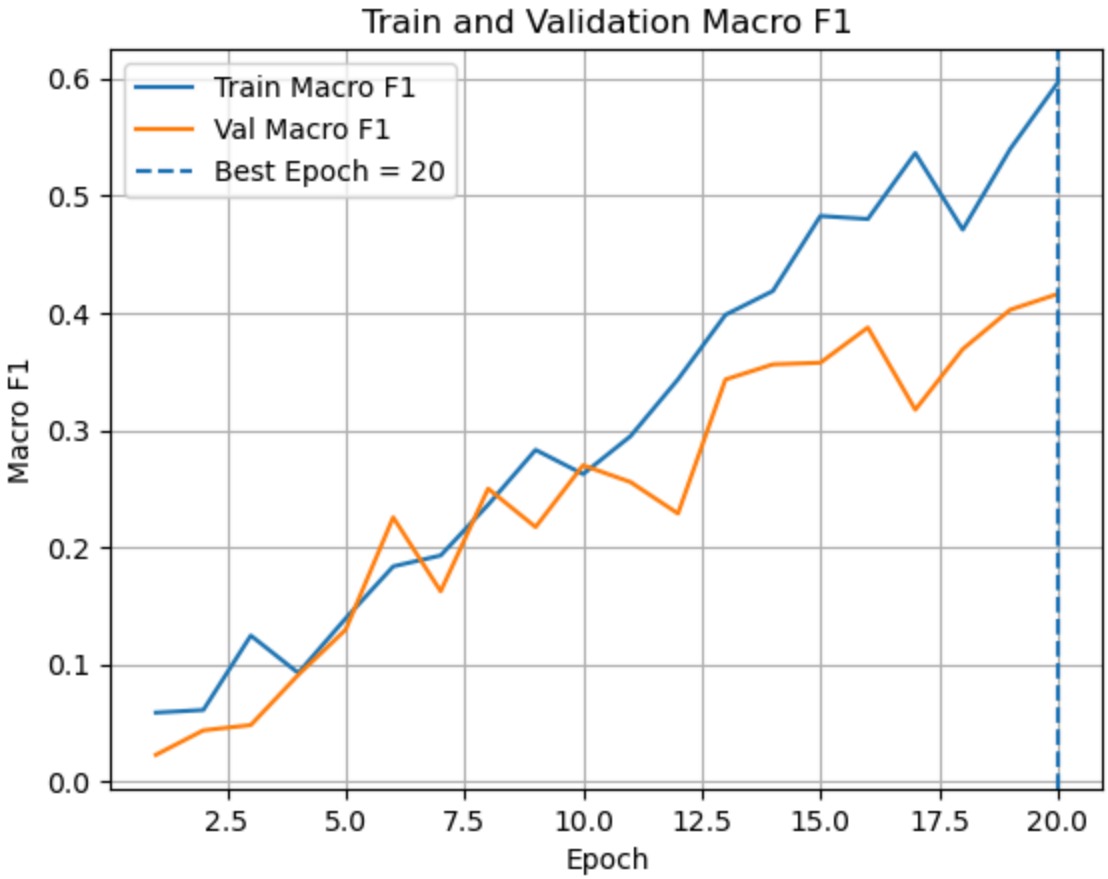

Train and Validation Macro F1
Train Macro F1
Val Macro F1
Best Epoch = 15
Macro F1
Epoch
0.0
0.1
0.2
0.3
0.4
0.5
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

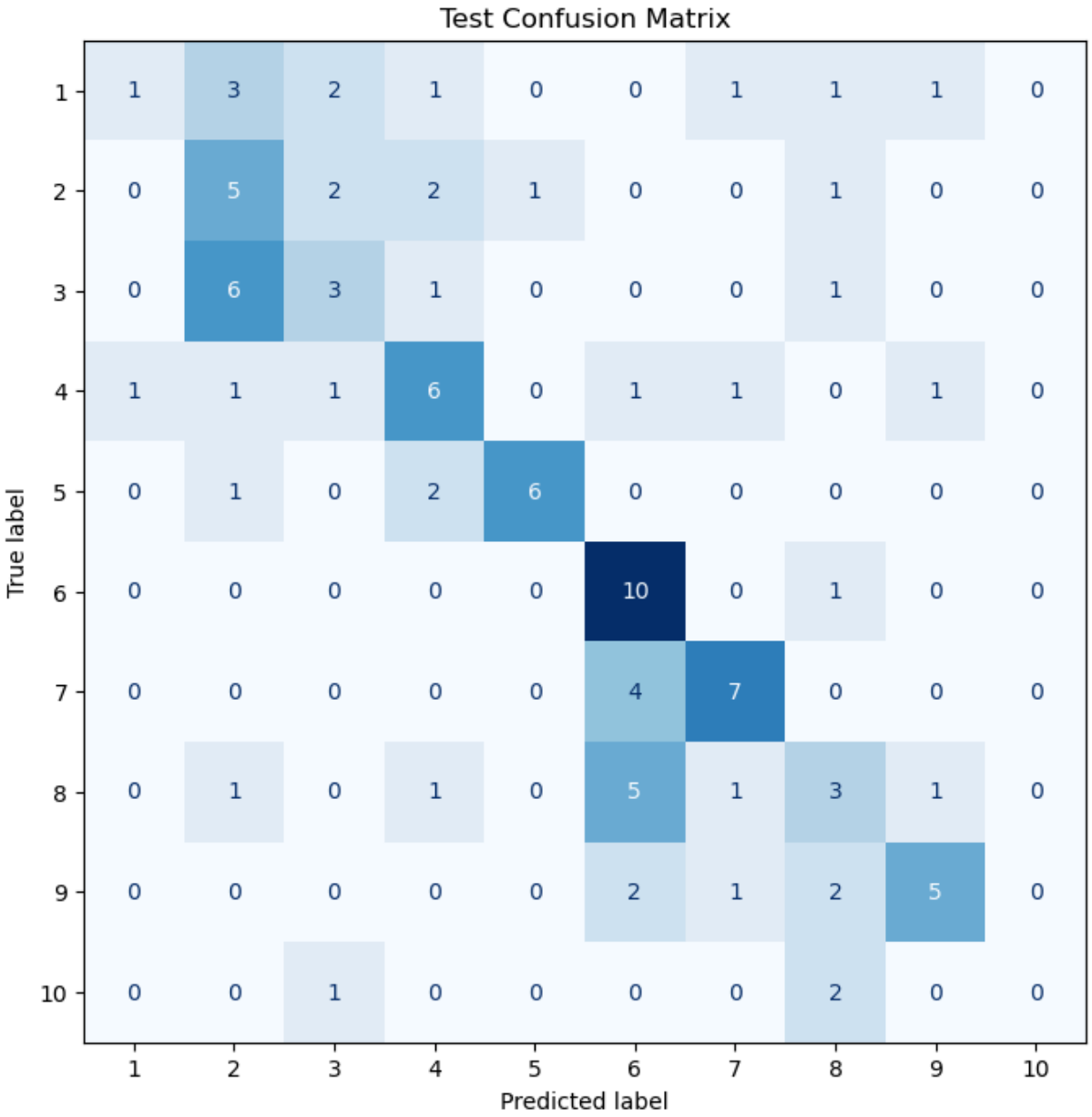



```
Best Epoch (by Val Macro F1): 15
Train Loss: 1.5602
Train Accuracy: 0.3750
Train Macro F1: 0.3290
Val Loss: 2.2786
Val Accuracy: 0.2188
Val Macro F1: 0.1452
Test Loss: 2.0301
Test Accuracy: 0.2250
Test Macro F1: 0.1815
Overall Percentage Correct: 22.50%
```

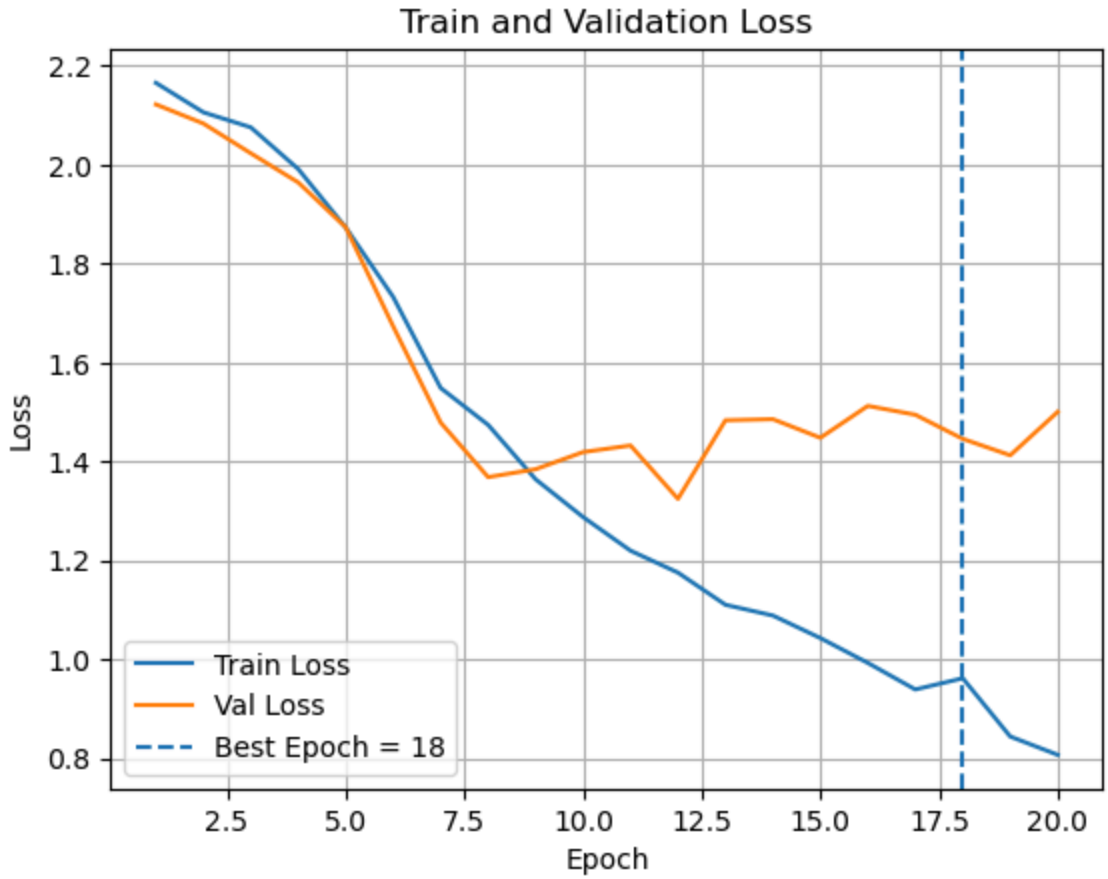

Train and Validation Loss
Loss
Epoch
Train Loss
Val Loss
Best Epoch = 20
2.2
2.0
1.8
1.6
1.4
1.2
1.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

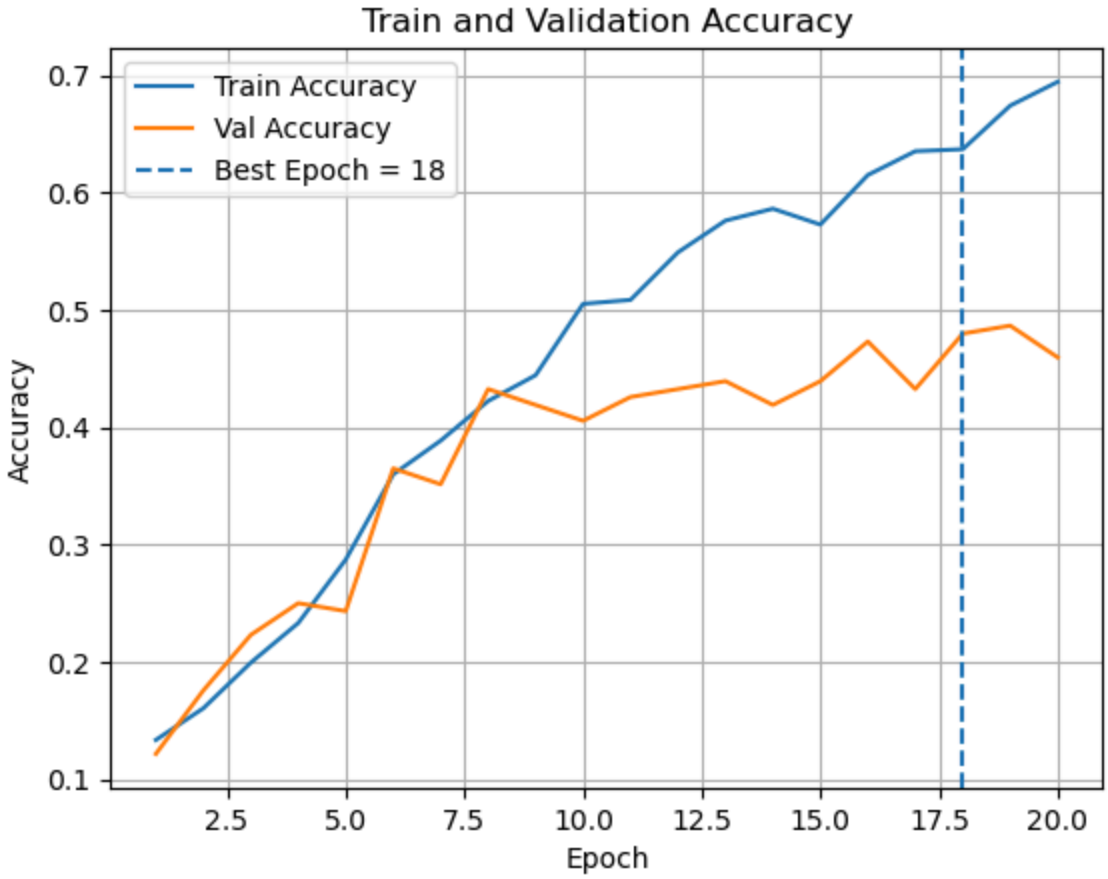

Train and Validation Accuracy
Accuracy
Epoch
Train Accuracy
Val Accuracy
Best Epoch = 20
0.6
0.5
0.4
0.3
0.2
0.1
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

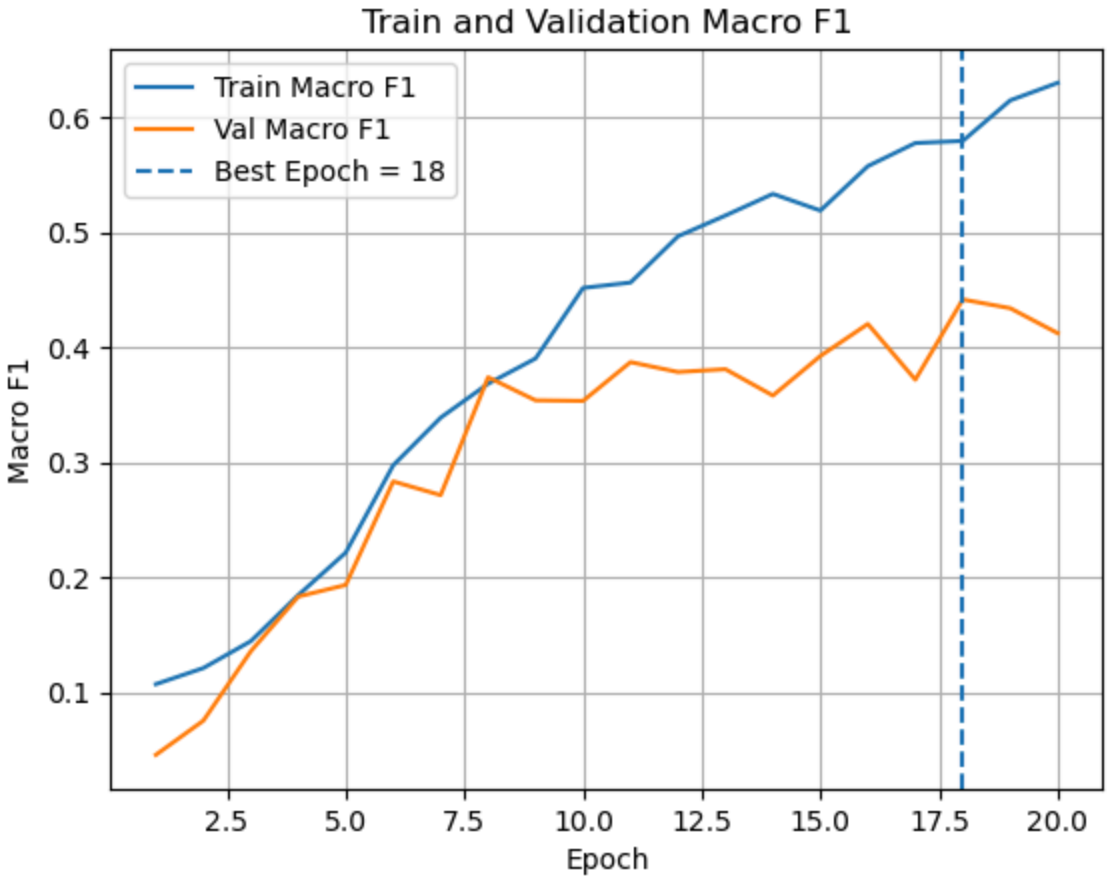

Train and Validation Macro F1
Train Macro F1
Val Macro F1
Best Epoch = 20
Macro F1
Epoch
0.0
0.1
0.2
0.3
0.4
0.5
0.6
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

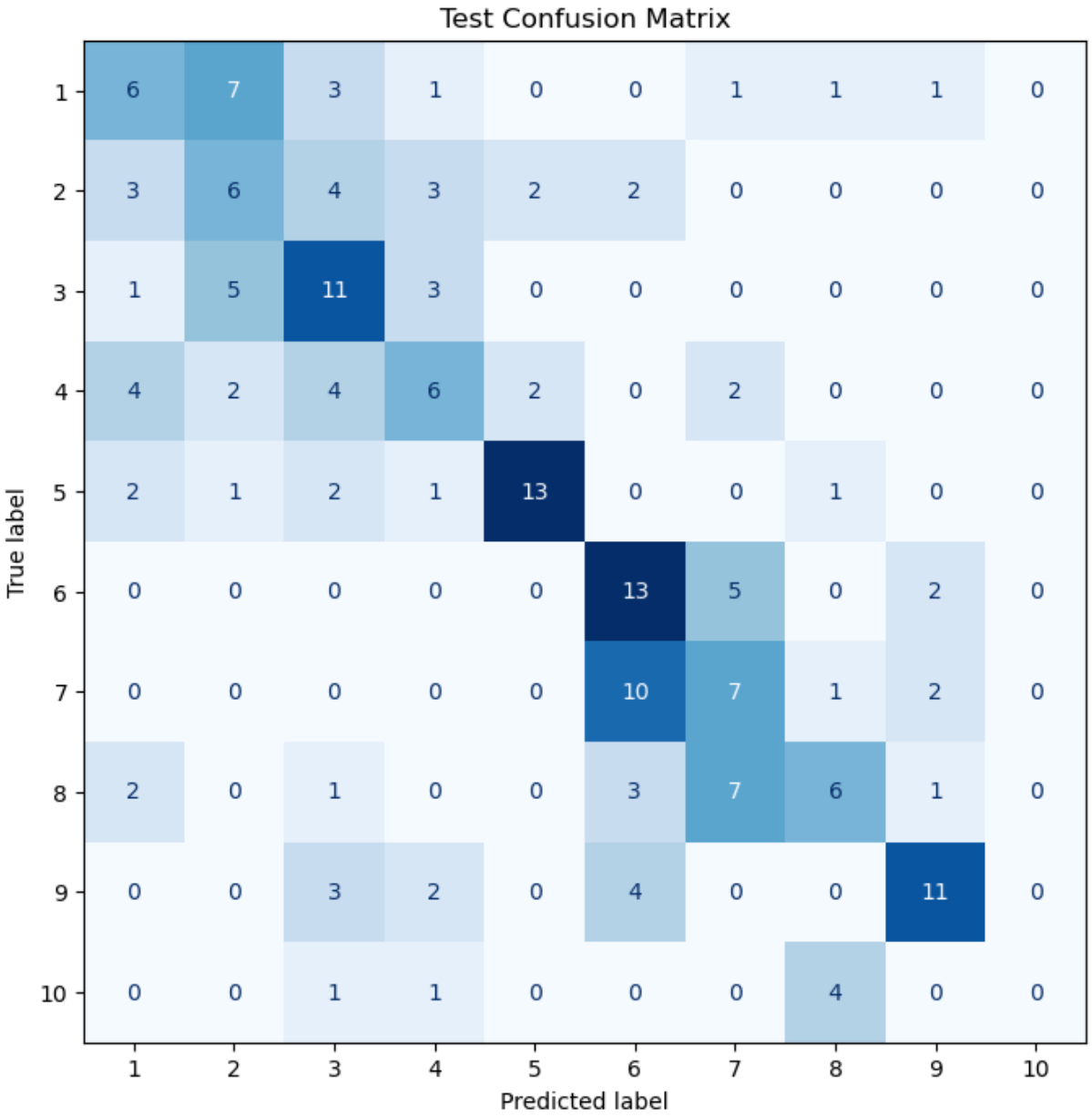



```
Best Epoch (by Val Macro F1): 20
Train Loss: 0.7602
Train Accuracy: 0.7188
Train Macro F1: 0.6529
Val Loss: 1.6525
Val Accuracy: 0.4375
Val Macro F1: 0.4160
Test Loss: 1.7356
Test Accuracy: 0.4600
Test Macro F1: 0.4113
Overall Percentage Correct: 46.00%
```


Train and Validation Loss
Loss
Epoch
Train Loss
Val Loss
Best Epoch = 18
2.2
2.0
1.8
1.6
1.4
1.2
1.0
0.8
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Train and Validation Accuracy
Accuracy
Epoch
Train Accuracy
Val Accuracy
Best Epoch = 18
0.7
0.6
0.5
0.4
0.3
0.2
0.1
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0


Train and Validation Macro F1
Train Macro F1
Val Macro F1
Best Epoch = 18
Macro F1
Epoch
0.1
0.2
0.3
0.4
0.5
0.6
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0




```
Best Epoch (by Val Macro F1): 18
Train Loss: 0.7102
Train Accuracy: 0.7230
Train Macro F1: 0.6584
Val Loss: 1.4458
Val Accuracy: 0.4797
Val Macro F1: 0.4417
Test Loss: 1.4863
Test Accuracy: 0.4247
Test Macro F1: 0.3876
Overall Percentage Correct: 42.47%
```

Endgame

\boldUsing the whole alchemy and uspto dataset, run the transformer by itself to see if chemical reactions are well-predicted. Then, run the GNN, couple it to the transfomer, and see if reactions are even more greatly correctly predicted!

In [18]:
```
# # run the final model
#
# results = train_and_test(
#     model_class=GNN,
#     graph_dataset=graph_dataset,
#     test_size=0.1,
#     batch_size=112,
#     epochs=70,
#     hidden_dim=124,
#     dropout=0.225,
#     lr=6e-4,
#     loss_type="smoothl1",
#     beta=0.6,
#     device=device,
#     random_state=67,
#     show_progress=False,
#     verbose=True,
#     make_plots=True
# )
```

In [ ]:
```
# # this cell will take around an hour to run on mps single core
# # run this at the end of the project to get predictions
# reaction_types_df["reactant_preds"] = [
#     get_structure_preds(results["model"], x, device)
#     for x in tqdm(reaction_types_df["reactant"])
# ]
#
# reaction_types_df["product_preds"] = [
#     get_structure_preds(results["model"], x, device)
#     for x in tqdm(reaction_types_df["product"])
# ]
#
# display(reaction_types_df)
```

In [ ]:
```
# transfomer with gnn coupling
# smiles_results = train_and_test_reaction_transformer(
#     df=reaction_types_df,
#     include_gnn_vec=False,
#     device=device,
#     epochs=20,
#     batch_size=32,
#     show_progress=False,
#     verbose=True,
#     make_plots=True,
# )
```

In [ ]:
```
# transfomer w/o gnn coupling
# gnn_augmented_results = train_and_test_reaction_transformer(
#     df=reaction_types_df,
#     include_gnn_vec=True,
#     device=device,
#     epochs=20,
#     batch_size=32,
#     show_progress=False,
#     verbose=True,
#     make_plots=True,
# )
```

In [ ]:
```
# then a final analysis if the gnn was at all useful
```